LLD

LOW-LEVEL DESIGN IN GO

LLD in Go

> ఇంటర్‌ఫేసులు, composition, patterns – Go యొక్క సొంత శైలిలో. Go lo inheritance లేదు, అదే దీన్ని ఆసక్తికరం చేస్తుంది.

భావనలకి `LLD_Telugu.pdf` · interview problems కి `LLD_Design_Problems_Telugu.pdf` చూడండి. ఇది వాటి *Go అమలు.*

తెలుగు + ENGLISH

YASWANTH · REFERENCE

విషయ సూచిక (Table of Contents)

Part 1 – Go LLD పునాదులు (Foundations)

1. LLD అంటే ఏమిటి, Go దృష్టితో (HLD vs LLD, "class లేదు" mindset shift)
2. Go's design model – struct + interface + composition; no inheritance; implicit interfaces (duck typing)
3. Encapsulation in Go – packages, exported/unexported, `private` keyword లేదు
4. Composition over Inheritance – embedding deep, has-a vs is-a
5. Program to Interface – accept interfaces, return structs; small interfaces (io.Reader/Writer)
6. Class relationships in Go – association / aggregation / composition via fields & embedding
7. UML → Go mapping (class diagram చదివి Go రాయడం)

Part 2 – SOLID & Principles in Go

8. SOLID in Go – SRP, OCP, LSP, ISP, DIP (before/after Go code)
9. DRY, KISS, YAGNI in Go
10. Law of Demeter, Encapsulate what varies, Favor immutability (value semantics)

Part 3 – Creational Patterns (Go idiomatic)

11. Singleton (`sync.Once` , package-level var, `init()`)
12. Factory / Factory Method (`NewX` constructor funcs, interfaces return)
13. Abstract Factory
14. Builder – మరియు idiomatic **Functional Options Pattern**
15. Prototype (copy semantics, deep vs shallow)
16. Object Pool (`sync.Pool`)

Part 4 – Structural Patterns

17. Adapter
18. Decorator (`io.Reader/Writer` wrapping, http middleware)
19. Proxy (lazy, protection, remote)
20. Facade
21. Composite
22. Bridge
23. Flyweight

Part 5 – Behavioral Patterns

24. Strategy (func types – idiomatic)
25. Observer (channels & callbacks)
26. Command

27. State (state machine)
28. Template Method (embedding + interface)
29. Iterator (channels, Go 1.23 range-over-func)
30. Chain of Responsibility (middleware chains)
31. Mediator
32. Memento
33. Visitor (Go లో ఎందుకు awkward)
34. Interpreter

Part 6 – Go-specific Design

35. Functional Options Pattern (full deep dive)
36. Dependency Injection in Go (manual, wire, fx overview)
37. Concurrency design patterns as LLD building blocks (worker pool, pub-sub, pipeline)
38. Error-handling design (sentinel vs wrapped vs typed, error as value)

Part 7 – LLD Case Studies in Go

39. Design a Parking Lot (structs, interfaces, strategy pricing)
40. Design an LRU Cache (map + container/list, mutex)
41. Design a Rate Limiter (token bucket, sliding window)
42. Design a Thread-safe Logger (levels, async via channel)
43. Design a Pub/Sub / Notification system (channels, observer)
44. Design a Vending Machine (State pattern)
45. LLD Interview approach in Go + checklist + Memory Tips + Common Mistakes

---

Part 1 – Go LLD పునాదులు (Foundations)

> Patterns లోకి దూకే ముందు పునాది: Go లో "class లేదు" అంటే ఏమిటి, struct/interface/composition ఎలా పనిచేస్తాయి, encapsulation ఎలా, UML ని Go కి ఎలా translate చేయాలి. ఈ పునాది లేకుండా తర్వాత patterns "ఎందుకు ఇలా రాశారు?" అనిపిస్తాయి.

---

1. LLD అంటే ఏమిటి, Go దృష్టితో

వివరణ

LLD (Low-Level Design) = ఒక system ని **types, methods, interfaces, relationships** స్థాయిలో design చేయడం. "ఈ feature ని ఏ types తో, ఏ interfaces తో, ఎలా organize చేసి రాయాలి?" అనే ప్రశ్నకి సమాధానం.

- **HLD (High-Level Design)** = పెద్ద బొమ్మ. Services, databases, message queues, load balancers, API boundaries.
- **LLD (Low-Level Design)** = ఒక్క service/package లోపల **type-level blueprint** – Go లో అంటే packages, structs, interfaces, functions ఎలా arrange చేయాలి.

Go లో LLD కి ఒక **ప్రత్యేకమైన mindset shift** అవసరం. చాలామంది Java/C++ నుండి వచ్చి `class`, `extends`, `abstract class`, `implements` వెతుకుతారు – **Go లో ఇవి ఏవీ లేవు**. Go కి కేవలం:

- `struct` – data (fields) ని group చేయడానికి. Method లు struct కి attach చేయవచ్చు (`func (r Receiver) Method()`).
- `interface` – behavior (method set) ని describe చేయడానికి. **Implicit** – `implements` keyword అవసరం లేదు.
- `composition (embedding)` – inheritance కి బదులుగా. ఒక struct లో ఇంకో struct/interface ని embed చేయడం.

అందుకే Go LLD = **"ఏ classes రాయాలి?" కాదు – "ఏ small interfaces + concrete structs రాయాలి, వాటిని ఎలా compose చేయాలి?"**

Real-life Scenario

> **ఇల్లు కట్టడం** analogy:
>
> - **HLD** = ఎన్ని గదులు, ఎక్కడ kitchen, ఎక్కడ borewell – master plan.
> - **LLD** = ఒక్క గోడకి ఏ ఇటుకలు, switch board ఎక్కడ, wiring ఎలా – detailed engineering drawing.
>
> ఇప్పుడు **Go twist**: Java లో "గది" అంటే ఒక ready-made prefab room (class) – పైన ఇంకో అంతస్తు (subclass) కట్టొచ్చు. Go లో ready-made rooms లేవు – నీ దగ్గర **ఇటుకలు (structs)** మరియు **"ఈ గదికి ఏ ఏ తలుపులు ఉండాలి" అనే specification (interface)** మాత్రమే ఉంటాయి. గదులని ఒకదానిపై ఒకటి పేర్చవు (no inheritance) – పక్కపక్కన కలిపి (composition) పెద్ద ఇల్లు కడతావు.

HLD vs LLD (Go context)

అంశం	HLD	LLD (Go)
Level	System / Architecture	Package / type / interface

అంశం	HLD	LLD (Go)
ప్రశ్న	ఏ services అవసరం?	ఆ service లోపల ఏ structs/interfaces?
Output	Architecture diagram	Package layout, type definitions, code
Go ఉదా	"Order service + Kafka + Postgres"	"`OrderService` struct, `PaymentGateway` interface, `Strategy` for pricing"
Audience	Architects	Developers

Java mindset vs Go mindset (ముఖ్యమైన table)

నువ్వు Java లో అనుకునేది	Go లో అసలు ఏమి ఉంటుంది
`class Dog { ... }`	`type Dog struct { ... }` + methods
`class Dog extends Animal`	**లేదు** – `type Dog struct { Animal }` (embed)
`implements Runnable`	**Implicit** – method set match అయితే చాలు
`abstract class Shape`	`type Shape interface { Area() float64 }`
`private int x`	`x int` (lowercase = unexported)
`public int X`	`X int` (Uppercase = exported)
`new Dog()`	`&Dog{}` లేదా `NewDog()` constructor func
`Dog.class` reflection	`reflect` package
Constructor overloading	**లేదు** – వేరే `NewX` funcs / functional options

LLD ఎందుకు ముఖ్యం (Go లో ప్రత్యేకంగా)

1. **Idiomatic Go** = maintainable Go. Go community "clever" code ని ఇష్టపడదు – small, boring, composable types ని ఇష్టపడుతుంది.
2. **Interfaces చిన్నగా ఉంటే** (ISP), testing/mocking సులభం – Go లో test doubles రాయడానికి frameworks అవసరం లేదు.
3. **Composition** వల్ల class hierarchy explosion ఉండదు – కొత్త behavior కలపడం సులభం.
4. **Concurrency** Go కి first-class – LLD లో goroutine/channel design decisions ముఖ్యం (worker pool, pub-sub).
5. **Interviews** – Go teams "నువ్వు Java ని Go లో రాస్తున్నావా, లేక idiomatic Go రాస్తున్నావా?" అని చూస్తాయి.

ఎప్పుడు వాడాలి / వద్దు

- **వాడాలి:** ఏ nontrivial feature/service అయినా – ముందు types + interfaces గీసుకో, తర్వాత code.
- **వద్దు (over-design):** చిన్న script కి interfaces, factories అవసరం లేదు. Go motto – **"ముందు concrete struct రాయి, అవసరం వచ్చినప్పుడు interface extract చేయి."** (premature abstraction

anti-pattern).

Gotchas

- **Java patterns ని blindly port చేయడం** = అతిపెద్ద Go LLD తప్పు. AbstractSingletonProxyFactoryBean లాంటివి Go లో వద్దు.
- Go లో **interface ముందే రాయకు** – concrete type రాసి, consumer వైపు నుండి interface define చెయ్యి ("interfaces belong to the consumer").
- `class = struct` అనుకుంటే సరిపోదు – Go లో behavior (interface) మరియు data (struct) **వేరు వేరు**. Java లో అవి ఒకే class లో కలిసి ఉంటాయి.

Key Points

- LLD = types/interfaces/relationships స్థాయి design. Go లో ఇది **struct + interface + composition** ఆట.
- Go లో `class`, `inheritance`, `extends`, `implements` keyword లు **లేవు** – ఇదే మొదటి mindset shift.
- "ఏ classes?" కాదు – **"ఏ small interfaces + concrete structs, ఎలా compose?"** అని ఆలోచించు.
- Idiomatic Go = small, boring, composable. Java patterns ని blindly copy చేయకు.

2. Go's Design Model – struct + interface + composition

GO యొక్క DESIGN MODEL · EMBEDDING + IMPLICIT INTERFACES

GO lo INHERITANCE లేదు – EMBEDDING ఉంది

ఇతర భాషల్లో (inheritance)	Go lo (embedding)
class Dog extends Animal Dog "ఒక" Animal – గట్టి బంధం Parent మారితే children విరుగుతాయి	type Dog struct { Animal } Dog కి Animal యొక్క methods వస్తాయి కానీ Dog ఒక Animal **కాదు** – కలిగి ఉంది

INTERFACE SATISFACTION – implicit

ఇతర భాషల్లో	Go lo
class Dog implements Speaker Explicit గా ప్రకటించాలి Interface ముందు ఉండాలి	Speak() method ఉంటే చాలు Dog కి Speaker గురించి తెలియనవసరం లేదు Interface ని **consumer** నిర్వచిస్తాడు

దీని పెద్ద పరిణామం

మీరు వేరే package నుంచి తెచ్చిన type కి కూడా – మీ interface ని సంతృప్తి పరచొచ్చు,
ఆ package ని ముట్టుకోకుండా. అందుకే Go lo adapter pattern చాలా అరుదుగా అవసరం.
నియమం: **"Accept interfaces, return structs"**.

వివరణ

Go మొత్తం design ఈ **మూడు building blocks** మీద ఆధారపడి ఉంటుంది:

1. **struct** = data. Fields ని group చేస్తుంది. Methods ని attach చేయవచ్చు.
2. **interface** = behavior contract. **Method set** మాత్రమే – data లేదు.
3. **composition** = struct లో struct/interface embed చేయడం. Inheritance కి బదులు.

కీలకం: Go interfaces **implicit** (duck typing). Java లో `class Duck implements Quacker` అని రాయాలి. Go లో `Duck` కి `Quack()` method ఉంటే – అది **automatic** గా `Quacker` interface ని satisfy చేస్తుంది. ఎక్కడా `implements` రాయనవసరం లేదు. "If it quacks like a duck, it IS a Quacker."

ఇది చాలా powerful: నీ struct **ఏ interface ని satisfy చేస్తుందో** ముందే తెలియనవసరం లేదు. వేరే ఎవరో తర్వాత ఒక interface define చేస్తే, నీ existing struct దాన్ని free గా satisfy చేస్తుంది (retroactive).

Real-life Scenario

> **"పని చేసేవాడు" (Worker) interview**: నీకు ఒక పని కావాలి – "code రాయగలవా?" అని అడుగుతావు. అభ్యర్థి Java నుండి వచ్చాడా, Python నుండి వచ్చాడా – నీకు అనవసరం. **అతను code రాయగలిగితే** (behavior) అతను "Coder" – ఏ certificate (`implements`) చూపనవసరం లేదు. ఇదే **implicit interface**: method ఉంటే చాలు, declare చేయనవసరం లేదు.

Code

```
package main

import "fmt"

// interface = behavior contract (method set మాత్రమే, data లేదు)
type Speaker interface {
	Speak() string
}

// struct = data
type Dog struct {
	Name string
}

// method — Dog కి Speak() attach చేస్తున్నాం.
// (d Dog) = receiver. ఇది Dog ని Speaker interface satisfy చేస్తుంది —
// "implements Speaker" ఎక్కడా రాయలేదు! (implicit)
func (d Dog) Speak() string {
	return d.Name + " says Woof!"
}

type Cat struct {
	Name string
}

func (c Cat) Speak() string {
	return c.Name + " says Meow!"
}

// function interface ని accept చేస్తుంది — Dog నా Cat నా అనవసరం.
// polymorphism — inheritance లేకుండా!
func announce(s Speaker) {
	fmt.Println(s.Speak())
}

func main() {
	announce(Dog{Name: "Bruno"}) // Bruno says Woof!
```

```
        announce(Cat{Name: "Kitty"}) // Kitty says Meow!

        // interface slice — వేర్వేరు concrete types, ఒకే contract
        animals := []Speaker{Dog{"Tommy"}, Cat{"Whiskers"}}
        for _, a := range animals {
                fmt.Println(a.Speak())
        }
}
```

struct vs interface vs class (mapping)

అంశం	**Java** `class`	**Go**
Data holds	class fields	`struct` fields
Behavior contract	`interface` / `abstract`	`interface` (method set only)
Declare satisfy	`implements X` (explicit)	**ఏమీ రాయవద్దు** (implicit)
Reuse	`extends` (inheritance)	embedding (composition)
Polymorphism	via base class / interface	via interface only
Data + behavior	ఒకే class లో కలిసి	**వేరు వేరు** (struct=data, interface=behavior)

Value receiver vs Pointer receiver (ముఖ్యమైన Go decision)

Method receiver `(d Dog)` (value) నా `(d *Dog)` (pointer) నా — ఇది LLD లో పెద్ద decision:

అంశం	**Value receiver** `(d Dog)`	**Pointer receiver** `(d *Dog)`
Copy అవుతుందా?	అవును (whole struct copy)	కాదు (address పంపుతుంది)
Struct ని modify	**చేయలేదు** (copy మీద పని)	**చేయగలదు** (original మారుతుంది)
పెద్ద struct	ఖరీదు (copy overhead)	చౌక
Interface satisfy	value & pointer రెండూ satisfy	pointer మాత్రమే satisfy చేస్తుంది
ఎప్పుడు	small, immutable value objects	mutation అవసరం / పెద్ద struct / consistency

> **నియమం:** ఒక type కి కొన్ని methods pointer receiver వాడితే, **అన్నీ** pointer receiver వాడు (consistency). Mutation అవసరమా, struct పెద్దదా, లేక sync primitive ఉందా (`sync.Mutex` copy చేయకూడదు) → pointer receiver.

ఎప్పుడు వాడాలి / వద్దు

- **interface వాడు:** multiple implementations ఉన్నప్పుడు (Strategy), testing కోసం mock కావాల్సినప్పుడు, boundary దగ్గర (package APIs).
- **interface వద్దు:** ఒకే implementation ఉన్నప్పుడు premature గా interface రాయకు. "Concrete first, abstract later."

- **struct వాడు:** ఎప్పుడూ – data hold చేయడానికి Go లో struct ఒక్కటే మార్గం.

Gotchas

- **Empty interface `interface{}` / `any`** – ఏదైనా hold చేస్తుంది, కానీ type safety పోతుంది. అవసరమైతే మాత్రమే (generics వచ్చాక చాలాచోట్ల `any` అవసరం లేదు).
- **nil interface trap:** interface కి type ఉండి value nil అయితే, `iface == nil` **false** అవుతుంది. (typed nil bug) – Section 38 లో వివరంగా.
- Value receiver method ఉన్న type ని interface variable లో పెట్టి **modify చేయలేవు** – copy మీద పని జరుగుతుంది.
- Interface ని **pointer గా పంపవద్దు** (`*Speaker`) – interface అప్పటికే reference-like. `func f(s Speaker)` సరిపోతుంది.

Key Points

- Go = **struct (data) + interface (behavior) + composition (reuse)**. Class లేదు.
- Interfaces **implicit** – `Quack()` ఉంటే `Quacker` – declare చేయనవసరం లేదు.
- Behavior మరియు data **వేరు వేరు** – ఇది Go's biggest departure from Java.
- Value vs pointer receiver – mutation/size/consistency బట్టి decide చెయ్యి; ఒక type లో mix చేయకు.

3. Encapsulation in Go – Packages & Case

వివరణ

Java లో `private`, `protected`, `public` keywords ఉంటాయి. **Go లో ఇవి లేవు.** బదులుగా Go లో encapsulation **package** level లో, **identifier యొక్క మొదటి అక్షరం (case)** ద్వారా జరుగుతుంది:

- **Uppercase మొదటి అక్షరం** (`Name`, `Calculate`, `Config`) = **exported** = package బయట నుండి కనిపిస్తుంది (public లాంటిది).
- **lowercase మొదటి అక్షరం** (`name`, `calculate`, `config`) = **unexported** = ఆ package లోపల మాత్రమే కనిపిస్తుంది (private లాంటిది).

ముఖ్యమైన తేడా: Go లో privacy unit **class కాదు – package.** ఒకే package లోని అన్ని files ఒకదాని unexported fields ని access చేయగలవు. అంటే encapsulation boundary = **package folder**, individual struct కాదు.

Constructor pattern: unexported struct + exported `NewX()` function వాడి encapsulation enforce చేస్తారు.

Real-life Scenario

> **ఆఫీసు building** analogy: ఒక company (package) లో అన్ని employees (functions/types) ఒకరి desk drawer లోని files (unexported fields) ని చూడగలరు — అదే team కాబట్టి. కానీ **బయటి visitors (వేరే package)** కి reception లో పెట్టిన public brochure (exported) మాత్రమే కనిపిస్తుంది. Java లో drawer ఒక్కో వ్యక్తికి (class) private; Go లో drawer మొత్తం team (package) కి common.

Code

```
// ---------- file: bank/account.go ----------
package bank

import "errors"

// Account — struct. balance lowercase = unexported (బయటి package చూడలేదు)
type Account struct {
	Owner   string // exported — బయట చదవొచ్చు
	balance int    // unexported — package బయట నుండి direct access కుదరదు
}

// NewAccount — exported constructor. invariant enforce చేస్తుంది.
// (Go లో constructor keyword లేదు — convention: NewX)
func NewAccount(owner string, initial int) (*Account, error) {
	if initial < 0 {
		return nil, errors.New("initial balance negative కాకూడదు")
	}
	return &Account{Owner: owner, balance: initial}, nil
}

// Deposit — controlled mutation. balance ని validate చేసి మారుస్తుంది.
func (a *Account) Deposit(amount int) error {
	if amount <= 0 {
		return errors.New("amount positive గా ఉండాలి")
	}
	a.balance += amount
	return nil
}

// Balance — getter (read-only access to unexported field)
func (a *Account) Balance() int {
	return a.balance
}
```

```
// ---------- file: main.go ----------
package main

import (
	"fmt"
	"myapp/bank"
)

func main() {
```

```go
	acc, _ := bank.NewAccount("Ravi", 1000)
	acc.Deposit(500)
	fmt.Println(acc.Balance()) // 1500
	fmt.Println(acc.Owner)     // Ravi (exported — OK)

	// acc.balance = 999999  // ❌ COMPILE ERROR — unexported, బయట access కుదరదు
}
```

Java access modifiers → Go mapping

Java	Go equivalent
`public`	Uppercase identifier (exported)
`private`	lowercase identifier (unexported, package-scoped)
`protected`	**direct equivalent లేదు** — embedding + unexported దగ్గరగా
package-private (default)	lowercase (సరిగ్గా same idea)
`final` field	unexported + getter (setter లేకుండా)

Encapsulation techniques in Go

అవసరం	Go idiom
Field hide	lowercase field + `Field()` getter method
Invariant enforce	unexported struct + `NewX()` constructor
Immutability	unexported fields, getters only, no setters
"package లోపలే వాడాలి" helper	lowercase function
Interface hide implementation	exported interface + unexported concrete struct

ఎప్పుడు వాడాలి / వద్దు

- **Getter రాయి:** unexported field ని బయటికి చదవాలంటే. కానీ **అన్ని fields కి blindly getter/setter రాయకు** (Java bean anti-pattern) — అవసరమైన వాటికే.
- **Setter వద్దు:** వీలైనంత immutable గా ఉంచు. Mutation కి బదులు validated methods (`Deposit`) రాయి.
- **Exported field vs getter:** simple data struct (config, DTO) కి exported field OK. Invariant ఉంటే getter + constructor.

Gotchas

- **అన్ని fields exported చేయడం** = encapsulation పోతుంది. కానీ **అన్నీ unexported + getters** = Java boilerplate. Balance చూడు.
- Go లో `get` / `set` prefix వాడరు — getter పేరు `Balance()`, `GetBalance()` కాదు. Setter అవసరమైతే `SetBalance()`.

- ఒకే package లో అన్నీ చూడగలవు కాబట్టి, **strong encapsulation కావాలంటే separate package** లో పెట్టు.
- Struct copy చేసినప్పుడు unexported fields కూడా copy అవుతాయి – `sync.Mutex` ఉన్న struct ని value గా copy చేయకు (lock state కూడా copy అవుతుంది → bug).

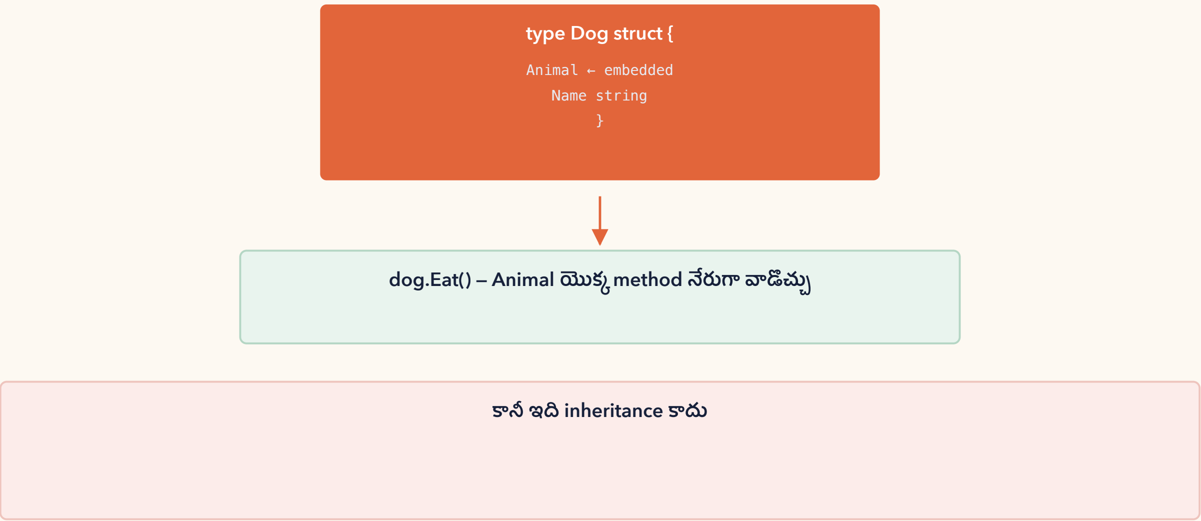
Key Points

- Go లో `private` / `public` keyword లేదు – **case ద్వారా** (Uppercase=exported, lowercase=unexported).
- Encapsulation boundary = **package**, class కాదు.
- Constructor keyword లేదు – convention `NewX()`. దీంతో invariants enforce చెయ్యి.
- Blindly getters/setters రాయకు; immutability + validated methods prefer చెయ్యి.

4. Composition over Inheritance – Embedding Deep

EMBEDDING · METHOD PROMOTION, INHERITANCE కాదు

EMBEDDING – method promotion



`var a Animal = dog` – ఇది **పని చేయదు**. Dog ఒక Animal కాదు.Embedding = composition + syntactic sugar. Method promotion మాత్రమే.Interface satisfy అవుతుంది కానీ – type hierarchy ఏర్పడదు.

వివరణ

Java లో reuse కి **inheritance** (`extends`) వాడతాం. **Go లో inheritance అసలు లేదు.** బదులుగా **composition** – దానికి Go native support: **embedding**.

Embedding = ఒక struct లో ఇంకో struct/interface ని **field name లేకుండా** పెట్టడం. అప్పుడు outer type, inner type యొక్క fields & methods ని **promote** చేసుకుంటుంది – inheritance లాగే కనిపిస్తుంది, కానీ అది **has-a**, **is-a కాదు**.

రెండు రకాల relationships:

- **is-a** (inheritance): "Dog IS-A Animal." Go లో **direct లేదు** – దీన్ని interface satisfaction తో represent చేస్తాం.

- **has-a** (composition): "Car HAS-A Engine." Go లో embedding / field తో.

Go philosophy: **"is-a కంటే has-a చాలా flexible."** Inheritance rigid tree; composition = Lego blocks, ఎలా కావాలంటే అలా కలపొచ్చు.

Real-life Scenario

> **మనిషి vs skills** analogy: Java inheritance = "నేను నా తండ్రి కొడుకుని కాబట్టి అతని లక్షణాలు అన్నీ నాకు వచ్చాయి" – నువ్వు మార్చలేవు (rigid, compile-time). Composition = "నేను driving నేర్చుకున్నా (Driver embed), cooking నేర్చుకున్నా (Cook embed)" – నీకు కావలసిన skills ని కలిపి నిన్ను నువ్వు నిర్మించుకుంటావు. రేపు కొత్త skill కావాలంటే add చేసుకో – తండ్రిని మార్చాల్సిన అవసరం లేదు.

Code – Embedding (struct in struct)

```
package main

import "fmt"

// Engine — reusable component
type Engine struct {
	Horsepower int
}

func (e Engine) Start() string {
	return fmt.Sprintf("Engine started (%d HP)", e.Horsepower)
}

// Car — Engine ని EMBED చేస్తోంది (field name లేదు).
// ఇది has-a: Car HAS-A Engine. కానీ Engine methods promote అవుతాయి.
type Car struct {
	Engine // embedded — Start() car మీద direct గా available
	Brand  string
}

func main() {
	c := Car{
		Engine: Engine{Horsepower: 150},
		Brand:  "Tata",
	}
	// Start() Engine ది — కానీ Car మీద direct గా పిలవొచ్చు (method promotion)
	fmt.Println(c.Start())        // Engine started (150 HP)
	fmt.Println(c.Horsepower)     // 150 — field కూడా promote అయింది
	fmt.Println(c.Engine.Start()) // explicit గా కూడా పిలవొచ్చు
}
```

Code – Method override (embedding తో)

```
package main

import "fmt"
```

```
type Logger struct{}

func (l Logger) Log(msg string) {
        fmt.Println("[LOG]", msg)
}

// TimestampLogger — Logger ని embed చేసి Log() ని "override" చేస్తోంది
type TimestampLogger struct {
        Logger // embedded
}

// ఇదే name తో method రాస్తే outer ది గెలుస్తుంది (shadowing = pseudo-override)
func (t TimestampLogger) Log(msg string) {
        fmt.Print("[2026-07-15] ")
        t.Logger.Log(msg) // embedded ది explicit గా పిలవడం (super.log() లాంటిది)
}

func main() {
        t := TimestampLogger{}
        t.Log("server started")
        // [2026-07-15] [LOG] server started
}
```

Inheritance vs Composition (Go)

అంశం	Inheritance (Java `extends`)	Composition/Embedding (Go)
Relationship	is-a	has-a
Coupling	Tight (parent మారితే child విరుగుతుంది)	Loose
Flexibility	Compile-time, rigid tree	Runtime-ish, mix & match
Multiple reuse	Java: single inheritance only	ఎన్ని types అయినా embed చేయవచ్చు
Diamond problem	ఉంది (C++)	Ambiguity ఉంటే compile error → explicit resolve
"super" call	`super.method()`	`t.Logger.Log()` (embedded name)
Go లో ఉందా?	**లేదు**	**Native, preferred**

Embedding interface కూడా చేయవచ్చు

```
// interface embedding — io.ReadWriter ఇలానే నిర్మించారు
type Reader interface { Read(p []byte) (int, error) }
type Writer interface { Write(p []byte) (int, error) }

type ReadWriter interface {
        Reader // embedded interface
        Writer // embedded interface
}
// ReadWriter = Read + Write రెండూ కావాలి. చిన్న interfaces కలిపి పెద్దది.
```

ఎప్పుడు వాడాలి / వద్దు

- **Embedding వాడు:** ఒక type యొక్క behavior ని reuse చేసి, పైన కొంచెం జోడించాలంటే (middleware, decorators). Interfaces కలపడానికి (io.ReadWriter).
- **వద్దు (over-embedding):** deep embedding chains (`A embeds B embeds C embeds D`) – debugging నరకం. 1-2 levels దాటకు.
- **is-a బలవంతంగా modeling వద్దు:** "Manager is-a Employee" అని embed చేసేకంటే, common behavior ని interface గా extract చేసి రెండూ satisfy చేయనివ్వు.

Gotchas

- **Method promotion ≠ inheritance.** Embedded type కి outer type గురించి తెలియదు. `Engine.Start()` లో `Car.Brand` access చేయలేవు – Java's `this` polymorphism లేదు ఇక్కడ. **ఇది చాలామంది తప్పుగా అనుకునేది.**
- **No virtual dispatch:** embedded method లోపల ఇంకో method పిలిస్తే, అది **embedded type ది** వాడుతుంది – outer override ది **కాదు**. (Template Method pattern Go లో embedding తో సరిగా పని చేయదు – Section 28 చూడు).
- **Name collision:** రెండు embedded types కి same method పేరు ఉంటే, outer లో explicit గా resolve చేయాలి, లేదా ambiguous access compile error.
- Embedded field ని **explicit name తో access** చేయవచ్చు (`c.Engine`) – ఇది "has-a" నిజం అని రుజువు.

Key Points

- Go లో **inheritance లేదు** – composition (embedding) native మరియు preferred.
- Embedding = has-a, method/field **promotion** ఇస్తుంది, కానీ ఇది inheritance **కాదు**.
- Embedded methods కి outer struct కనిపించదు – **no virtual dispatch** (ఇదే Template Method ని Go లో awkward చేస్తుంది).
- Interfaces ని కూడా embed చేసి పెద్ద contracts నిర్మించవచ్చు (io.ReadWriter).

5. Program to Interface – Accept Interfaces, Return Structs

ACCEPT INTERFACES, RETURN STRUCTS

ACCEPT INTERFACES, RETURN STRUCTS

తప్పు	సరైనది
func New() Storer { … } Caller కి concrete type కనిపించదు కొత్త methods చేర్చినా వాడలేరు	func New() *PostgresStore { … } func Save(s Storer, …) – accept interface Caller కి పూర్తి type; function కి flexibility

ఎందుకు

Interface ని **వాడేవాడు** నిర్వచించాలి – ఇచ్చేవాడు కాదు.
అప్పుడు ప్రతి consumer తనకి కావలసిన కనిష్ఠ interface ని నిర్వచించుకుంటాడు (ISP సహజంగా).
Producer package lo interface పెడితే – అందరూ దాన్నే వాడాలి, అది చాలా పెద్దదవుతుంది.

వివరణ

Go community లో అత్యంత famous design proverb: **"Accept interfaces, return structs."** ఇది program-to-interface principle యొక్క Go flavor.

- **Accept interfaces (parameters):** ఒక function ఏమి accept చేస్తుందో, దాన్ని **interface** గా declare చెయ్యి – అప్పుడు ఏ concrete type అయినా (అది contract satisfy చేస్తే) పంపొచ్చు. Testing లో mock పంపొచ్చు.
- **Return structs (concrete types):** function ఏమి తిరిగి ఇస్తుందో, దాన్ని **concrete struct** గా return చెయ్యి – caller కి full type information ఉంటుంది, తర్వాత అవసరమైతే interface లోకి assign చేసుకోవచ్చు.

రెండో crucial Go rule: **"interfaces belong to the consumer, not the producer."** అంటే – నీ package ఒక struct ఇస్తే, దానికి interface **నువ్వు రాయకు**. ఆ struct ని వాడే package (consumer) కి ఏ methods కావాలో, ఆ package లోనే చిన్న interface define చేసుకుంటుంది.

Go మరో గొప్పతనం: **small interfaces**. Standard library `io.Reader`, `io.Writer` కేవలం **ఒక్క method**. చిన్న interfaces = ఎక్కువ composability, సులభమైన mocking.

Real-life Scenario

> **Power socket** analogy: గోడ socket ఒక **interface** (`ThreePinPlug`) ని accept చేస్తుంది – laptop charger, phone charger, fan – ఏదైనా ఆ shape ఉంటే పని చేస్తుంది. Socket "ఏ company device?" అని అడగదు (concrete type మీద depend అవదు). కానీ **నీ charger box** (return) నీకు పూర్తి specs తో వస్తుంది (concrete struct) – తర్వాత నువ్వే దాన్ని socket కి తగిన plug (interface) గా వాడతావు.

Code – Accept interfaces, return structs

```go
package main

import (
	"fmt"
	"strings"
)

// చిన్న interface — ఒక్క method (io.Writer లాంటిది)
type Notifier interface {
	Notify(msg string) error
}

// ---- concrete implementations ----
type EmailNotifier struct{ Addr string }

func (e EmailNotifier) Notify(msg string) error {
	fmt.Printf("Email to %s: %s\n", e.Addr, msg)
	return nil
}

type SMSNotifier struct{ Phone string }
```

```
func (s SMSNotifier) Notify(msg string) error {
	fmt.Printf("SMS to %s: %s\n", s.Phone, msg)
	return nil
}

// ACCEPT INTERFACE — ఏ Notifier అయినా పంపొచ్చు.
// OrderService కి EmailNotifier / SMSNotifier గురించి తెలియదు (loose coupling).
type OrderService struct {
	notifier Notifier
}

// RETURN STRUCT — *OrderService (concrete), interface కాదు.
func NewOrderService(n Notifier) *OrderService {
	return &OrderService{notifier: n}
}

func (o *OrderService) PlaceOrder(item string) {
	o.notifier.Notify("Order placed: " + item)
}

func main() {
	// same OrderService, వేర్వేరు notifiers — code మారలేదు
	NewOrderService(EmailNotifier{"ravi@x.com"}).PlaceOrder("Book")
	NewOrderService(SMSNotifier{"99999"}).PlaceOrder("Pen")

	_ = strings.TrimSpace // (import demo)
}
```

Testing benefit — mock ఎంత సులభమో

```
// test file లో — framework అవసరం లేదు, ఒక struct చాలు
type spyNotifier struct{ lastMsg string }

func (s *spyNotifier) Notify(msg string) error {
	s.lastMsg = msg
	return nil
}

// అప్పుడు: NewOrderService(spy).PlaceOrder("X") పిలిచి spy.lastMsg check చేయవచ్చు.
// EmailNotifier నిజంగా email పంపే బదులు spy వాడతాం.
```

"Accept interfaces, return structs" ఎందుకు?

నియమం	ఎందుకు
Accept interface	Caller ఏ implementation అయినా పంపగలడు; mock సులభం; loose coupling
Return struct	Caller కి full API/fields కనిపిస్తాయి; premature abstraction లేదు; forward-compatible (కొత్త methods add చేయవచ్చు)
Interface consumer-side	Producer అనవసర interfaces రాయడు; consumer కి కావలసినంతే
Small interface	Composability ఎక్కువ; implement చేయడం సులభం; ISP

big interface vs small interface

Big interface (avoid)	Small interface (Go way)
`type Repository interface { 15 methods }`	`type Reader interface { Read(...) }`
Implement చేయడం కష్టం	ఒక method – trivial
Mock రాయడం నరకం	Mock trivial
Change ఎక్కువ మందిని affect చేస్తుంది	Isolated
"God interface" anti-pattern	`io.Reader`, `io.Writer`, `fmt.Stringer`

ఎప్పుడు వాడాలి / వద్దు

- **Accept interface వాడు:** dependency ని inject చేసేటప్పుడు (DB, notifier, clock, logger). Test చేయాల్సిన boundaries దగ్గర.
- **Interface వద్దు:** ఒకే implementation ఎప్పటికీ ఉంటుందని ఖచ్చితంగా తెలిస్తే premature interface వద్దు. Return values కి interface వద్దు (return struct).
- **Return interface ఎప్పుడు OK:** `error` (built-in interface), factory that genuinely returns multiple types (Section 12), `io.Reader` returning wrappers.

Gotchas

- **Return interface వల్ల నష్టం:** caller concrete fields access చేయలేడు; nil interface trap వస్తుంది; API extend చేయడం కష్టం. అందుకే **return struct** default.
- **Producer-side big interface** = Go anti-pattern. `UserService` interface లో 20 methods పెట్టి, ఒకటే implementation ఉంటే – అది అనవసరం.
- Interface parameter ని nil గా పంపి method పిలిస్తే **panic**. nil check అవసరమైతే చేయి.
- Over-abstraction: ప్రతి struct కి interface రాయడం "enterprise Go" smell. అవసరం వచ్చినప్పుడే extract చెయ్యి.

Key Points

- **"Accept interfaces, return structs"** – Go's #1 design proverb.
- **Interfaces belong to the consumer** – వాడేవాడు define చేస్తాడు, ఇచ్చేవాడు కాదు.
- **Small interfaces** (1-3 methods) = ఎక్కువ composability, సులభ mocking (ISP native).
- Return struct → premature abstraction తప్పించు; interface return చేయాల్సింది కొన్ని చోట్ల (`error`, genuine factories).

6. Class Relationships in Go – Association, Aggregation, Composition

వివరణ

OOD లో types మధ్య నాలుగు ముఖ్య relationships ఉంటాయి. Go లో ఇవి **fields, embedding, interface parameters** ద్వారా represent అవుతాయి (arrows/UML కి బదులు):

1. **Association** ("uses-a"): ఒక type ఇంకో type ని వాడుతుంది, కానీ own చేయదు. Go: **method parameter** గా తీసుకుంటుంది.
2. **Aggregation** ("has-a, weak"): ఒక type ఇంకో type ని hold చేస్తుంది, కానీ life-cycle వేరు (బయట నుండి inject అవుతుంది, share అవ్వొచ్చు). Go: **pointer/interface field**.
3. **Composition** ("has-a, strong / owns"): ఒక type ఇంకో దానిని పూర్తిగా own చేస్తుంది; parent పోతే child పోతుంది. Go: **value field** లేదా embedded value.
4. **Dependency** ("depends-on"): temporary use – parameter, return value, local variable.

Real-life Scenario

> **మనిషి & వస్తువులు** analogy:
>
> - **Association (uses-a):** నేను bus ని వాడతాను – bus నాది కాదు, నేను ఎక్కి దిగుతా. (parameter)
> - **Aggregation (has-a, weak):** ఒక team కి players ఉంటారు – team రద్దయినా players బతికే ఉంటారు, వేరే team కి వెళ్తారు. (shared pointer)
> - **Composition (owns-a):** ఇంటికి గదులు ఉంటాయి – ఇల్లు కూల్చేస్తే గదులు కూడా పోతాయి. (owned value)

Code – నాలుగింటిని ఒకచోట

```
package main

import "fmt"

// ---------- Composition (strong, owns) ----------
type Address struct { // Employee లో value గా – owns, life-cycle కలిసి
	City string
}

// ---------- Aggregation (weak, shared) ----------
type Department struct { // pointer – బయట నుండి share అవుతుంది
	Name string
}

// ---------- Association / uses-a (parameter) ----------
type PayrollService struct{}

func (p PayrollService) Pay(e *Employee, amount int) { // Employee ని వాడతాడు, own
చేయడు
```

```
	fmt.Printf("Paid %d to %s (%s)\n", amount, e.Name, e.Address.City)
}

type Employee struct {
	Name    string
	Address Address     // COMPOSITION — value field, Employee owns Address
	Dept    *Department // AGGREGATION — pointer, Department shared/external
}

func main() {
	sales := &Department{Name: "Sales"} // బయట పుట్టింది (shared)
	e := &Employee{
		Name:    "Ravi",
		Address: Address{City: "Hyderabad"}, // owned
		Dept:    sales,                       // shared
	}

	payroll := PayrollService{}   // DEPENDENCY (local)
	payroll.Pay(e, 50000)         // ASSOCIATION (e ని parameter గా వాడుతోంది)

	// sales ని ఇంకో employee తో share చేయవచ్చు (aggregation రుజువు)
	e2 := &Employee{Name: "Kiran", Dept: sales}
	fmt.Println(e2.Dept.Name) // Sales
}
```

Relationship → Go representation (కీలక table)

Relationship	అర్థం	Go representation	Life-cycle
Association	uses-a (temp)	method **parameter** / return	separate
Dependency	depends-on	local var, parameter, import	separate, temporary
Aggregation	has-a (weak)	**pointer** / interface field (injected)	separate, shared
Composition	owns-a (strong)	**value** field / embedded value	together (owned)
"is-a"	inheritance	Go లో నేరుగా లేదు → **interface** satisfaction	–

Value field vs Pointer field (aggregation vs composition సంకేతం)

```
type Engine struct{ HP int }

type CarA struct {
	Engine Engine // COMPOSITION — CarA తో Engine పుడుతుంది/చస్తుంది, own చేస్తుంది
}

type CarB struct {
	Engine *Engine // AGGREGATION — Engine బయట పుట్టింది, share అవ్వొచ్చు, nil అవ్వొచ్చు
}
```

అంశం	Value field (`Engine`)	Pointer field (`*Engine`)
Meaning	Composition (owns)	Aggregation (references/shares)
nil అవ్వగలదా?	కాదు (zero value ఉంటుంది)	అవును (nil possible)
Shared?	కాదు (copy)	అవును (అదే instance)
Mutation propagate	కాదు (copy)	అవును (అదే object మారుతుంది)

ఎప్పుడు వాడాలి / వద్దు

- **Value field (composition):** child పూర్తిగా parent కి చెందితే, small & copyable అయితే (`Address` , `time.Time`).
- **Pointer/interface field (aggregation):** dependency ని inject చేసేటప్పుడు, share చేయాల్సినప్పుడు, nil possible అయినప్పుడు (DB, logger).
- **Parameter (association):** temporary use కి – struct లో store చేయనవసరం లేకపోతే.

Gotchas

- **Value field copy semantics:** value field ఉన్న struct ని copy చేస్తే inner value కూడా copy – mutation share అవ్వదు (కావాలంటే pointer వాడు).
- **Aggregation with pointer = shared mutation risk:** ఒక `*Department` ని ఇద్దరు employees share చేస్తే, ఒకరు మార్చితే ఇద్దరికీ మారుతుంది (concurrency లో race).
- **Circular references:** `A` holds `*B` , `B` holds `*A` – Go GC handle చేస్తుంది కానీ design smell (Mediator/interface తో break చెయ్యి).
- Interface field = aggregation + polymorphism కలిపి (dependency inject చేసే idiomatic way).

Key Points

- OOD relationships Go లో **fields (value/pointer/interface) + parameters** ద్వారా represent అవుతాయి.
- **Value field = composition (owns)**, **pointer/interface field = aggregation (shares)**, **parameter = association (uses)**.
- "is-a" కి Go లో direct లేదు → **interface satisfaction** తో represent చెయ్యి.
- Pointer/interface field = idiomatic dependency injection (aggregation).

---

7. UML → Go Mapping (Class Diagram చదివి Go రాయడం)

వివరణ

Interviews లో UML class diagram ఇస్తారు, లేదా నువ్వు గీసి Go లోకి translate చేయాలి. Go లో class లేదు కాబట్టి mapping కొంచెం వేరు – UML "class" ని **struct + methods + optional interface** గా విడగొట్టాలి.

కీలక mapping:

- UML **class** → Go `struct` (fields) + methods (`func (r T) ...`).

- UML **abstract class / interface** → Go `interface`.
- UML **inheritance (hollow triangle arrow)** → Go: **interface satisfaction** (is-a behavior) లేదా embedding (reuse).
- UML **composition (filled diamond)** → value field.
- UML **aggregation (hollow diamond)** → pointer/interface field.
- UML **association (plain arrow)** → parameter / field.
- UML **multiplicity (1..*, 0..1)** → slice `[]T`, map `map[K]T`, pointer `*T`.

Real-life Scenario

> **Blueprint** → **building** analogy: UML = architect గీసిన blueprint (boxes, arrows). Go code = actual construction. Blueprint లో "wall" (class box) ఉంటే, నువ్వు ఇటుకలు (struct fields) + తలుపు spec (interface) గా విడగొడతావు. Arrows ఏ గోడ ఏ గదికి connect అవుతుందో చెప్తాయి (relationships → fields).

Example UML → Go

భావించు ఇలాంటి diagram ఇచ్చారు:

```
          <<interface>>
           PaymentMethod
          + Pay(amount) error
                △
                │ (satisfied by)
         ┌──────┴──────────┐
     CreditCard        UPI
     - number          - vpa
     + Pay()           + Pay()

     Order ◆────── OrderItem  (composition, 1..*)
     Order ──────> PaymentMethod  (association / uses)
     Order ◇────── Customer  (aggregation)
```

Go లోకి translate:

```
package main

import (
        "errors"
        "fmt"
)

// <<interface>> PaymentMethod → Go interface
type PaymentMethod interface {
        Pay(amount int) error
}

// CreditCard, UPI → structs satisfying PaymentMethod (UML inheritance triangle)
type CreditCard struct {
        number string // '-' = private = unexported
```

```
}

func (c CreditCard) Pay(amount int) error {
	fmt.Printf("Paid %d via CreditCard %s\n", amount, c.number)
	return nil
}

type UPI struct {
	vpa string
}

func (u UPI) Pay(amount int) error {
	if u.vpa == "" {
		return errors.New("invalid VPA")
	}
	fmt.Printf("Paid %d via UPI %s\n", amount, u.vpa)
	return nil
}

// OrderItem — composition target
type OrderItem struct {
	Name  string
	Price int
}

// Customer — aggregation target (shared, pointer)
type Customer struct {
	Name string
}

// Order — filled diamond (composition) = value slice; 1..* = slice
//         hollow diamond (aggregation) = pointer field
//         association to PaymentMethod = interface field/param
type Order struct {
	Items    []OrderItem // COMPOSITION, 1..* → []value (Order owns items)
	Customer *Customer   // AGGREGATION → pointer (shared, separate lifecycle)
}

// association (uses) → PaymentMethod parameter
func (o *Order) Checkout(pm PaymentMethod) error {
	total := 0
	for _, it := range o.Items {
		total += it.Price
	}
	fmt.Printf("Order for %s, total %d\n", o.Customer.Name, total)
	return pm.Pay(total)
}

func main() {
	cust := &Customer{Name: "Ravi"}
	order := &Order{
		Items:    []OrderItem{{"Book", 300}, {"Pen", 50}},
		Customer: cust,
	}
	order.Checkout(UPI{vpa: "ravi@upi"})        // 350 via UPI
	order.Checkout(CreditCard{number: "**1234"}) // 350 via CreditCard
}
```

UML → Go cheat table

UML element	Go representation
Class box	`struct` + methods
`<<interface>>` / abstract	`interface`
Field `- x: int` (private)	`x int` (unexported)
Field `+ X: int` (public)	`X int` (exported)
Method `+ Foo()`	`func (r T) Foo()`
Inheritance (△ hollow triangle)	interface satisfaction (behavior) / embed
Composition (◆ filled diamond)	value field
Aggregation (◇ hollow diamond)	pointer / interface field
Association (→ arrow)	parameter / field
Dependency (dashed →)	parameter / local / import
Multiplicity `1..*`	`[]T`
Multiplicity `0..1`	`*T` (nil అవ్వగలదు)
Multiplicity `*` keyed	`map[K]T`
Enum	`const` block + `iota` typed const

Enum mapping (UML enum → Go iota)

```
type OrderStatus int

const (
        Pending OrderStatus = iota // 0
        Paid                       // 1
        Shipped                    // 2
        Delivered                  // 3
)

func (s OrderStatus) String() string { // fmt.Stringer — pretty print
        return [...]string{"Pending", "Paid", "Shipped", "Delivered"}[s]
}
```

ఎప్పుడు వాడాలి / వద్దు

- **UML మొదట గీయి:** interview లో – entities (nouns → structs), behaviors (verbs → interface methods), relationships (arrows → fields) గుర్తించడానికి.
- **Over-modeling వద్దు:** ప్రతి UML class కి Go లో interface రాయకు – behavior polymorphic అయినప్పుడు మాత్రమే interface. మిగతావి plain structs.

- **1:1 translation వద్దు:** Java UML ని అలాగే copy చేయకు – Go లో inheritance chains ని composition + interfaces గా re-model చెయ్యి.

Gotchas

- UML inheritance ని **embedding గా** translate చేయడం చాలాసార్లు తప్పు – అది is-a behavior అయితే **interface satisfaction** వాడు, code reuse అయితే embedding.
- Multiplicity `1..*` ని pointer slice `[]*T` నా value slice `[]T` నా – mutation/sharing అవసరం బట్టి decide చెయ్యి.
- Bidirectional association (రెండు వైపులా arrows) → circular pointers → design smell. ఒక వైపు interface గా break చెయ్యి.
- UML లో "getter/setter" methods ఉన్నా, Go లో అన్నిటికీ getter/setter రాయకు (Section 3).

Key Points

- UML class → **struct + methods (+ interface behavior polymorphic అయితే)**.
- Inheritance triangle → **interface satisfaction** (behavior) లేదా embedding (reuse) – తేడా గుర్తించు.
- Diamonds: **filled = composition = value field**, **hollow = aggregation = pointer field**.
- Multiplicity → `[]T`, `map[K]V`, `*T`. Enums → `const` + `iota` + `String()`.

Part 2 – SOLID & Principles in Go

> SOLID principles Go కి కూడా వర్తిస్తాయి – కానీ language వేరు కాబట్టి **అమలు (implementation)** వేరు. ISP అయితే Go లో default (small interfaces), DIP అయితే interfaces మీద depend చేయడం. ఒక్కో principle ని **before/after Go code** తో చూద్దాం.

---

8. SOLID in Go – SRP, OCP, LSP, ISP, DIP

SOLID IN GO · చిన్న INTERFACES

SOLID – Go యొక్క రూపంలో

S	చిన్న types, ఒక్క పని
O	interface + composition (embedding కాదు)
L	interface contract ని గౌరవించడం
I	చిన్న interfaces – io.Reader ఒక్క method
D	concrete కాదు, interface ని accept చేయడం

Go lo ISP సహజం

Standard library చూడండి: io.Reader (1 method), io.Writer (1), io.Closer (1).
పెద్దవి కావాలంటే – వాటిని కలపడం (io.ReadWriteCloser).
"The bigger the interface, the weaker the abstraction" – Rob Pike.

వివరణ

SOLID = maintainable OOD కి 5 principles. Go లో "class" లేకపోయినా principles వర్తిస్తాయి – struct/interface/package level లో. కీలకం: **Go లో ISP మరియు DIP దాదాపు native** (small interfaces + accept interfaces). ఒక్కోదాన్ని before/after Go code తో చూద్దాం.

Real-life Scenario

> **మంచి restaurant kitchen** analogy: ఒక్కో chef ఒక్క పని (SRP – grill chef, salad chef). కొత్త dish వస్తే కొత్త chef ని జోడిస్తారు, పాత recipe మార్చరు (OCP). ఏ grill chef అయినా grill station లో పని చేయగలడు (LSP). ఒక్కో chef కి కావలసిన tools మాత్రమే ఇస్తారు, మొత్తం kitchen కాదు (ISP). Chef "heat source" మీద depend అవుతాడు – gas నా induction నా అనవసరం (DIP).

S – Single Responsibility Principle (SRP)

ఒక type = ఒక కారణం change అవ్వడానికి. ఒక struct data చదవడం, business logic, print చేయడం అన్నీ చేస్తే – మూడు కారణాలకి change అవుతుంది.

```
// ❌ BEFORE — Report ఒకటే struct: data + formatting + saving అన్నీ
type Report struct{ Data []int }

func (r Report) Calculate() int { /* sum */ return 0 }
func (r Report) FormatHTML() string { return "<html>..." } // presentation
func (r Report) SaveToFile(path string) error { return nil } // persistence
// 3 responsibilities → 3 reasons to change → SRP violation
```

```
// ✅ AFTER — ఒక్కో responsibility ఒక్కో type
type Report struct{ Data []int }
func (r Report) Total() int { s := 0; for _, v := range r.Data { s += v }; return s }

type HTMLFormatter struct{}
func (HTMLFormatter) Format(r Report) string { return "<html>...</html>" }

type FileStore struct{}
func (FileStore) Save(path, content string) error { return nil }
// ఇప్పుడు formatting మారితే HTMLFormatter మాత్రమే మారుతుంది
```

O – Open/Closed Principle (OCP)

Extension కి open, modification కి closed. కొత్త behavior add చేయాలంటే existing code మార్చకూడదు. Go లో ఇది **interfaces** తో – కొత్త type add చేస్తే switch/if మార్చనవసరం లేదు.

```
// ❌ BEFORE — కొత్త shape వస్తే ఈ function మార్చాలి (closed కాదు)
func Area(shapeType string, dim float64) float64 {
        switch shapeType {
        case "circle":
                return 3.14 * dim * dim
        case "square":
                return dim * dim
        // కొత్త "triangle" → ఇక్కడ ఎడిట్ చేయాలి → OCP violation
        }
        return 0
}
```

```
// ✅ AFTER — interface. కొత్త shape = కొత్త type, ఈ code touch చేయకుండా
type Shape interface{ Area() float64 }

type Circle struct{ R float64 }
func (c Circle) Area() float64 { return 3.14 * c.R * c.R }

type Square struct{ Side float64 }
func (s Square) Area() float64 { return s.Side * s.Side }

// ఈ function ఎప్పటికీ మారదు (closed), కానీ కొత్త Shape లు వచ్చినా పని చేస్తుంది (open)
```

```
func TotalArea(shapes []Shape) float64 {
	total := 0.0
	for _, s := range shapes {
		total += s.Area()
	}
	return total
}
// కొత్త Triangle struct రాస్తే చాలు — TotalArea మారదు
```

L — Liskov Substitution Principle (LSP)

ఒక interface ని satisfy చేసే ఏ type అయినా, ఆ interface వాడేచోట పని చేయాలి — surprise ఇవ్వకూడదు. Go లో "subtype" లేదు కానీ interface satisfaction ఉంది — ఒక implementation contract ని violate చేస్తే (panic, wrong behavior) LSP break.

```
// ❌ BEFORE — ReadOnlyFile "Writer" అని claim చేస్తుంది కానీ Write panic చేస్తుంది
type Storage interface {
	Read() string
	Write(s string) error
}

type ReadOnlyFile struct{ content string }
func (r ReadOnlyFile) Read() string { return r.content }
func (r ReadOnlyFile) Write(s string) error {
	panic("cannot write!") // ❌ contract break — caller expect చేయడు
}
// Storage వాడే code ReadOnlyFile పంపితే crash → LSP violation
```

```
// ✅ AFTER — interfaces విడగొట్టు (ISP కూడా). ReadOnlyFile కి Write లేనే లేదు
type Reader interface{ Read() string }
type Writer interface{ Write(s string) error }

type ReadOnlyFile struct{ content string }
func (r ReadOnlyFile) Read() string { return r.content } // Reader మాత్రమే satisfy

// ఇప్పుడు ReadOnlyFile ని Writer expect చేసేచోట పంపలేవు — compile time లోనే safe
```

> **Go లో LSP గుర్తుంచుకో:** ఒక interface satisfy చేస్తే, దాని **behavioral contract** ని కూడా గౌరవించాలి (nil return కాదు, panic కాదు, unexpected side effects కాదు). లేదంటే type system OK అన్నా runtime లో విరుగుతుంది.

I — Interface Segregation Principle (ISP)

క్లయింట్‌లకు అవసరం లేని methods మీద depend చేయవద్దు. పెద్ద "god interface" కాదు — చిన్న, focused interfaces. **Go దీన్ని ఇష్టపడుతుంది** — `io.Reader`, `io.Writer` కేవలం ఒక్క method.

```
// ❌ BEFORE — fat interface. Printer కి Scan, Fax అవసరం లేదు కానీ implement చేయాలి
type Machine interface {
	Print(doc string)
```

```
        Scan(doc string)
        Fax(doc string)
}

type OldPrinter struct{}
func (OldPrinter) Print(doc string) {}
func (OldPrinter) Scan(doc string)  { panic("no scanner!") } // ❌ బలవంతం
func (OldPrinter) Fax(doc string)   { panic("no fax!") }     // ❌
```

```
// ✅ AFTER — చిన్న interfaces. Client తనకి కావలసినది మాత్రమే డిమాండ్ చేస్తాడు
type Printer interface{ Print(doc string) }
type Scanner interface{ Scan(doc string) }
type Faxer interface{ Fax(doc string) }

// composition — కావాలంటే కలుపు (io.ReadWriter లాంటిది)
type MultiFunction interface {
        Printer
        Scanner
        Faxer
}

type OldPrinter struct{}
func (OldPrinter) Print(doc string) {} // Printer మాత్రమే — Scan/Fax అవసరం లేదు

// function తనకి కావలసిన చిన్న interface మాత్రమే accept చేస్తుంది
func printAll(p Printer, docs []string) {
        for _, d := range docs {
                p.Print(d)
        }
}
```

D – Dependency Inversion Principle (DIP)

High-level modules low-level modules మీద కాదు – రెండూ abstraction (interface) మీద depend అవ్వాలి. Go లో idiomatic DI = **constructor లోకి interface pass చేయడం** ("accept interfaces").

```
// ❌ BEFORE — OrderService నేరుగా MySQLDB (concrete) మీద depend
type MySQLDB struct{}
func (MySQLDB) Save(order string) {}

type OrderService struct {
        db MySQLDB // ❌ concrete dependency — test చేయలేం, Postgres కి మార్చలేం
}
func (o OrderService) Place(order string) { o.db.Save(order) }
```

```
// ✅ AFTER — abstraction (interface) మీద depend. concrete ని బయట నుండి inject
type OrderStore interface { // high-level module తనకి కావలసిన contract define చేస్తుంది
        Save(order string) error
}
```

```go
type OrderService struct {
	store OrderStore // interface — ఏ implementation అయినా
}

func NewOrderService(s OrderStore) *OrderService { // DI via constructor
	return &OrderService{store: s}
}
func (o *OrderService) Place(order string) error { return o.store.Save(order) }

// ఇప్పుడు: MySQLStore, PostgresStore, InMemoryStore (test) — ఏదైనా inject
type InMemoryStore struct{ orders []string }
func (m *InMemoryStore) Save(o string) error { m.orders = append(m.orders, o); return
nil }
```

> **DIP + "interfaces belong to consumer":** `OrderStore` interface ని `OrderService` package లోనే define చేస్తాం (వాడేవాడు), MySQL package లో కాదు. ఇది Go idiom.

SOLID in Go — summary table

Principle	ఒక్క ముక్కలో	Go idiom
SRP	ఒక type = ఒక change reason	చిన్న focused structs, responsibilities విడగొట్టు
OCP	Extension open, modification closed	interface + polymorphism (switch తగ్గించు)
LSP	Substitute surprise ఇవ్వకూడదు	interface contract గౌరవించు (panic/nil కాదు)
ISP	చిన్న interfaces	`io.Reader` style, 1-3 methods (**Go native**)
DIP	Abstraction మీద depend	accept interfaces, constructor injection

ఎప్పుడు వాడాలి / వద్దు

- **SOLID వాడు:** codebase పెరుగుతున్నప్పుడు, multiple implementations వచ్చినప్పుడు, testing అవసరమైనప్పుడు.
- **Over-apply వద్దు:** చిన్న program కి 5 interfaces + factories = over-engineering (Section 9 YAGNI). Go లో "concrete first" — అవసరం వచ్చినప్పుడు interface extract చెయ్యి.
- **ISP గుర్తుంచుకో:** Go లో fat interface రాయడం చాలా అరుదుగా justified — దాదాపు ఎప్పుడూ చిన్నవి.

Gotchas

- **DIP ని Java-style గా over-do చేయడం:** ప్రతి struct కి interface, factory, DI container — Go లో అనవసరం. Manual constructor injection చాలుకి సరిపోతుంది.
- **OCP కోసం premature interface:** ఒకే implementation ఉంటే interface వద్దు — future కోసం guess చేయకు (YAGNI).
- **LSP silent break:** Go compiler interface satisfaction check చేస్తుంది కానీ **behavioral contract** ని కాదు — nil/panic/wrong result runtime లో మాత్రమే తెలుస్తుంది. Tests రాయి.
- ISP interfaces ని consumer-side define చేయి; producer package లో god interface వద్దు.

Key Points

- SOLID Go కి వర్తిస్తుంది – struct/interface/package level లో.
- **ISP + DIP దాదాపు Go native** (small interfaces + accept interfaces).
- OCP = interface polymorphism (switch తగ్గించు); LSP = interface **behavioral** contract గౌరవించు.
- Java-style over-abstraction వద్దు – "concrete first, extract interface when needed."

9. DRY, KISS, YAGNI in Go

వివరణ

ఇవి "meta-principles" – ఏ pattern కంటే ముందు వచ్చే commonsense rules. Go community ఇవి చాలా బలంగా నమ్ముతుంది (**"clear is better than clever"** – Go proverb).

- **DRY (Don't Repeat Yourself):** ఒకే logic ని రెండుసార్లు రాయకు – extract చెయ్యి (function, method, generic).
- **KISS (Keep It Simple, Stupid):** simplest solution ఎంచుకో. Go clever code ని discourage చేస్తుంది.
- **YAGNI (You Aren't Gonna Need It):** "future కి అవసరం అవ్వొచ్చు" అని ఇప్పుడు రాయకు. అవసరం వచ్చినప్పుడు రాయి.

Go లో ఒక subtle rule: **"A little copying is better than a little dependency."** (Go proverb) – DRY ని గుడ్డిగా follow చేసి అనవసర coupling తీసుకురావద్దు.

Real-life Scenario

> **వంట** analogy: DRY = మసాలా mix ఒకసారి చేసి అన్ని curries కి వాడు (repeat చేయకు). KISS = సాధారణ recipe – 20 ingredients కూరవద్దు. YAGNI = రేపు guests రావొచ్చని ఈరోజే 50 మందికి వండకు – వచ్చినప్పుడు వండు. కానీ overly-DRY = ఒక common పాత్రలో అన్నీ కలిపి, ఒకటి మార్చాలంటే అన్నీ affect అవుతాయి (coupling).

Code – DRY (extract + generics)

```
package main

import "fmt"

// ❌ NOT DRY — same loop మూడుసార్లు
func sumInts(xs []int) int   { s := 0; for _, v := range xs { s += v }; return s }
// ...ఇలా floats కి మళ్ళీ రాయడం

// ✅ DRY with generics (Go 1.18+) — ఒకసారి రాసి అన్నిటికీ
type Number interface{ ~int | ~int64 | ~float64 }

func Sum[T Number](xs []T) T {
        var total T
```

```go
	for _, v := range xs {
		total += v
	}
	return total
}

func main() {
	fmt.Println(Sum([]int{1, 2, 3}))        // 6
	fmt.Println(Sum([]float64{1.5, 2.5}))   // 4
	_ = sumInts
}
```

Code – KISS (simple over clever)

```go
// ❌ CLEVER — reflection + interface{} maze, ఎవరూ maintain చేయలేరు
func processClever(data interface{}) interface{} { /* reflect gymnastics */ return
nil }

// ✅ KISS — explicit, boring, readable
func maxInt(a, b int) int {
	if a > b {
		return a
	}
	return b
}
// Go 1.21+ లో built-in max(a, b) కూడా ఉంది — inbuilt ఉంటే అది వాడు (ఇంకా KISS)
```

Code – YAGNI (అవసరం లేనిది రాయకు)

```go
// ❌ YAGNI violation — ఒకటే DB, కానీ "future కోసం" 5 interfaces + factory + config
layers
// type StorageFactory interface { ... }
// type StorageProvider interface { ... }  // అనవసరం, ఒకటే implementation

// ✅ YAGNI — ఇప్పుడు కావలసింది మాత్రమే. concrete struct.
type UserStore struct{ users map[int]string }

func NewUserStore() *UserStore { return &UserStore{users: map[int]string{}} }
func (u *UserStore) Get(id int) string { return u.users[id] }
// రేపు రెండో store అవసరం వస్తే — అప్పుడు interface extract చేస్తాం (30 సెకన్ల పని)
```

DRY vs "little copying" – trade-off table

పరిస్థితి	ఏం చేయాలి
ఒకే business logic 3+ చోట్ల	Extract (DRY)
రెండు packages coincidentally similar	Copy చేయి (coupling తీసుకురావద్దు)
Complex shared helper for 2 callers only	Inline / copy (KISS > DRY)
Type-only difference (int/float/string)	Generics (DRY)

పరిస్థితి	ఏం చేయాలి
"future కోసం" abstraction	రాయకు (YAGNI)

ఎప్పుడు వాడాలి / వద్దు

- **DRY వాడు:** genuine domain logic duplicate అయినప్పుడు. Generics/functions/methods తో extract.
- **DRY వద్దు:** rushed premature abstraction – రెండు similar-looking codes actually వేరు వేరు reasons కి change అయితే, వాటిని కలపవద్దు (అది false DRY).
- **YAGNI ఎప్పుడూ default:** doubt వస్తే simpler option ఎంచుకో.

Gotchas

- **Over-DRY = tight coupling.** ఒక shared function ని 10 callers వాడితే, ఒకరికి change అవసరమైతే అందరూ risk. "Rule of three" – 3 సార్లు repeat అయ్యాకే extract.
- **KISS ని "no abstraction" గా misread చేయవద్దు** – sometimes ఒక interface simpler చేస్తుంది. Simple = least cognitive load, "least code" కాదు.
- **YAGNI vs extensibility balance:** genuinely known future requirement ఉంటే design చేయి; guess అయితే వద్దు.
- Go generics ని over-use చేయకు – simple case లో concrete type చాలు (KISS).

Key Points

- **DRY:** repeat చేయకు (functions, methods, generics) – కానీ **"a little copying is better than a little dependency."**
- **KISS:** "clear is better than clever" – Go clever code ని discourage చేస్తుంది. Built-ins (`max`, `min`) వాడు.
- **YAGNI:** future కోసం guess చేసి రాయకు – "concrete first, abstract when needed."
- Rule of three – 3 సార్లు repeat అయ్యాకే extraction ఆలోచించు.

10. Law of Demeter, Encapsulate What Varies, Favor Immutability

వివరణ

మూడు practical principles, ఒకచోట:

1. **Law of Demeter (LoD) – "least knowledge":** ఒక method తన **direct friends** తోనే మాట్లాడాలి – strangers తో కాదు. `a.GetB().GetC().DoSomething()` (train wreck) = LoD violation. Go లో embedded field chains ఇలా జరగవచ్చు – జాగ్రత్త.
2. **Encapsulate What Varies:** ఏది తరచూ మారుతుందో దాన్ని isolate చెయ్యి (interface వెనుక దాచు) – మార్పు ఒకేచోట. ఇదే చాలా design patterns వెనుక ఉన్న core idea.

3. **Favor Immutability (value semantics):** వీలైనంత immutable data వాడు – Go value types (struct copy) దీనికి బాగా సరిపోతాయి. Shared mutable state = concurrency bugs.

Real-life Scenario

> **LoD:** పిజ్జా delivery – నువ్వు delivery boy కి డబ్బు ఇస్తావు (direct friend). అతని wallet లో చెయ్యి పెట్టి, అందులో నుండి change తీసుకుని, restaurant owner కి ఇవ్వవు (`boy.wallet.cash.give()` – train wreck). నువ్వు boy తో మాత్రమే deal చేస్తావు. **Encapsulate what varies:** menu లో ధరలు మారతాయి కాబట్టి menu ని ఒక board మీద రాస్తారు – recipe books లో కాదు. మారేది ఒకచోట. **Immutability:** bill print అయ్యాక దాన్ని మార్చరు – కొత్త bill ఇస్తారు.

Code – Law of Demeter

```
package main

import "fmt"

type Engine struct{ fuel int }
func (e *Engine) burn() { e.fuel-- }

type Car struct{ engine *Engine }

// ❌ LoD violation — caller car లోపలి engine ని లోతుగా తెలుసుకోవాలి
// driver code: car.engine.fuel -= 1   // strangers తో మాట్లాడటం

// ✅ LoD — Car ఒక method expose చేస్తుంది; caller engine గురించి తెలియనవసరం లేదు
func (c *Car) Drive() {
        c.engine.burn() // Car తన direct friend (engine) తో మాట్లాడుతుంది
        fmt.Println("Driving...")
}

func main() {
        c := &Car{engine: &Engine{fuel: 10}}
        c.Drive() // driver కి engine.fuel గురించి తెలియదు — good
}
```

Code – Encapsulate What Varies (Strategy preview)

```
// ధర లెక్కింపు తరహా మారుతుంది → interface వెనుక దాచు (encapsulate)
type PricingStrategy interface {
        Price(base float64) float64
}

type RegularPricing struct{}
func (RegularPricing) Price(base float64) float64 { return base }

type FestivalPricing struct{ Discount float64 }
func (f FestivalPricing) Price(base float64) float64 { return base * (1 - f.Discount)
}

type Checkout struct{ strategy PricingStrategy } // మారేది ఒక్క field లోనే isolate
```

```
func (c Checkout) Total(base float64) float64 { return c.strategy.Price(base) }
// కొత్త pricing వస్తే → కొత్త struct, Checkout మారదు (OCP కూడా)
```

Code – Favor Immutability (value semantics)

```
package main

import "fmt"

// immutable-style: unexported fields, no setters, methods కొత్త value return చేస్తాయి
type Money struct {
	amount   int    // paise లో
	currency string
}

func NewMoney(rupees int) Money { return Money{amount: rupees * 100, currency: "INR"}
}

// Add mutate చేయదు — కొత్త Money return చేస్తుంది (immutable pattern)
func (m Money) Add(other Money) Money {
	return Money{amount: m.amount + other.amount, currency: m.currency}
}

func (m Money) String() string { return fmt.Sprintf("₹%.2f", float64(m.amount)/100) }

func main() {
	a := NewMoney(100)
	b := NewMoney(50)
	c := a.Add(b) // a, b మారలేదు — c కొత్తది
	fmt.Println(a, b, c) // ₹100.00 ₹50.00 ₹150.00
}
```

Value semantics vs Pointer semantics (immutability lens)

అంశం	Value (`Money`)	Pointer (`*Money`)
Copy on pass	అవును (immutable-friendly)	కాదు (shared, mutable)
Concurrency safe	ఎక్కువ (each goroutine తన copy)	తక్కువ (shared → mutex అవసరం)
Mutation	కొత్త value return చేయాలి	in-place
ఎప్పుడు	small, immutable value objects	పెద్ద struct, shared identity, mutation

LoD – ఎన్ని dots OK?

Code	LoD?
`a.DoSomething()`	✅ OK (direct friend)
`a.B().DoSomething()`	⚠️ borderline (B returned by a)
`a.B().C().DoSomething()`	❌ train wreck (stranger C)

Code	LoD?
`car.Drive()`	✅ OK (delegation)
`car.Engine().Fuel().Level()`	❌ leak – internals expose అవుతున్నాయి

ఎప్పుడు వాడాలి / వద్దు

- **LoD వాడు:** delegation methods రాసి internals hide చెయ్యి. కానీ **fluent builders** (`b.WithX().WithY()`) LoD violation కాదు – అది same object return చేస్తుంది (chaining, train wreck కాదు).
- **Encapsulate what varies:** requirement తరచూ మారే భాగాన్ని interface వెనుక పెట్టు. మారని దాన్ని over-abstract చేయకు (YAGNI).
- **Immutability వాడు:** value objects, config, DTOs కి. Concurrency ఉన్నచోట బలంగా prefer చెయ్యి.
- **Immutability వద్దు:** పెద్ద structs constant గా copy చేస్తే performance hit – అప్పుడు pointer + mutex.

Gotchas

- **Embedding LoD ని దాచొచ్చు కానీ break చేయదు:** `car.Start()` (promoted from Engine) OK, కానీ `car.Engine.Internals.X` leak.
- **Go లో "immutable" strict కాదు:** `const` కేవలం primitives కి. Struct immutability = convention (unexported fields + no setters). Reflection/same-package code ఇంకా మార్చొచ్చు.
- **Slice/map fields immutability break చేస్తాయి:** struct copy చేసినా, అందులో slice/map ఉంటే అది **shared** (reference). Deep copy అవసరమైతే explicit గా చెయ్యి (Section 15 Prototype).
- **Mutex ఉన్న struct ని value గా copy చేయకు** – `go vet` warn చేస్తుంది. Immutability కావాలంటే mutex లేని design.

Key Points

- **LoD:** direct friends తోనే మాట్లాడు – `a.b.c.d()` train wreck వద్దు; delegation methods రాయి.
- **Encapsulate what varies:** మారే భాగాన్ని interface వెనుక దాచు – patterns అన్నిటి వెనుక core idea.
- **Favor immutability:** Go value semantics (struct copy) దీనికి బాగా సరిపోతాయి; concurrency లో బలంగా prefer.
- Slice/map fields shared reference – struct copy చేసినా deep copy కాదు; జాగ్రత్త.

Part 3 – Creational Patterns (Go Idiomatic)

> "Objects ఎలా పుట్టాలి?" అనే ప్రశ్నకి సమాధానాలు. కానీ Go లో ఇవి Java కంటే చాలా simpler – `new` keyword ఉన్నా, idiom `&T{}` లేదా `NewX()` constructor. Singleton = `sync.Once`, Builder ≈ **functional options**. ప్రతిదానికి **Go idiom** section లో classic version నుండి తేడా చూద్దాం.

---

11. Singleton (sync.Once, package-level var, init())

SINGLETON IN GO · SYNC.ONCE

sync.Once – Go యొక్క idiomatic singleton

ఇతర భాషల శైలి

if instance == nil { … }
Race condition – ఇద్దరు ఒకేసారి
Double-checked locking సంక్లిష్టం

Go శైలి

var once sync.Once
once.Do(func(){ … })
Thread-safe, ఒక్కసారే – హామీ

కానీ – నిజంగా singleton కావాలా?

Go lo package-level variable + init() కూడా ఒక ఎంపిక.
ఇంకా మేలైనది: dependency injection – struct lo ఒక field గా పంపడం.
Singleton test lo ఇబ్బంది పెడుతుంది – Go lo అది ఇంకా ఎక్కువ, ఎందుకంటే global state package-wide.

వివరణ

Singleton = మొత్తం program కి ఒకే ఒక్క instance. Config, logger, DB connection pool, metrics registry – ఇలాంటి వాటికి.

Go లో singleton రాయడానికి మూడు మార్గాలు:

1. **`sync.Once`** – thread-safe lazy initialization (most idiomatic). మొదటి access దగ్గర ఒక్కసారే create.
2. **Package-level var** – eager, program start లోనే. Simple కానీ lazy కాదు.
3. **`init()` function** – package load అయినప్పుడు ఒకసారి run అవుతుంది.

Go లో singleton Java కంటే simpler – private constructor, double-checked locking boilerplate అవసరం లేదు. `sync.Once` అన్నీ handle చేస్తుంది.

Real-life Scenario

> **ఒకే CEO** analogy: ఒక company కి ఒక్కడే CEO. ఎవరు అడిగినా అదే వ్యక్తి. కొత్త CEO ని create చేయవు – ఉన్నవాడినే reference చేస్తావు. `sync.Once` = "CEO ఇంకా appoint కాలేదా? అయితే ఒక్కసారి appoint చెయ్యి; అయితే ఉన్నవాడినే ఇవ్వు" – ఎంతమంది ఏకకాలంలో అడిగినా (concurrent) ఒక్కడే appoint అవుతాడు.

Code – sync.Once (idiomatic, thread-safe, lazy)

```
package config

import (
	"sync"
)

type Config struct {
	AppName string
	Port    int
}

var (
	instance *Config
	once     sync.Once // ఒక్కసారే run అయ్యేలా guarantee చేస్తుంది
)

// GetConfig — ఎంతమంది goroutines ఏకకాలంలో పిలిచినా, init ఒక్కసారే జరుగుతుంది
func GetConfig() *Config {
	once.Do(func() {
		// ఖరీదైన setup (file చదవడం, env parse) ఇక్కడ ఒక్కసారే
		instance = &Config{AppName: "MyApp", Port: 8080}
	})
	return instance
}
```

```
// వాడకం (వేరే package నుండి)
// c1 := config.GetConfig()
// c2 := config.GetConfig()
// c1 == c2  → true (అదే instance)
```

Code – Package-level var (eager) & init()

```
package logger

import "log"

// eager singleton — package load అయినప్పుడే create
var Default = &Logger{prefix: "[APP] "}

type Logger struct{ prefix string }
func (l *Logger) Info(msg string) { log.Println(l.prefix, msg) }

// init() — package load సమయంలో ఒకసారి automatic గా run అవుతుంది
```

```
func init() {
	log.SetFlags(log.LstdFlags | log.Lshortfile)
}
```

మూడు మార్గాల పోలిక

మార్గం	Lazy?	Thread-safe?	ఎప్పుడు
`sync.Once`	✅ అవును	✅ (built-in)	ఖరీదైన init, lazy కావాలి (default choice)
Package-level `var`	❌ eager	✅ (single init)	చౌక init, ఎప్పుడూ అవసరం
`init()`	❌ eager	✅ (runtime guarantees)	setup side-effects (register, flags)

Go idiom

> **Classic (Java) vs Go:** Java Singleton = private constructor + private static instance + `getInstance()` + double-checked locking with `volatile` (boilerplate చాలా). **Go లో ఇదంతా `sync.Once` ఒక్కటే solve చేస్తుంది** — race-free, lazy, 5 lines.
>
> **ఇంకా Go-er approach:** చాలామంది Go veterans అసలు singleton **వద్దు** అంటారు — బదులుగా **dependency injection**. Global singleton = hidden dependency, testing కష్టం. `Config` ని ఒకసారి `main()` లో create చేసి, అవసరమైన చోట్లకి **explicit గా pass** చెయ్యి. ఇది idiomatic Go (Section 36 DI).
>
> అంటే: `sync.Once` singleton ఎలా రాయాలో తెలియాలి, కానీ **default గా DI ఎంచుకో**.

ఎప్పుడు వాడాలి / వద్దు

- **వాడు:** genuinely global, stateless-ish resources — logger, metrics registry, connection pool, config (read-only).
- **వద్దు:** mutable shared state, business logic objects, testable units. వీటికి DI వాడు.
- **వద్దు (Go smell):** singleton వల్ల tests ఒకదానిపై ఒకటి depend అవుతున్నాయంటే — అది global state problem. Refactor to DI.

Gotchas

- **Testing నరకం:** global singleton ఉంటే tests reset చేయడం కష్టం (state leak between tests). `sync.Once` ని reset చేయలేవు.
- **Naive singleton race:** `if instance == nil { instance = ... }` (sync లేకుండా) = data race, రెండు goroutines రెండు instances create చేయొచ్చు. **ఎప్పుడూ `sync.Once` వాడు** (or package var).
- **Init order:** package-level vars, `init()` order dependencies మధ్య tricky అవ్వొచ్చు — cross-package init order మీద depend చేయకు.
- **Panic in `once.Do`:** func panic అయితే `Once` "done" అనుకుంటుంది — instance nil గా మిగలొచ్చు. Init లో error handling జాగ్రత్త.

Key Points

- Go singleton = `sync.Once` (idiomatic, lazy, thread-safe) – Java boilerplate అవసరం లేదు.
- Package-level var (eager) / `init()` కూడా options – init cost, laziness బట్టి ఎంచుకో.
- **Go veterans default గా DI prefer చేస్తారు** – singleton global state testing ని కష్టం చేస్తుంది.
- Naive `if nil` singleton = race. ఎప్పుడూ `sync.Once` లేదా package var.

12. Factory / Factory Method

వివరణ

Factory = object creation logic ని ఒకచోట centralize చేయడం. Client `new` / `&T{}` నేరుగా చేయకుండా, ఒక function/method ని అడిగి object తీసుకుంటాడు – ఏ concrete type వస్తుందో runtime లో decide అవుతుంది.

Go లో factory చాలా natural – **constructor functions (`NewX`)** అన్నీ factory లే. కీలక Go idiom: factory function ఒక **interface return** చేస్తుంది (concrete type కాదు) – ఎందుకంటే genuinely multiple types return చేస్తుంది. (ఇది "return struct" rule కి valid exception – నిజమైన polymorphic factory).

రెండు రూపాలు:

- **Simple Factory:** ఒక function, input బట్టి interface return (`func NewShape(kind string) Shape`).
- **Factory Method:** ఒక interface లో "create చేసే" method – different implementations different objects create చేస్తాయి.

Real-life Scenario

> **Restaurant kitchen** analogy: నువ్వు "dosa" order చేస్తావు (input). Kitchen (factory) ఏ pan, ఏ batter, ఎలా వండాలో అంతా లోపల handle చేసి, ready dosa (interface – తినగలిగేది) ఇస్తుంది. నువ్వు వంట details తెలుసుకోవు – order పెట్టి plate తీసుకుంటావు. కొత్త dish add చేస్తే kitchen లో change, నీ ordering process మారదు.

Code – Simple Factory (returns interface)

```
package main

import (
	"fmt"
	"strings"
)

// product interface – factory ఇదే return చేస్తుంది
type Notifier interface {
	Send(msg string) error
}
```

```go
type emailNotifier struct{ addr string } // unexported — factory ద్వారా మాత్రమే
func (e emailNotifier) Send(msg string) error {
	fmt.Printf("Email→%s: %s\n", e.addr, msg)
	return nil
}

type smsNotifier struct{ phone string }
func (s smsNotifier) Send(msg string) error {
	fmt.Printf("SMS→%s: %s\n", s.phone, msg)
	return nil
}

// FACTORY — input బట్టి సరైన Notifier (interface) return చేస్తుంది.
// concrete types unexported → client factory మీదే ఆధారపడతాడు.
func NewNotifier(kind, dest string) (Notifier, error) {
	switch strings.ToLower(kind) {
	case "email":
		return emailNotifier{addr: dest}, nil
	case "sms":
		return smsNotifier{phone: dest}, nil
	default:
		return nil, fmt.Errorf("unknown notifier: %q", kind)
	}
}

func main() {
	for _, k := range []string{"email", "sms"} {
		n, err := NewNotifier(k, "dest-123")
		if err != nil {
			fmt.Println("err:", err)
			continue
		}
		n.Send("Hello!")
	}
}
```

Code — Registry-based factory (OCP-friendly, switch తప్పించడం)

```go
// switch పెరిగితే → map[string]constructor. కొత్త type add చేయడానికి switch edit అవసరం లేదు.
type NotifierMaker func(dest string) Notifier

var registry = map[string]NotifierMaker{}

func Register(kind string, maker NotifierMaker) { registry[kind] = maker }

func Create(kind, dest string) (Notifier, error) {
	maker, ok := registry[kind]
	if !ok {
		return nil, fmt.Errorf("unknown: %s", kind)
	}
	return maker(dest), nil
}

// init() లో register — కొత్త type కి కొత్త file లో Register() చాలు (OCP)
```

```
// func init() { Register("email", func(d string) Notifier { return emailNotifier{d}
}) }
```

Factory Method (interface లో create method)

```
// Factory Method — "ఏ product create చేయాలో" subtype decide చేస్తుంది
type Dialog interface {
        CreateButton() Button // factory method
}
type Button interface{ Render() string }

type WindowsDialog struct{}
func (WindowsDialog) CreateButton() Button { return winButton{} }

type WebDialog struct{}
func (WebDialog) CreateButton() Button { return htmlButton{} }

type winButton struct{}
func (winButton) Render() string { return "[Windows Button]" }
type htmlButton struct{}
func (htmlButton) Render() string { return "<button>Web</button>" }
```

Simple Factory vs Factory Method

అంశం	Simple Factory	Factory Method
ఏమిటి	ఒక function, switch/map	interface method, ఒక్కో type overrides
Extension	function/map మార్పు	కొత్త type కొత్త CreateX() ఇస్తుంది
Go లో సాధారణం	✅ చాలా common (`NewX`)	తక్కువ (Go embedding తో అరుదు)
ఉదా	`NewNotifier("sms", ...)`	`dialog.CreateButton()`

Go idiom

> **Classic vs Go:** Java factory = abstract class `Creator` + `factoryMethod()` overridden by subclasses (inheritance heavy). **Go లో అంత ceremony అవసరం లేదు** — దాదాపు అన్ని factories కేవలం `NewX()` **functions** (idiomatic constructors). ఇవి interface return చేస్తే polymorphic factory అవుతాయి.
>
> **కీలక Go rules:**
>
> - Constructor పేరు `NewT` (single type in package) లేదా `NewThing` (multiple).
> - Genuine polymorphic factory అయితే **interface return OK** (return-struct rule కి exception).
> - Switch పెరిగితే **map registry** (OCP). కొత్త type add చేయడానికి `init()` లో `Register()`.
> - Concrete types unexported చేస్తే client factory మీదే depend అవుతాడు (encapsulation).

ఎప్పుడు వాడాలి / వద్దు

- **వాడు:** creation logic complex (validation, wiring), runtime లో type decide అవుతుంది, concrete types hide చేయాలి.
- **వద్దు:** creation trivial అయితే (`&T{}` చాలు) factory అనవసరం. Go లో "everything needs a factory" = Java smell.
- **Registry వాడు:** switch cases పెరిగినప్పుడు (plugins, many types).

Gotchas

- **Interface return + nil trap:** factory error case లో `return nil, err` OK, కానీ `return (*T)(nil), nil` typed nil అయితే `iface == nil` false → bug (Section 38).
- **Over-factory:** ఒకటే type ఎప్పటికీ ఉంటే factory అనవసరం – `&T{}` చాలు (YAGNI).
- **Registry init order:** `init()` లో register చేస్తే, import అవ్వకపోతే register అవ్వదు (blank import `_ "pkg"` అవసరం అవ్వొచ్చు).
- Factory అన్నిచోట్లా interface return చేయకు – polymorphic అయితేనే. లేదంటే return struct.

Key Points

- Go factory = `NewX()` **constructor functions** – Java abstract Creator ceremony అవసరం లేదు.
- Polymorphic factory **interface return చేయవచ్చు** ("return struct" rule కి valid exception).
- Switch పెరిగితే **map registry** (OCP) + `init()` `Register()`.
- Trivial creation కి factory వద్దు (`&T{}` చాలు) – over-factory Java smell.

13. Abstract Factory

వివరణ

Abstract Factory = "factories యొక్క factory". సంబంధిత objects యొక్క **కుటుంబాన్ని (family)** ఒకేసారి create చేస్తుంది – అవి ఒకదానితో ఒకటి match అయ్యేలా. Simple factory ఒక product ఇస్తే, abstract factory పూర్తి **matching family** ఇస్తుంది.

Go లో abstract factory = ఒక **interface** ఇందులో multiple `CreateX()` methods, ఒక్కో concrete factory ఒక family create చేస్తుంది.

Real-life Scenario

> **Furniture showroom** analogy: "Victorian style" అడిగితే – Victorian chair + Victorian table + Victorian sofa అన్నీ ఒకే style లో వస్తాయి. "Modern" అడిగితే మొత్తం modern family. Styles **mix అవ్వవు** – Victorian chair తో modern table రాదు. Abstract factory ఈ consistency guarantee చేస్తుంది.

Code

```
package main

import "fmt"

// ---- product interfaces (family members) ----
type Button interface{ Render() string }
type Checkbox interface{ Render() string }

// ---- Abstract Factory interface — పూర్తి family create చేస్తుంది ----
type GUIFactory interface {
	CreateButton() Button
	CreateCheckbox() Checkbox
}

// ---- Family 1: Light theme ----
type lightButton struct{}
func (lightButton) Render() string { return "☀️ Light Button" }
type lightCheckbox struct{}
func (lightCheckbox) Render() string { return "☀️ Light Checkbox" }

type LightFactory struct{}
func (LightFactory) CreateButton() Button     { return lightButton{} }
func (LightFactory) CreateCheckbox() Checkbox { return lightCheckbox{} }

// ---- Family 2: Dark theme ----
type darkButton struct{}
func (darkButton) Render() string { return "🌙 Dark Button" }
type darkCheckbox struct{}
func (darkCheckbox) Render() string { return "🌙 Dark Checkbox" }

type DarkFactory struct{}
func (DarkFactory) CreateButton() Button     { return darkButton{} }
func (DarkFactory) CreateCheckbox() Checkbox { return darkCheckbox{} }

// client ఒక factory తీసుకొని పూర్తి matching UI నిర్మిస్తుంది.
// Button/Checkbox ఎప్పుడూ same family — mix అవ్వవు.
func buildUI(f GUIFactory) {
	fmt.Println(f.CreateButton().Render())
	fmt.Println(f.CreateCheckbox().Render())
}

func getFactory(theme string) GUIFactory {
	if theme == "dark" {
		return DarkFactory{}
	}
	return LightFactory{}
}

func main() {
	fmt.Println("--- User dark mode ఎంచుకున్నాడు ---")
	buildUI(getFactory("dark")) // అన్నీ dark
	fmt.Println("--- User light mode ఎంచుకున్నాడు ---")
	buildUI(getFactory("light")) // అన్నీ light
}
```

Factory Method vs Abstract Factory

	Factory Method	Abstract Factory
ఏం create చేస్తుంది	ఒక్క product	Product **family** (multiple related)
ఉదా	`CreateButton()`	`CreateButton()` + `CreateCheckbox()` matched
ముఖ్యం	ఒక object type ఎంపిక	Family అంతా match అవ్వడం (consistency)
Go interface	1 method	multiple methods (ఒక్కో product కి)

Go idiom

> **Classic vs Go:** structure దాదాపు same (interface + concrete factories), కానీ Go లో inheritance లేకపోవడం వల్ల **abstract factory base class boilerplate లేదు** – కేవలం interface + structs.
>
> **Go-er alternatives:**
>
> - **Config struct + functional options** (Section 14/35) చాలాసార్లు abstract factory ని replace చేస్తుంది – theme ని ఒక option గా pass చేసి, ఒకే builder వాడొచ్చు.
> - **Factory function returning a struct of funcs:** `type GUIFactory struct { CreateButton func() Button; CreateCheckbox func() Checkbox }` – interface కంటే lightweight (closures).
> - నిజంగా family consistency guarantee ముఖ్యం అయితేనే full abstract factory. లేదంటే over-engineering.

ఎప్పుడు వాడాలి / వద్దు

- **వాడు:** ఒక product family ఎప్పుడూ కలిసి match అవ్వాలి (cross-platform UI: Windows/Mac/Linux widgets; DB drivers with matching connection+cursor+tx).
- **వద్దు:** products మధ్య family constraint లేకపోతే (independent objects) – simple factory చాలు.
- **వద్దు:** ఒకటే family ఉంటే (YAGNI) – abstract factory అనవసరం.

Gotchas

- **Family explosion:** N products × M families = N×M concrete types. Family చాలా ఉంటే maintenance భారం.
- **కొత్త product type add చేయడం కష్టం:** `GUIFactory` interface కి కొత్త method (`CreateSlider()`) add చేస్తే **అన్ని factories** update చేయాలి (OCP break for new product types – ఇది abstract factory యొక్క inherent weakness).
- **Go లో over-abstraction:** interface + 6 structs రాసేకంటే, config struct + switch తో theme handle చేయడం సులభమేమో ఆలోచించు.

Key Points

- Abstract Factory = **matching product family** ని ఒకేసారి create చేస్తుంది (consistency guarantee).
- Go = interface (multiple `CreateX` methods) + concrete factory structs; base class boilerplate లేదు.
- Go alternatives: **struct-of-funcs** లేదా **functional options** – తరచూ simpler.
- కొత్త product type add చేయడం అన్ని factories ని touch చేస్తుంది (pattern weakness). YAGNI గుర్తుంచుకో.

---

14. Builder – మరియు idiomatic Functional Options Pattern

FUNCTIONAL OPTIONS · GO యొక్క BUILDER

FUNCTIONAL OPTIONS – Go యొక్క builder

Telescoping constructor

NewServer(addr, port, timeout,
maxConn, tls, logger, ...)
7 arguments – ఏది ఏమిటో గుర్తుండదు

Functional options

NewServer(addr,
WithTimeout(5*time.Second),
WithTLS(cfg))

ఎలా పని చేస్తుంది

type Option func(*Server) – ఒక్కో option ఒక function.
NewServer(addr string, opts ...Option) – variadic గా తీసుకుని, ఒక్కొక్కటిగా apply.
డిఫాల్ట్‌లు constructor lo; caller కావలసినవి మాత్రమే మారుస్తాడు.

ఎందుకు ఇది Go lo builder కంటే మేలు

Builder కి ఒక అదనపు struct + .Build() కావాలి. Options కి function types చాలు.
కొత్త option చేర్చినా – ఉన్న callers ఏవీ మారవు (backward compatible).

వివరణ

Builder = చాలా (ముఖ్యంగా optional) parameters ఉన్న object ని **అడుగడుగునా** నిర్మించడం. Go లో constructor overloading లేదు, default parameters లేవు – కాబట్టి "10 fields ఉన్న struct ని ఎలా create చేయాలి?" అనేది real problem.

Go లో రెండు solutions:

1. **Classic Builder** – separate builder struct + chained methods (`.WithX().WithY().Build()`). Java-ish.
2. **Functional Options Pattern** – Go's **signature idiom**. Variadic `...Option` functions. Standard library, gRPC, Kubernetes అంతా ఇదే వాడతాయి. ఇది **Go లో ఖచ్చితమైన default choice**.

Functional options: `func NewServer(addr string, opts ...Option) *Server` – required args positional, optional args functions గా.

Real-life Scenario

> **Subway sandwich** analogy: bread ఎంచుకో → veggies → sauce → toast? ఒక్కో step optional, skip చేయవచ్చు. **Functional options** = నువ్వు కేవలం కావలసిన toppings మాత్రమే చెప్తావు ("extra cheese, no onion") – మిగతావి default. Sandwich maker (constructor) మిగతా అన్నీ defaults తో fill చేస్తాడు.

Code – Functional Options (THE Go way)

```
package main

import (
	"fmt"
	"time"
)

// Server — చాలా optional configs ఉన్న struct
type Server struct {
	addr    string
	port    int
	timeout time.Duration
	tls     bool
	maxConn int
}

// Option — ఒక Server ని modify చేసే function type. ఇదే pattern గుండె.
type Option func(*Server)

// ఒక్కో option ఒక closure return చేస్తుంది
func WithPort(p int) Option {
	return func(s *Server) { s.port = p }
}
func WithTimeout(d time.Duration) Option {
	return func(s *Server) { s.timeout = d }
}
func WithTLS() Option {
	return func(s *Server) { s.tls = true }
}
func WithMaxConn(n int) Option {
	return func(s *Server) { s.maxConn = n }
}

// constructor: required (addr) positional, optional variadic ...Option
func NewServer(addr string, opts ...Option) *Server {
	// 1. sensible defaults పెట్టు
	s := &Server{
		addr:    addr,
		port:    8080,
		timeout: 30 * time.Second,
		maxConn: 100,
	}
	// 2. options apply చెయ్యి (order లో)
	for _, opt := range opts {
		opt(s)
	}
```

```
        return s
}

func main() {
        // కేవలం required తో — అన్నీ defaults
        s1 := NewServer("localhost")
        fmt.Printf("%+v\n", *s1)

        // కావలసిన options మాత్రమే, ఏ order లోనైనా
        s2 := NewServer("api.x.com",
                WithPort(443),
                WithTLS(),
                WithTimeout(10*time.Second),
        )
        fmt.Printf("%+v\n", *s2)
}
```

Code – Classic Builder (comparison)

```
type HTTPRequest struct {
        method, url string
        headers     map[string]string
        body        string
}

type RequestBuilder struct{ req HTTPRequest }

func NewRequestBuilder(method, url string) *RequestBuilder {
        return &RequestBuilder{req: HTTPRequest{method: method, url: url, headers:
map[string]string{}}}
}

// chained methods — ప్రతి ఒకటి *RequestBuilder return చేస్తుంది (this-return)
func (b *RequestBuilder) Header(k, v string) *RequestBuilder { b.req.headers[k] = v;
return b }
func (b *RequestBuilder) Body(body string) *RequestBuilder   { b.req.body = body;
return b }
func (b *RequestBuilder) Build() HTTPRequest                 { return b.req }

// వాడకం:
// req := NewRequestBuilder("POST", "/api").Header("Auth", "token").Body("
{}").Build()
```

Functional Options vs Classic Builder

అంశం	Functional Options	Classic Builder
Go idiomatic?	✅✅ **YES** (stdlib, gRPC, k8s)	⚠️ works కానీ Java-ish
Boilerplate	తక్కువ (options funcs)	ఎక్కువ (builder struct + methods)
Defaults	సులభం (constructor లో)	సులభం
Extend (కొత్త option)	కొత్త `WithX` func – **OCP** ✅	కొత్త method – struct touch

అంశం	Functional Options	Classic Builder
Validation	constructor లో aggregate	Build() లో
Error handling	`Option func(*S) error` variant	Build() returns error
Readability	`NewX(addr, WithTLS())`	`.WithTLS().Build()`

Go idiom

> **Classic vs Go:** classic Builder (GoF) = separate Builder + Director + product. **Go లో దీన్ని దాదాపు ఎవరూ వాడరు** – బదులుగా **functional options**. ఇది Rob Pike / Dave Cheney popularize చేసిన Go-native idiom.
>
> **ఎందుకు functional options గొప్పవి:**
>
> - **Backward compatible:** కొత్త option add చేస్తే పాత callers break అవ్వరు (వాళ్ళు ఆ option pass చేయరు).
> - **Self-documenting:** `WithTimeout(10s)` positional `NewServer("x", 8080, 30, true, 100)` కంటే చాలా readable.
> - **Defaults easy:** constructor లో set చేసి, options వాటిని override చేస్తాయి.
> - **OCP:** కొత్త option = కొత్త function, existing code touch చేయకుండా.
>
> **Error-returning variant** (validation కోసం): `type Option func(*Server) error` – apply loop లో error check.

ఎప్పుడు వాడాలి / వద్దు

- **Functional options వాడు:** 3+ optional parameters, public API/library, future extensibility కావాలి, defaults ఉన్నాయి.
- **వద్దు (functional options):** 1-2 required fields మాత్రమే అయితే over-kill – simple `NewX(a, b)` చాలు.
- **Classic builder వాడు:** step-by-step construction genuinely stateful అయితే (SQL query builder, complex DSL), లేదా options mutate order matter చేస్తే. కానీ చాలా అరుదు.

Gotchas

- **Options order:** options sequential గా apply అవుతాయి – ఒకటి ఇంకోదాన్ని override చేయొచ్చు (`WithPort(80), WithPort(443)` → 443). Order-dependent options design smell.
- **Required vs optional confusion:** required fields ని option గా పెట్టకు – వాటిని positional args గా పెట్టు (compiler enforce చేస్తుంది).
- **Validation timing:** options apply అయ్యాక aggregate validation చెయ్యి (`if s.port < 0 { return nil, err }`) – ప్రతి option లో కాదు.
- **nil option:** variadic లో nil `Option` pass అయితే apply loop panic అవుతుంది – nil check అవసరమైతే చెయ్యి.

Key Points

- Go లో **Functional Options Pattern = default** for configurable object creation (stdlib, gRPC, k8s).
- `func NewX(required, opts ...Option)` – required positional, optional as `Option` `func(*T)` closures.
- **Backward-compatible + OCP + self-documenting + easy defaults** – అందుకే Go idiom.
- Classic Builder Go లో అరుదు – genuinely stateful step-by-step construction కి మాత్రమే. Section 35 లో full deep dive.

15. Prototype (Copy Semantics, Deep vs Shallow)

వివరణ

Prototype = ఒక existing object ని **copy (clone)** చేసి కొత్తది తయారు చేయడం – scratch నుండి create చేయడం ఖరీదైనప్పుడు, లేదా existing configuration ని base గా వాడాలనుకున్నప్పుడు.

Go లో ఇది చాలా interesting ఎందుకంటే Go కి **built-in copy semantics** ఉన్నాయి: struct ని assign/pass చేస్తే **automatic shallow copy** అవుతుంది. కానీ ఇక్కడే trap: struct లో **pointer/slice/map/channel** fields ఉంటే, అవి **shared** (reference copy) అవుతాయి – deep copy కాదు.

కాబట్టి Go లో "Prototype pattern" = `Clone()` **method** రాయడం, deep vs shallow ని consciously handle చేయడం.

Real-life Scenario

> **Resume template** analogy: నీ friend తన resume ఇచ్చాడు, నువ్వు దాన్ని copy చేసి నీ details మార్చుకుంటావు (scratch నుండి రాయవు). **Shallow copy** = photocopy తీసి, కానీ దాంట్లో "references" section అదే original folder కి link చేస్తే – నువ్వు folder మార్చితే friend ది కూడా మారుతుంది (danger). **Deep copy** = references folder కూడా విడిగా photocopy – పూర్తిగా independent.

Code – Shallow copy trap

```
package main

import "fmt"

type Config struct {
        Name    string
        Tags    []string          // slice – reference type!
        Limits  map[string]int    // map – reference type!
}

func main() {
        original := Config{
```

```
                Name:   "prod",
                Tags:   []string{"a", "b"},
                Limits: map[string]int{"cpu": 4},
        }

        // Go's automatic copy — struct assign = shallow copy
        shallow := original
        shallow.Name = "staging"              // OK — string copy అయింది
        shallow.Tags[0] = "MODIFIED"          // ⚠️ original.Tags కూడా మారింది!
        shallow.Limits["cpu"] = 99            // ⚠️ original.Limits కూడా మారింది!

        fmt.Println("original:", original) // Tags[0]=MODIFIED, cpu=99 — leaked!
        fmt.Println("shallow :", shallow)
}
```

Code — Deep copy via Clone()

```
package main

import "fmt"

type Config struct {
        Name   string
        Tags   []string
        Limits map[string]int
}

// Clone — deep copy. slice/map fields ని explicit గా copy చేస్తుంది.
func (c Config) Clone() Config {
        // value receiver → struct అప్పటికే shallow-copied. ఇప్పుడు reference fields fix
చెయ్యి.
        newTags := make([]string, len(c.Tags))
        copy(newTags, c.Tags) // built-in copy

        newLimits := make(map[string]int, len(c.Limits))
        for k, v := range c.Limits {
                newLimits[k] = v
        }

        return Config{Name: c.Name, Tags: newTags, Limits: newLimits}
}

func main() {
        original := Config{Name: "prod", Tags: []string{"a", "b"}, Limits:
map[string]int{"cpu": 4}}

        clone := original.Clone()
        clone.Tags[0] = "MODIFIED"
        clone.Limits["cpu"] = 99

        fmt.Println("original:", original) // Tags[0]=a, cpu=4 — safe! ఏమీ leak కాలేదు
        fmt.Println("clone   :", clone)    // MODIFIED, 99
}
```

Shallow vs Deep copy

అంశం	**Shallow copy (`b := a`)**	**Deep copy (`a.Clone()`)**
Value fields (int, string)	independent copy	independent copy
Slice/map/pointer	**shared** (same underlying)	independent (new allocation)
Cost	చౌక	ఖరీదు (allocations)
Safety	reference fields మారితే leak	పూర్తి safe
Go automatic	✅ (struct assign/pass)	❌ (manual `Clone()` రాయాలి)

Deep copy techniques (Go)

Technique	ఎప్పుడు
Manual `Clone()` (above)	Best – explicit, fast, type-safe (default)
`copy()` for slices	slice fields కి
`map` loop copy	map fields కి
JSON marshal/unmarshal	quick & dirty deep copy (slow, loses types)
`encoding/gob` round-trip	complex nested (slow)
Reflection-based libs	generic కానీ slow, avoid unless needed

Go idiom

> **Classic vs Go:** GoF Prototype = `Cloneable` interface + `clone()`. Java `Object.clone()` broken గా పేరు (shallow by default, checked exceptions). **Go లో pattern name అరుదుగా వాడతారు** – బదులుగా simply:
>
> - Struct value copy Go కి built-in (assignment).
> - Deep copy కావాలంటే **explicit `Clone()` method** – reference fields ని hand copy.
>
> **Go idiom:** "value semantics ని embrace చెయ్యి." చాలా Go types (small structs, no reference fields) ని copy చేయడం free మరియు safe. Reference fields (slice/map/pointer/chan) ఉంటేనే `Clone()` అవసరం. అందుకే **immutable value objects** design చేస్తే prototype అనవసరం.

ఎప్పుడు వాడాలి / వద్దు

- **వాడు:** ఖరీదైన initialization ఉన్న object ని base గా చాలాసార్లు వాడాలి (pre-configured templates, game entities, complex config).
- **వాడు:** ఒక "template" object నుండి slight variations create చేయాలి.
- **వద్దు:** creation చౌక అయితే prototype అనవసరం – కొత్తది create చెయ్యి.

- **వద్దు (accidental):** struct copy shallow అని మర్చిపోయి reference fields leak – ఇది pattern కాదు, bug.

Gotchas

- **Shallow copy silent bug:** Go struct assign shallow – slice/map/pointer shared. చాలా production bugs ఇక్కడే. `Clone()` రాయకుండా copy చేసి mutate చేస్తే original corrupt.
- **`sync.Mutex` copy:** mutex ఉన్న struct ని copy చేస్తే lock state కూడా copy – `go vet` warn. Copyable types లో mutex వద్దు.
- **Nested reference fields:** slice of structs where structs have slices – recursive deep copy అవసరం. Manual `Clone()` లో గుర్తుంచుకో.
- **JSON deep copy pitfalls:** unexported fields lose అవుతాయి, `time.Time` format issues, slow.

Key Points

- Go struct assign = **automatic shallow copy** – value fields safe, **reference fields (slice/map/pointer/chan) shared**.
- Deep copy = explicit **`Clone()` method**, reference fields ని `copy()`/loop తో hand copy.
- Shallow copy silent bug = Go లో common production bug – reference fields leak.
- **Value semantics + immutable objects** design చేస్తే prototype అవసరమే రాదు (Go idiom).

16. Object Pool (sync.Pool)

వివరణ

Object Pool = ఖరీదైన objects ని ప్రతిసారి create/destroy చేయకుండా, ఒక **pool** లో పెట్టి reuse చేయడం. అవసరం అయితే pool నుండి తీసుకో (borrow), అయిపోయాక తిరిగి పెట్టు (return). Buffers, DB connections, goroutine-heavy allocations కి.

Go లో దీనికి standard library primitive: `sync.Pool`. ఇది thread-safe, GC-aware temporary object cache. కీలకం: `sync.Pool` GC సమయంలో objects ని clear చేస్తుంది – కాబట్టి ఇది **persistent pool కాదు** (DB connection pool లాంటిది కాదు), కేవలం **allocation pressure తగ్గించే short-lived object cache**.

Real-life Scenario

> **Cricket nets ball bucket** analogy: practice కి ప్రతిసారి కొత్త balls కొనవు – ఒక bucket (pool) లో ఉన్న balls తీసుకుని, అయ్యాక తిరిగి bucket లో వేస్తావు. కొత్తవి తయారుచేసే ఖర్చు (GC pressure) తగ్గుతుంది. కానీ ఎవరో balls తీసుకెళ్ళిపోతే (GC clears), bucket ఖాళీ అయితే – కొత్తవి తయారుచేసుకుంటావు (`New`).

Code – sync.Pool for buffer reuse

```
package main

import (
	"bytes"
	"fmt"
	"sync"
)

// bufferPool – bytes.Buffer objects ని reuse చేస్తుంది (allocation తగ్గించడానికి)
var bufferPool = sync.Pool{
	// New – pool ఖాళీగా ఉంటే ఇది కొత్తది create చేస్తుంది
	New: func() any {
		fmt.Println("(కొత్త buffer create అయింది)")
		return new(bytes.Buffer)
	},
}

func buildMessage(name string) string {
	// 1. borrow – pool నుండి తీసుకో (లేకపోతే New create చేస్తుంది)
	buf := bufferPool.Get().(*bytes.Buffer)

	// 2. IMPORTANT – reuse ముందు reset చెయ్యి (పాత data ఉండొచ్చు)
	buf.Reset()

	// 3. defer తో return – పని అయ్యాక తిరిగి pool లో పెట్టు
	defer bufferPool.Put(buf)

	buf.WriteString("Hello, ")
	buf.WriteString(name)
	return buf.String()
}

func main() {
	fmt.Println(buildMessage("Ravi"))  // కొత్త buffer create
	fmt.Println(buildMessage("Kiran")) // reuse (కొత్తది create కాదు)
	fmt.Println(buildMessage("Anu"))   // reuse
}
```

Code – Custom pool (persistent, DB-connection style)

```
// sync.Pool GC-cleared కాబట్టి persistent resources (DB conns) కి channel-based pool
వాడతాం
type ConnPool struct {
	conns chan *Conn // buffered channel = fixed-size pool
}

type Conn struct{ id int }

func NewConnPool(size int) *ConnPool {
	p := &ConnPool{conns: make(chan *Conn, size)}
	for i := 0; i < size; i++ {
		p.conns <- &Conn{id: i} // pre-fill
	}
	return p
}
```

```
// Acquire — pool నుండి conn తీసుకో (ఖాళీగా ఉంటే block అవుతుంది = backpressure)
func (p *ConnPool) Acquire() *Conn { return <-p.conns }

// Release — తిరిగి పెట్టు
func (p *ConnPool) Release(c *Conn) { p.conns <- c }
```

sync.Pool vs Custom (channel) pool

అంశం	`sync.Pool`	Channel-based custom pool
ఉద్దేశం	GC pressure తగ్గించడం (temp objects)	Persistent, limited resources
GC clears?	✅ అవును (objects పోతాయి)	❌ కాదు (నువ్వు control చేస్తావు)
Fixed size?	❌ (unbounded, GC managed)	✅ (channel capacity)
Backpressure	లేదు	✅ (ఖాళీ అయితే block)
ఉదా	`bytes.Buffer`, `[]byte` scratch	DB connections, worker slots

Go idiom

> **Classic vs Go:** GoF Object Pool = manual free-list + acquire/release + synchronization. **Go లో `sync.Pool` built-in** — thread-safe, lock-free-ish, GC-integrated. కానీ ముఖ్యమైన Go-specific విషయం: `sync.Pool` **persistent pool కాదు**. GC objects ని ఎప్పుడైనా తీసేయొచ్చు.
>
> **ఎప్పుడు ఏది:**
>
> - **Short-lived, expensive-to-allocate** (buffers, encoders) → `sync.Pool`.
> - **Limited persistent resources** (DB conns, sockets, rate slots) → **buffered channel** pool.
> - నిజానికి `database/sql` package దాని own connection pool ఇస్తుంది — DB కి custom pool అరుదుగా అవసరం.
>
> **Go warning:** `sync.Pool` ని premature గా వాడకు — profiling లో allocation hot-spot కనిపిస్తేనే. Naive వాడితే code complexity పెరిగి benefit ఉండదు.

ఎప్పుడు వాడాలి / వద్దు

- **వాడు (`sync.Pool`):** hot path లో చాలా temporary allocations (JSON encoding buffers, `[]byte` scratch), GC pressure profiling లో కనిపిస్తే.
- **వాడు (channel pool):** fixed-count expensive resources (connections, licenses, worker slots).
- **వద్దు:** cheap objects కి — pool overhead > benefit. Premature optimization (profile first).
- **వద్దు (`sync.Pool` for state):** stateful objects ని Put చేసి reset మర్చిపోతే — పాత state leak అవుతుంది.

Gotchas

- **Reset before reuse:** `Get()` చేసిన object లో **పాత data ఉండొచ్చు** – వాడేముందు `Reset()`/clear చెయ్యి. మర్చిపోతే data leak/corruption bug.
- `sync.Pool` **GC-cleared:** persistent guarantee లేదు – critical resources కి వాడకు.
- **Put after Get పోగొట్టుకోవడం:** `Put()` మర్చిపోతే pool benefit లేదు (కొత్తవి create అవుతూనే ఉంటాయి). `defer Put()` idiom.
- **Pointer vs value in pool:** `sync.Pool` లో pointers store చెయ్యి (`*bytes.Buffer`) – values store చేస్తే copy overhead + interface boxing.
- **Channel pool deadlock:** అందరూ Acquire చేసి Release చేయకపోతే pool empty → forever block. Timeout/context వాడు.

Key Points

- **Object Pool** = ఖరీదైన objects reuse (create/destroy తగ్గించి GC pressure తగ్గించడం).
- Go: `sync.Pool` = temp objects (buffers) కి; **buffered channel** = persistent limited resources (conns) కి.
- `sync.Pool` **GC-cleared** – persistent కాదు; DB conns కి channel pool లేదా `database/sql`.
- **Reset before reuse** (state leak తప్పించడం) + `defer Put()` + profile-first (premature optimization వద్దు).

Part 4 – Structural Patterns

> "Objects ఎలా కలవాలి?" – types ని compose చేసి పెద్ద structures నిర్మించడం. Go లో ఇవి **embedding, interfaces, wrapping** తో natural. Decorator = `io.Reader` wrapping (Go stdlib లోనే), Proxy = interface wrapper, Facade = simple package API. చాలా వరకు Go embedding వల్ల simpler.

---

17. Adapter

వివరణ

Adapter = రెండు incompatible interfaces ని కలపడం – ఒక existing type యొక్క interface ని, client expect చేసే interface లోకి **translate** చేసే wrapper. మనం మార్చలేని 3rd-party/legacy code ని మన system కి కలపడానికి.

Go లో adapter = ఒక struct, ఇది adaptee ని embed/hold చేసి, target interface methods ని implement చేస్తుంది (internally adaptee methods కి translate చేస్తూ).

Real-life Scenario

> **Travel plug adapter** analogy: నీ Indian charger (existing) కి US socket (client interface) సరిపోదు. Plug adapter మధ్యలో ఉండి – Indian plug ని accept చేసి, US socket కి కావలసిన shape ఇస్తుంది. Charger ని మార్చలేదు, socket ని మార్చలేదు – మధ్యలో translator పెట్టాం.

Code

```
package main

import "fmt"

// ---- Target: మన system ఇది expect చేస్తుంది ----
type PaymentProcessor interface {
	Pay(amountRupees int) error
}

// ---- Adaptee: 3rd-party library, మనం మార్చలేం. వేరే signature. ----
type StripeAPI struct{}

func (StripeAPI) MakePayment(dollarsCents int64, currency string) error {
	fmt.Printf("Stripe charged %d cents %s\n", dollarsCents, currency)
	return nil
}

// ---- Adapter: StripeAPI ని PaymentProcessor లోకి translate చేస్తుంది ----
```

```go
type StripeAdapter struct {
	stripe StripeAPI // adaptee ని hold చేస్తుంది
	rate   int64     // ₹1 = ఎన్ని cents (demo)
}

func NewStripeAdapter() *StripeAdapter {
	return &StripeAdapter{rate: 120} // ₹1 ≈ 1.2 cents (demo)
}

// target method ని implement చేసి, internally adaptee call కి translate
func (a *StripeAdapter) Pay(amountRupees int) error {
	cents := int64(amountRupees) * a.rate
	return a.stripe.MakePayment(cents, "USD") // signature/units translate
}

func main() {
	// మన system కేవలం PaymentProcessor మీద depend అవుతుంది
	var processor PaymentProcessor = NewStripeAdapter()
	processor.Pay(500) // Stripe charged 60000 cents USD
}
```

Object Adapter vs "embedding" adapter

Go లో adapter రెండు రకాలుగా:

రకం	ఎలా	ఎప్పుడు
Object adapter	adaptee ని **field** గా hold	default, most flexible
Embedding adapter	adaptee ని **embed** (methods promote)	adaptee methods చాలావరకు అలాగే వాడితే, కొన్ని మాత్రమే override

```go
// embedding adapter — StripeAPI methods promote అవుతాయి, కొత్తవి మాత్రమే add
type StripeAdapter2 struct {
	StripeAPI // embedded — MakePayment promote అవుతుంది
}
func (a StripeAdapter2) Pay(rupees int) error { return
a.MakePayment(int64(rupees)*120, "USD") }
```

Go idiom

> **Classic vs Go:** structure same (wrapper), కానీ Go లో adapter చాలా common ఎందుకంటే **implicit interfaces** – నీ adapter target interface ని declare చేయనవసరం లేదు, methods match అయితే చాలు.
>
> **Function adapter (Go-special):** target ఒక single-method interface అయితే, adapter ని **function type** గా రాయవచ్చు. Stdlib `http.HandlerFunc` classic example:
>
> ```
> type Handler interface { ServeHTTP(w, r) }
> type HandlerFunc func(w, r)                          // adapter type
> func (f HandlerFunc) ServeHTTP(w, r) { f(w, r) }    // plain func ని Handler గా
> adapt చేస్తుంది
> ```
>
> ఇది plain function ని interface కి adapt చేసే idiomatic Go trick – Section 24 (Strategy) తో related.

ఎప్పుడు వాడాలి / వద్దు

- **వాడు:** 3rd-party/legacy code మార్చలేని interface ని మన contract కి కలపాలి; multiple external services ని ఒకే internal interface వెనుక ఏకం చేయాలి.
- **వాడు (function adapter):** plain func ని single-method interface కి fit చేయాలి (`http.HandlerFunc`).
- **వద్దు:** నువ్వు రెండు interfaces ని control చేస్తే – నేరుగా compatible గా design చెయ్యి, adapter అనవసరం.

Gotchas

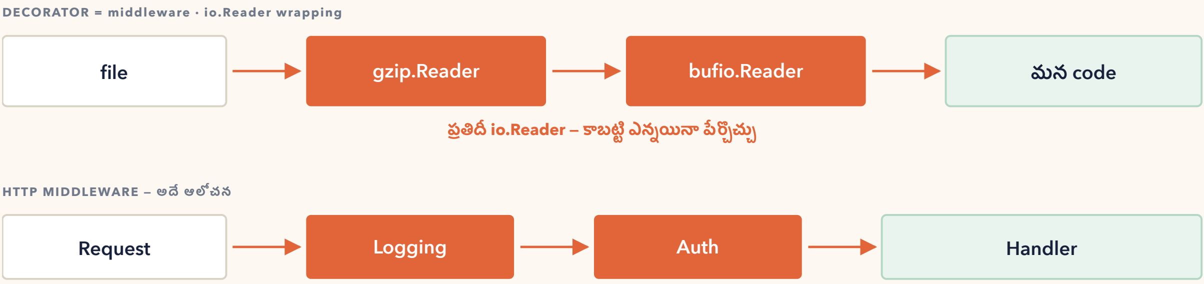
- **Leaky adapter:** adaptee errors/quirks ని adapter properly translate చేయకపోతే leak అవుతాయి (Stripe error format ని మన error కి map చేయాలి).
- **Unit/semantic mismatch:** rupees↔cents లాంటి unit conversion adapter లో miss అయితే silent money bug. Test చెయ్యి.
- **Over-adapting:** ప్రతి external call కి adapter అనవసరం – genuine interface mismatch ఉంటేనే.
- **Embedding adapter risk:** adaptee methods అన్నీ promote అవుతాయి – నీవు hide చేయాలనుకున్న methods కూడా bleak అవుతాయి. Object adapter (field) ఎక్కువ control ఇస్తుంది.

Key Points

- **Adapter** = incompatible interface ని client expect చేసే interface లోకి translate చేసే wrapper.
- Go: struct that **holds (field) లేదా embeds** adaptee + implements target interface.
- **Function adapter** (`http.HandlerFunc`) = plain func ని single-method interface కి adapt చేసే Go idiom.
- Unit/error translation జాగ్రత్త (leaky adapter); embedding adapter unwanted methods leak చేయొచ్చు.

18. Decorator (io.Reader/Writer wrapping, HTTP middleware)

DECORATOR IN GO · IO WRAPPING మరియు MIDDLEWARE



ఎందుకు ఇది Go lo ఇంత సహజం

func(http.Handler) http.Handler – ఒక handler తీసుకుని, ఇంకో handler ఇచ్చే function.
Interface ఒక్క method కాబట్టి wrap చేయడం చాలా సులభం.
ఇదే decorator pattern – కానీ Go lo దీనికి ప్రత్యేక పేరు అవసరం లేదు, అది సహజ శైలి.

వివరణ

Decorator = ఒక object కి, దాని interface మార్చకుండా, **runtime లో పొరలుగా (layers)** కొత్త behavior జోడించడం. ప్రతి decorator అదే interface ని satisfy చేస్తూ, inner object ని wrap చేసి, ముందు/తర్వాత extra పని చేస్తుంది.

Go లో decorator = **THE most natural pattern** ఎందుకంటే standard library మొత్తం ఇదే idiom మీద నిర్మించారు: `io.Reader` / `io.Writer` wrapping (`gzip.NewReader(bufio.NewReader(file))`), `http` middleware. చిన్న interfaces + composition = perfect decorator soil.

Real-life Scenario

> **Coffee + milk + sugar** analogy: base coffee (₹10). Milk decorator wrap చేస్తే coffee-with-milk (₹15). దానిపై sugar decorator wrap చేస్తే coffee-with-milk-sugar (₹17). ప్రతి పొర "నేనూ ఒక drink (same interface), కానీ inner drink + నా extra" అంటుంది. ఎన్ని పొరలైనా stack చేయవచ్చు.

Code – Classic decorator (coffee)

```
package main

import "fmt"

// interface – అన్ని decorators ఇదే satisfy చేస్తాయి
type Beverage interface {
        Cost() int
        Desc() string
}

// base component
type Coffee struct{}
func (Coffee) Cost() int    { return 10 }
```

```go
func (Coffee) Desc() string { return "Coffee" }

// decorator — Beverage ని wrap చేసి, Beverage ని satisfy చేస్తుంది
type MilkDecorator struct{ wrapped Beverage }
func (m MilkDecorator) Cost() int    { return m.wrapped.Cost() + 5 }
func (m MilkDecorator) Desc() string { return m.wrapped.Desc() + " + Milk" }

type SugarDecorator struct{ wrapped Beverage }
func (s SugarDecorator) Cost() int    { return s.wrapped.Cost() + 2 }
func (s SugarDecorator) Desc() string { return s.wrapped.Desc() + " + Sugar" }

func main() {
	var drink Beverage = Coffee{}
	drink = MilkDecorator{drink}  // పొర 1
	drink = SugarDecorator{drink} // పొర 2
	fmt.Printf("%s = ₹%d\n", drink.Desc(), drink.Cost())
	// Coffee + Milk + Sugar = ₹17
}
```

Code — io.Writer decorator (Go stdlib idiom)

```go
package main

import (
	"fmt"
	"io"
	"strings"
)

// upperWriter — io.Writer ని wrap చేసి, data ని uppercase చేసి forward చేస్తుంది
type upperWriter struct{ w io.Writer }

func (u upperWriter) Write(p []byte) (int, error) {
	return u.w.Write([]byte(strings.ToUpper(string(p))))
}

func main() {
	var out io.Writer = &strings.Builder{}
	out = upperWriter{out} // decorate — same io.Writer interface
	fmt.Fprint(out, "hello world")
	fmt.Println(out.(upperWriter).w) // (demo) → HELLO WORLD
}
```

Code — HTTP middleware (decorator, real-world Go)

```go
// http.Handler ని wrap చేసే middleware = decorator. చాలా common Go pattern.
type Handler func(msg string) string // simplified handler

func LoggingMiddleware(next Handler) Handler {
	return func(msg string) string {
		fmt.Println("→ request:", msg) // before
		res := next(msg)              // inner handler
		fmt.Println("← response:", res) // after
		return res
	}
```

```
}

func AuthMiddleware(next Handler) Handler {
        return func(msg string) string {
                if msg == "" {
                        return "401 Unauthorized"
                }
                return next(msg)
        }
}

// chaining: Logging(Auth(handler)) — పొరలు
// handler := LoggingMiddleware(AuthMiddleware(baseHandler))
```

Decorator vs Inheritance/Embedding

అంశం	Decorator (wrapping)	Embedding
When decided	**Runtime** (dynamic stacking)	Compile-time (static)
Stacking	✅ ఎన్ని పొరలైనా	limited
Interface	same interface preserved	methods promote
Go stdlib usage	`io`, `http` middleware	`sync.Mutex` embed etc.

Go idiom

> **Classic vs Go:** GoF Decorator = abstract Component + ConcreteComponent + abstract Decorator + ConcreteDecorators (heavy class tree). **Go లో ఇదంతా లేదు** – కేవలం:
>
> - Small interface (`io.Writer`, `Handler`).
> - Decorator = struct holding the interface **OR** a `func(T) T` (middleware style).
>
> Go's **middleware pattern** (`func(next Handler) Handler`) decorator యొక్క functional flavor – closures తో. ఇది HTTP servers, gRPC interceptors, gin/echo/chi అన్నిటిలో core.
>
> **Key insight:** Go small interfaces (io.Reader/Writer) decorator ని language-native చేస్తాయి – `gzip.NewWriter(bufio.NewWriter(f))` ప్రతి Go dev రోజూ రాసే decorator stack.

ఎప్పుడు వాడాలి / వద్దు

- **వాడు:** cross-cutting concerns (logging, auth, retry, compression, caching) ని core logic మార్చకుండా జోడించాలి; runtime లో behavior stack చేయాలి.
- **వాడు (middleware):** HTTP/gRPC request pipeline.
- **వద్దు:** behavior static & single అయితే – decorator అనవసరం, method లో నేరుగా రాయి.
- **వద్దు:** చాలా పొరలు debugging కష్టం చేస్తే – order matter చేస్తే జాగ్రత్త.

Gotchas

- **Order matters:** `Logging(Auth(h))` vs `Auth(Logging(h))` వేరు వేరు behavior (auth ముందా, log ముందా). Design లో explicit గా decide చెయ్యి.
- **Interface must stay small:** decorator అన్ని methods forward చేయాలి – fat interface అయితే ప్రతి decorator లో boilerplate. (ISP ఎందుకు ముఖ్యమో ఇక్కడ కనిపిస్తుంది.)
- **Lost concrete type:** wrap చేశాక inner concrete type access కష్టం (type assertion అవసరం).
- **Middleware error handling:** ఒక middleware error return చేస్తే chain ఆగాలి – explicit గా handle చెయ్యి.

Key Points

- **Decorator** = runtime లో పొరలుగా behavior జోడించడం, same interface preserve చేస్తూ.
- Go లో ఇది **most natural pattern** – `io.Reader/Writer` wrapping, HTTP middleware stdlib idiom.
- Two flavors: **struct wrapping interface** (classic) లేదా **`func(next) next`** (middleware/functional).
- Small interfaces (ISP) decorator ని easy చేస్తాయి; **order matters**, జాగ్రత్తగా stack చెయ్యి.

19. Proxy (Lazy, Protection, Remote)

వివరణ

Proxy = ఒక object కి **ప్రతినిధి (placeholder/surrogate)**. Client proxy తో మాట్లాడతాడు, proxy మధ్యలో ఉండి real object కి forward చేస్తుంది – కానీ మధ్యలో extra control జోడిస్తుంది. Decorator behavior జోడిస్తే, **proxy access ని control** చేస్తుంది (అదే interface).

మూడు రకాలు:

- **Virtual/Lazy proxy:** real object ని అవసరం వచ్చేదాకా create చేయదు (ఖరీదైన init వాయిదా).
- **Protection proxy:** access control (permissions check).
- **Remote proxy:** network అవతల ఉన్న object కి local stand-in (RPC client).
- **Caching proxy:** results cache చేసి repeat calls save చేస్తుంది.

Real-life Scenario

> **Credit card** analogy: credit card నీ bank account కి **proxy**. నువ్వు card (proxy) వాడతావు – అది మధ్యలో ఉండి "limit దాటిందా? PIN correct నా?" (protection) check చేసి, తర్వాత account (real object) నుండి డబ్బు తీస్తుంది. నువ్వు నేరుగా bank vault తో deal చేయవు.

Code – Lazy + Caching + Protection proxy

```
package main

import "fmt"
```

```go
// interface — real & proxy రెండూ satisfy చేస్తాయి
type ImageService interface {
	Load(id string) string
}

// ---- Real subject: ఖరీదైనది (network/disk) ----
type RealImageService struct{}

func (RealImageService) Load(id string) string {
	fmt.Println("(ఖరీదైన disk/network load:", id, ")")
	return "image-data-" + id
}

// ---- Proxy: lazy init + cache + access control ----
type ImageProxy struct {
	real  *RealImageService // lazy — అవసరం వచ్చేదాకా nil
	cache map[string]string // caching
	admin bool              // protection
}

func NewImageProxy(admin bool) *ImageProxy {
	return &ImageProxy{cache: map[string]string{}, admin: admin}
}

func (p *ImageProxy) Load(id string) string {
	// protection proxy
	if !p.admin && id == "secret" {
		return "403 Forbidden"
	}
	// caching proxy
	if data, ok := p.cache[id]; ok {
		fmt.Println("(cache hit:", id, ")")
		return data
	}
	// lazy proxy — real object ని ఇప్పుడే create చెయ్యి
	if p.real == nil {
		p.real = &RealImageService{}
	}
	data := p.real.Load(id)
	p.cache[id] = data
	return data
}

func main() {
	proxy := NewImageProxy(false)
	fmt.Println(proxy.Load("cat"))    // ఖరీదైన load
	fmt.Println(proxy.Load("cat"))    // cache hit (real object touch కాలేదు)
	fmt.Println(proxy.Load("secret")) // 403 Forbidden (protection)
}
```

Proxy types comparison

రకం	ఏం చేస్తుంది	Go ఉదా
Virtual/Lazy	ఖరీదైన object ని వాయిదా వేయడం	lazy DB connection, image loader

రకం	ఏం చేస్తుంది	Go ఉదా
Protection	access control	RBAC wrapper, rate limiter
Remote	network object కి local stub	gRPC/RPC client stub
Caching	results cache	memoization wrapper, CDN
Smart reference	ref counting, logging on access	instrumentation wrapper

Proxy vs Decorator vs Adapter

అంశం	Proxy	Decorator	Adapter
ఉద్దేశం	access **control**	behavior **add**	interface **translate**
Interface	same as real	same as component	**different** → target
Client aware?	usually not (transparent)	not	yes (mismatch resolve)
Stacking	సాధారణంగా single	✅ multiple layers	single

Go idiom

> **Classic vs Go:** structure same (interface + wrapper holding real object). Go లో proxy చాలా common:
>
> - `database/sql` అంతా proxy layers (driver behind interface).
> - **gRPC generated clients** = remote proxies (local method call → network RPC).
> - `context` **-aware wrappers**, rate limiters, circuit breakers = protection proxies.
>
> **Go-specific:** proxy ని **middleware/decorator తో blur** అవుతుంది (రెండూ interface wrapping). తేడా *intent* లోనే – control (proxy) vs augment (decorator). Go community తరచూ రెండింటినీ "middleware/wrapper" అనే అంటుంది.
>
> **Lazy init idiom:** `sync.Once` proxy లో lazy initialization కి perfect (thread-safe first-touch create).

ఎప్పుడు వాడాలి / వద్దు

- **వాడు:** ఖరీదైన object creation defer (lazy), access control (protection), remote calls (RPC stub), caching/memoization.
- **వద్దు:** proxy logic (cache/control) core object లోనే ఉంటే బాగుంటే – separate proxy అనవసరం.
- **వద్దు (premature):** "future లో caching అవసరం అవ్వొచ్చు" అని proxy – YAGNI. అవసరం వచ్చినప్పుడు.

Gotchas

- **Concurrency in lazy proxy:** naive `if p.real == nil { create }` = race (రెండు goroutines రెండు real objects). `sync.Once`/mutex వాడు.
- **Cache invalidation:** caching proxy లో stale data – TTL/invalidation strategy అవసరం ("two hard things in CS").
- **Transparent proxy leak:** proxy real object యొక్క అన్ని quirks (errors, panics) forward చేయాలి – లేదంటే behavior మారి LSP break.
- **Proxy vs decorator confusion:** intent clear చేసుకో – control (proxy) vs augment (decorator). పేరు కంటే purpose ముఖ్యం.

Key Points

- **Proxy** = real object కి surrogate; **access control** జోడిస్తుంది (same interface).
- రకాలు: **lazy** (defer creation), **protection** (access), **remote** (RPC stub), **caching**.
- Go: `database/sql`, gRPC clients, rate limiters – అన్నీ proxies. `sync.Once` = thread-safe lazy init.
- Decorator (augment) vs Proxy (control) – తేడా intent; concurrency & cache invalidation జాగ్రత్త.

---

20. Facade

వివరణ

Facade = సంక్లిష్టమైన subsystem కి ఒక **సులభమైన ముఖద్వారం (single entry point)**. చాలా moving parts ఉన్న system ని, client కోసం ఒక clean high-level API వెనుక దాచడం. Client 10 sub-components తో deal చేయకుండా, facade కి ఒక్క call చేస్తాడు.

Go లో facade = ఒక struct (లేదా package), ఇది multiple internal components ని hold చేసి, వాటిని orchestrate చేసే simple methods expose చేస్తుంది.

Real-life Scenario

> **Car start button** analogy: నువ్వు start button నొక్కుతావు (facade). లోపల – fuel pump on, spark plugs fire, starter motor, ECU checks – 10 complex steps జరుగుతాయి. నువ్వు వాటిని తెలుసుకోవు, ఒక్క button (simple API). Facade ఆ complexity ని దాచుతుంది.

Code

```
package main

import "fmt"

// ---- సంక్లిష్ట subsystem components (చాలా parts) ----
type inventory struct{}
func (inventory) reserve(item string) bool { fmt.Println("Inventory: reserved",
item); return true }
```

```go
type payment struct{}
func (payment) charge(amt int) bool { fmt.Println("Payment: charged", amt); return
true }

type shipping struct{}
func (shipping) dispatch(item string) { fmt.Println("Shipping: dispatched", item) }

type notification struct{}
func (notification) send(msg string) { fmt.Println("Notify:", msg) }

// ---- Facade — ఈ complexity అంతా ఒక simple API వెనుక ----
type OrderFacade struct {
	inv    inventory
	pay    payment
	ship   shipping
	notify notification
}

func NewOrderFacade() *OrderFacade { return &OrderFacade{} }

// client కి ఒక్క method — లోపల 4 subsystems orchestrate అవుతాయి
func (f *OrderFacade) PlaceOrder(item string, amount int) error {
	if !f.inv.reserve(item) {
		return fmt.Errorf("out of stock: %s", item)
	}
	if !f.pay.charge(amount) {
		return fmt.Errorf("payment failed")
	}
	f.ship.dispatch(item)
	f.notify.send("Order placed for " + item)
	return nil
}

func main() {
	facade := NewOrderFacade()
	// client కి subsystems గురించి తెలియనవసరం లేదు — ఒక్క call
	if err := facade.PlaceOrder("Laptop", 50000); err != nil {
		fmt.Println("error:", err)
	}
}
```

Facade vs Adapter vs Proxy

అంశం	Facade	Adapter	Proxy
ఉద్దేశం	complexity **simplify**	interface **translate**	access **control**
Scope	multiple subsystems	ఒక object	ఒక object
Interface	కొత్త simple API	target interface కి fit	same as real
Client	subsystems దాచబడతాయి	mismatch resolve అవుతుంది	transparent stand-in

Go idiom

> **Classic vs Go:** structure same. Go లో facade తరచూ **package boundary** గా వ్యక్తమవుతుంది – package లోని exported functions/one struct = facade, unexported internals = subsystem. ఉదా: `net/http` package `http.Get()` ఇస్తుంది – లోపల connection pooling, DNS, TLS, redirects అన్నీ దాచి.
>
> **Go-specific insight:**
>
> - Facade struct **concrete return చెయ్యి** (accept interfaces, return structs) – client fields access చేయనవసరం లేకపోతే methods మాత్రమే expose.
> - Subsystem components ని **unexported** చేస్తే facade మాత్రమే వాటిని access చేస్తుంది (encapsulation).
> - Facade ని testable చేయాలంటే subsystems ని interfaces గా inject చెయ్యి (DI + facade కలిపి).

ఎప్పుడు వాడాలి / వద్దు

- **వాడు:** complex subsystem (multiple services, libraries) కి clean API కావాలి; client ని internal complexity నుండి కాపాడాలి; layered architecture లో layer boundary.
- **వాడు:** legacy/messy code చుట్టూ clean interface (gradually refactor చేయడానికి).
- **వద్దు:** subsystem అప్పటికే simple అయితే facade అనవసరం (extra indirection).
- **వద్దు (god facade):** facade లో అన్ని operations కూరి 50-method struct చేయకు – related operations మాత్రమే.

Gotchas

- **God object risk:** facade లో ఎక్కువ responsibilities కూరితే SRP break – ఒక "OrderFacade" 30 unrelated methods ఉంటే smell.
- **Leaky facade:** subsystem details (error types, structs) facade signature లో leak అయితే abstraction విఫలం.
- **Facade ≠ excuse for tight coupling:** facade లోపల subsystems ని hard-code చేస్తే testing కష్టం – interfaces గా inject చెయ్యి.
- **Over-facading:** ప్రతి package కి facade layer = unnecessary indirection. అవసరమైనచోట.

Key Points

- **Facade** = complex subsystem కి single simple entry point (complexity దాచడం).
- Go లో facade తరచూ **package boundary** (exported API + unexported internals) – `net/http` లాంటిది.
- Subsystems ని unexported (encapsulation) లేదా interfaces గా inject (testability) చెయ్యి.
- God facade / leaky facade వద్దు – related operations, clean high-level API.

21. Composite

వివరణ

Composite = objects ని **tree structure** గా arrange చేసి, individual objects (leaf) మరియు groups (composite) ని **ఒకేలా** treat చేయడం. Client ఒక single item నా, పూర్తి tree నా – తేడా తెలియకుండా same interface వాడతాడు.

Go లో composite = ఒక interface (`Component`), leaf structs + composite struct (children slice hold చేస్తుంది), రెండూ interface satisfy చేస్తాయి. Composite operation తన children మీద recursively apply చేస్తుంది.

Real-life Scenario

> **Folder లో folder** analogy: file explorer లో – ఒక file (leaf) కి size ఉంటుంది. ఒక folder (composite) కి size = అందులో అన్ని files + subfolders sizes కలిపి. నువ్వు "size ఎంత?" అని file ని అడిగినా, folder ని అడిగినా – same question, folder recursively లోపలంతా లెక్కిస్తుంది. Tree లో ఏ node అయినా same interface.

Code

```
package main

import "fmt"

// Component — leaf & composite రెండూ ఇదే satisfy చేస్తాయి
type FileSystemNode interface {
	Size() int
	Name() string
}

// ---- Leaf ----
type File struct {
	name string
	size int
}
func (f File) Size() int    { return f.size }
func (f File) Name() string { return f.name }

// ---- Composite ----
type Folder struct {
	name     string
	children []FileSystemNode // leaves మరియు folders రెండూ
}

func (d *Folder) Add(n FileSystemNode) { d.children = append(d.children, n) }
func (d *Folder) Name() string         { return d.name }

// composite operation — children మీద recursive
func (d *Folder) Size() int {
	total := 0
	for _, child := range d.children {
```

```
            total += child.Size() // leaf నా folder నా అనవసరం — same interface
        }
        return total
}

func main() {
        root := &Folder{name: "root"}
        root.Add(File{name: "a.txt", size: 100})

        sub := &Folder{name: "sub"}
        sub.Add(File{name: "b.txt", size: 200})
        sub.Add(File{name: "c.txt", size: 50})
        root.Add(sub) // folder లో folder — tree

        fmt.Printf("%s total size = %d\n", root.Name(), root.Size()) // root total
size = 350
}
```

Leaf vs Composite

అంశం	Leaf (File)	Composite (Folder)
Children	లేవు	ఉన్నాయి (`[]Component`)
Operation	direct value	children మీద recursive aggregate
Add/Remove	meaningless	supports
Interface	same `Component`	same `Component`

Go idiom

> **Classic vs Go:** structure దాదాపు same. Go లో composite చాలా natural – interface + slice of interface. కానీ ఒక Go design question: **"Add/Remove methods ని ఎక్కడ పెట్టాలి?"**
>
> - **Transparent approach:** `Component` interface లోనే `Add()` పెడితే, leaf కి కూడా ఉంటుంది (కానీ leaf Add meaningless → panic/error). LSP risk.
> - **Safe approach (Go-preferred):** `Add()` ని composite struct మీద మాత్రమే పెట్టు (interface లో కాదు). Client composite అని తెలిస్తే type assert చేసి Add చేస్తాడు. ఇది Go idiom (safety > transparency).
>
> Real Go composites: `io.MultiWriter` (multiple writers ని ఒక writer గా), AST trees (`go/ast`), UI component trees.

ఎప్పుడు వాడాలి / వద్దు

- **వాడు:** genuine tree/hierarchy (file system, org chart, UI widgets, menu, AST, JSON/XML nodes); part-whole ని uniform గా treat చేయాలి.
- **వద్దు:** flat collection అయితే (`[]T` చాలు) composite అనవసరం.
- **వద్దు:** hierarchy లేకపోతే – bidirectional tree బలవంతం చేయకు.

Gotchas

- **Add on leaf:** `Add()` ని interface లో పెడితే leaf మీద meaningless → error/panic (LSP break). Go లో composite-only method prefer చెయ్యి.
- **Infinite recursion:** cyclic tree (child points to ancestor) → `Size()` infinite loop. Tree acyclic గా ఉంచు.
- **Deep recursion stack:** చాలా deep tree → stack overflow risk. Extreme depth ఉంటే iterative traversal.
- **Concurrent modification:** tree ని traverse చేస్తూ modify చేస్తే race – read/write lock లేదా copy.

Key Points

- **Composite** = tree structure, leaf & group ని **same interface** తో uniform treat.
- Go: `Component` interface + leaf structs + composite struct (`[]Component` children, recursive ops).
- Go idiom: **Add/Remove ని composite struct మీద మాత్రమే** (interface లో కాదు) – LSP safety.
- Real examples: `io.MultiWriter`, `go/ast`, UI trees. Cyclic/deep trees జాగ్రత్త.

22. Bridge

వివరణ

Bridge = **abstraction** ని దాని **implementation** నుండి విడదీయడం – రెండూ స్వతంత్రంగా మారగలిగేలా. రెండు dimensions స్వతంత్రంగా vary అవుతున్నప్పుడు (ఉదా: shape × rendering API), inheritance వాడితే **class explosion** (N×M classes) వస్తుంది. Bridge composition తో దాన్ని N+M కి తగ్గిస్తుంది.

Go లో bridge = abstraction struct ఒక implementation **interface** ని hold చేస్తుంది (embed కాదు – inject). Abstraction "ఏమి" చేయాలో decide చేస్తుంది, implementation "ఎలా" చేయాలో.

Real-life Scenario

> **Remote × Device** analogy: TV remote (abstraction) మరియు devices (TV, Radio, AC – implementations). నీకు BasicRemote, AdvancedRemote (2 abstractions) × TV, Radio (2 devices) ఉంటే – inheritance లో 4 classes (BasicTVRemote, AdvancedTVRemote...). Bridge తో: remote ఒక Device interface ని hold చేస్తుంది – ఏ remote అయినా ఏ device తో పని చేస్తుంది. 2+2=4 కాదు, mix & match.

Code

```go
package main

import "fmt"

// ---- Implementation interface (ఎలా) ----
```

```go
type Device interface {
	On()
	SetVolume(v int)
	Name() string
}

// ---- Concrete implementations ----
type TV struct{ volume int }
func (t *TV) On()            { fmt.Println("TV on") }
func (t *TV) SetVolume(v int) { t.volume = v; fmt.Println("TV volume:", v) }
func (t *TV) Name() string   { return "TV" }

type Radio struct{ volume int }
func (r *Radio) On()            { fmt.Println("Radio on") }
func (r *Radio) SetVolume(v int) { r.volume = v; fmt.Println("Radio volume:", v) }
func (r *Radio) Name() string   { return "Radio" }

// ---- Abstraction (ఏమి) — Device ని hold చేస్తుంది (bridge) ----
type Remote struct {
	device Device // BRIDGE — implementation ని inject
}
func (r Remote) TurnOn()          { r.device.On() }
func (r Remote) SetVolume(v int)  { r.device.SetVolume(v) }

// ---- Refined abstraction — Remote ని extend చేస్తుంది ----
type AdvancedRemote struct {
	Remote // embed base abstraction
}
func (a AdvancedRemote) Mute() { a.device.SetVolume(0); fmt.Println(a.device.Name(),
"muted") }

func main() {
	// ఏ remote అయినా ఏ device తో — mix & match (class explosion లేదు)
	basic := Remote{device: &TV{}}
	basic.TurnOn()
	basic.SetVolume(10)

	adv := AdvancedRemote{Remote{device: &Radio{}}}
	adv.TurnOn()
	adv.Mute() // Radio muted
}
```

Bridge – class explosion తగ్గింపు

Approach	2 abstractions × 3 implementations	కొత్త implementation
Inheritance	**6 classes** (2×3)	+2 classes
Bridge	**2 + 3 = 5 types**	+1 type
(10×10)	Inheritance: 100; Bridge: **20**	massive savings

Bridge vs Strategy vs Adapter

అంశం	Bridge	Strategy	Adapter
ఉద్దేశం	2 dimensions స్వతంత్రంగా vary	ఒక algorithm swap	interface translate
Structure	abstraction holds impl interface	context holds strategy	wrapper
When set	usually construction time	runtime swappable	fixed
Intent	design-time decoupling	behavior selection	compatibility

Go idiom

> **Classic vs Go:** GoF Bridge = abstract class hierarchy + implementor hierarchy. **Go లో inheritance లేదు కాబట్టి bridge చాలా natural** – abstraction struct ఒక implementation interface field hold చేస్తుంది (composition). ఇది Go లో "అదే మామూలు design" లా అనిపిస్తుంది.
>
> **Go-specific insight:** Bridge మరియు Strategy Go లో నిర్మాణపరంగా దాదాపు same (struct holds interface). తేడా **intent** లోనే: Bridge = రెండు orthogonal hierarchies ని decouple; Strategy = ఒక swappable algorithm. Go లో అనేకసార్లు "ఇది Bridge నా Strategy నా?" అని పేరు కోసం తలబద్దలు కొట్టుకోవద్దు – **interface field inject చెయ్యడమే idiom**, పేరు secondary.
>
> Go లో నిజమైన bridge: `database/sql` (abstraction) × `driver.Driver` (implementation – MySQL/Postgres/SQLite). `sql.DB` ఏ driver తోనైనా పని చేస్తుంది.

ఎప్పుడు వాడాలి / వద్దు

- **వాడు:** రెండు dimensions స్వతంత్రంగా vary అవుతాయని ముందే తెలిస్తే (shape × renderer, message × transport, DB API × driver).
- **వాడు:** class/type explosion (N×M) ని N+M కి తగ్గించాలి.
- **వద్దు:** ఒకే dimension మారుతుంటే – Strategy చాలు, bridge over-engineering.
- **వద్దు (premature):** future లో రెండో dimension వస్తుందని guess చేసి bridge – YAGNI.

Gotchas

- **Over-engineering:** bridge upfront complexity ఎక్కువ. ఒకే implementation ఉంటే అనవసరం – అవసరం స్పష్టంగా కనిపిస్తేనే.
- **Bridge vs Strategy confusion:** పేరు కంటే intent ముఖ్యం. Go లో రెండూ interface injection.
- **Deep abstraction chains:** refined abstractions ని embed చేస్తూ పోతే navigate కష్టం.
- **Implementation leak:** abstraction implementation యొక్క concrete details మీద depend అయితే bridge విఫలం – interface clean గా ఉంచు.

Key Points

- **Bridge** = abstraction ని implementation నుండి విడదీయడం; రెండూ స్వతంత్రంగా vary; class explosion (N×M → N+M).

- Go: abstraction struct ఒక implementation **interface field hold** చేస్తుంది (composition – inheritance లేదు కాబట్టి natural).
- Bridge vs Strategy = same structure, వేరే **intent** (decouple 2 hierarchies vs swap 1 algorithm). పేరుకంటే inject idiom ముఖ్యం.
- Real Go bridge: `database/sql` × `driver.Driver`. YAGNI – genuine 2-dimension variance ఉంటేనే.

23. Flyweight

వివరణ

Flyweight = వేలాది సారూప్య objects ఉన్నప్పుడు, వాటి **shared (intrinsic) state** ని ఒక్కసారే store చేసి share చేయడం – memory ఆదా చేయడానికి. ప్రతి object లో మారే భాగం (**extrinsic state**) మాత్రమే విడిగా ఉంటుంది.

- **Intrinsic state** = shared, object-independent (అడవిలో "oak చెట్టు" texture, color – అన్ని oak trees కి same).
- **Extrinsic state** = unique per object (ఆ చెట్టు position x, y – విడిగా pass).

Go లో flyweight = shared objects ని **map cache (factory)** లో store చేసి, అదే shared instance ని return చేయడం.

Real-life Scenario

> **అడవిలో చెట్లు** analogy: game లో 10 లక్షల చెట్లు render చేయాలి. ప్రతి చెట్టుకి texture, mesh, color store చేస్తే RAM పేలుతుంది. కానీ చెట్లు కేవలం 3 రకాలు (oak, pine, palm). కాబట్టి 3 tree **types** (texture/mesh) ఒక్కసారే store చేసి, ప్రతి చెట్టుకి కేవలం position (x, y) store చేస్తాం. 3 shared + 10L positions – memory ఆదా.

Code

```
package main

import "fmt"

// ---- Flyweight — shared intrinsic state (ఖరీదైన, common) ----
type TreeType struct {
	Name    string
	Color   string
	Texture string // ఊహించు — పెద్ద data (MBs)
}

func (t *TreeType) Draw(x, y int) { // extrinsic state (x, y) parameter గా వస్తుంది
	fmt.Printf("Draw %s tree at (%d,%d)\n", t.Name, x, y)
}

// ---- Flyweight Factory — shared instances ని cache చేసి reuse ----
```

```
type TreeFactory struct {
	types map[string]*TreeType // cache — ఒక్కో type ఒక్కసారే
}

func NewTreeFactory() *TreeFactory { return &TreeFactory{types:
map[string]*TreeType{}} }

func (f *TreeFactory) Get(name, color, texture string) *TreeType {
	key := name + color
	if t, ok := f.types[key]; ok {
		return t // ఉన్నదే return — కొత్తది create కాదు (shared)
	}
	t := &TreeType{Name: name, Color: color, Texture: texture}
	f.types[key] = t
	return t
}

// ---- Context — extrinsic state (unique per tree) + shared type pointer ----
type Tree struct {
	x, y     int
	treeType *TreeType // shared pointer (10L trees → 3 TreeType objects)
}

func main() {
	factory := NewTreeFactory()
	forest := []Tree{}

	// 5 trees కానీ TreeType objects కేవలం 2 (oak, pine) — shared
	for i := 0; i < 3; i++ {
		forest = append(forest, Tree{x: i, y: i, treeType: factory.Get("Oak",
"green", "oak-tex")})
	}
	for i := 0; i < 2; i++ {
		forest = append(forest, Tree{x: i, y: i, treeType:
factory.Get("Pine", "dark", "pine-tex")})
	}

	for _, t := range forest {
		t.treeType.Draw(t.x, t.y)
	}
	fmt.Println("Unique TreeType objects:", len(factory.types)) // 2 (5 trees
కాదు)
}
```

Intrinsic vs Extrinsic state

అంశం	Intrinsic (shared)	Extrinsic (unique)
Storage	flyweight object లో (ఒక్కసారే)	context లో / parameter గా
Example	tree texture, char font glyph	tree position, char position
Count	తక్కువ (types)	ఎక్కువ (instances)
Mutability	immutable (shared కాబట్టి)	per-object

Go idiom

> **Classic vs Go:** structure same (factory cache + shared objects). Go లో flyweight ఒక **map-based cache** – pointers share చేస్తాం (`*TreeType`). కీలకం:
>
> - Shared flyweight **immutable గా ఉండాలి** – ఒకరు మార్చితే అందరికీ మారుతుంది (shared pointer). Go లో unexported fields + no setters తో enforce.
> - Go **string interning** (built-in కాదు కానీ pattern), `sync.Pool` (Section 16) కొన్నిసార్లు related కానీ వేరు – flyweight = share for identity/memory, pool = reuse for allocation.
>
> **Go-specific caution:** Go చాలా memory-efficient అప్పటికే (value types, slices). Flyweight ని premature గా వాడకు – profiling లో నిజంగా memory hot-spot కనిపిస్తేనే (millions of objects). Naive flyweight complexity తెచ్చి benefit ఉండదు.

ఎప్పుడు వాడాలి / వద్దు

- **వాడు:** genuinely millions of similar objects (game entities, text editor glyphs, particle systems, map tiles); shared immutable state పెద్దది (textures, config).
- **వద్దు:** objects తక్కువ (వేలలో కాదు) – flyweight complexity worth కాదు.
- **వద్దు:** shared state mutable అయితే – flyweight dangerous (shared mutation bugs).
- **వద్దు (premature):** memory problem confirm కాకుండా – profile first.

Gotchas

- **Shared mutation bug:** flyweight mutable అయితే ఒకరు మార్చితే అన్ని contexts affect. **Immutable గా ఉంచు** (unexported + no setters).
- **Concurrency:** factory cache ని concurrent access చేస్తే map race → `sync.Mutex` / `sync.Map` వాడు.
- **Cache unbounded growth:** ఎక్కువ unique types create అయితే cache పెరిగి memory benefit పోతుంది.
- **Premature optimization:** Go already memory-efficient – flyweight worth ఉందో profile చూడు.

Key Points

- **Flyweight** = వేలాది similar objects; **intrinsic (shared, immutable)** state ఒక్కసారే, **extrinsic (unique)** విడిగా.
- Go: **map-based factory cache** + shared `*T` pointers; shared object **immutable గా ఉంచు**.
- Concurrent factory → `sync.Mutex` / `sync.Map` ; unbounded cache జాగ్రత్త.
- Go already efficient – **profile first**, premature flyweight వద్దు.

Part 5 — Behavioral Patterns

> "Objects ఎలా మాట్లాడుకోవాలి?" — responsibilities & communication. Go లో ఇవి చాలా వరకు **func types, channels, interfaces** తో idiomatic గా మారతాయి. Strategy = `func`, Observer = channels, Iterator = `range`/range-over-func, Chain of Responsibility = middleware. కొన్ని (Visitor) Go లో awkward — అది కూడా చూద్దాం.

---

24. Strategy (Func Types — Idiomatic)

STRATEGY IN GO · FUNCTION TYPES

STRATEGY — Go lo function type చాలు

Interface శైలి

type Sorter interface { Sort([]int) }
type QuickSort struct{}
func (q QuickSort) Sort(...) — struct అవసరమా?

Function type శైలి

type SortFn func([]int)
var quick SortFn = func(s []int){ ... }
Struct లేదు — function చాలు

ఎప్పుడు ఏది

Strategy కి **state అవసరం లేకపోతే** → function type. సులభం, చౌక.
State లేదా పలు methods కావాలంటే → interface.
Standard library ఉదాహరణ: http.HandlerFunc — ఒక function ని interface గా మార్చే adapter.

వివరణ

Strategy = ఒకే పనికి **swappable algorithms**, runtime లో ఏది వాడాలో decide చేయడం. If/else చెట్టు కాకుండా, ఒక్కో algorithm ని separate గా encapsulate చేసి, context కి inject చేస్తాం.

Go లో strategy కి రెండు రూపాలు:

1. **Interface-based** (classic) — strategy interface + concrete implementations.
2. **Function-type based** (idiomatic Go) — strategy కేవలం ఒక `func` **type**. ఇదే Go's signature approach — Go లో functions first-class citizens కాబట్టి, single-method interface కి బదులు plain function చాలు.

Real-life Scenario

> **Google Maps route** analogy: A నుండి B కి వెళ్ళాలి. "By car", "walking", "public transport" — మూడూ same problem (route), వేరే algorithms (strategies). నువ్వు runtime లో ఒకటి ఎంచుకుంటావు, Maps ఆ strategy తో route లెక్కిస్తుంది. కొత్త mode (cycling) add చేస్తే — కొత్త strategy, existing code మారదు.

Code – Function-type strategy (idiomatic Go)

```
package main

import "fmt"

// Strategy = ఒక func type. Interface కూడా అవసరం లేదు (single behavior).
type DiscountStrategy func(price float64) float64

// concrete strategies = plain functions
func NoDiscount(p float64) float64      { return p }
func TenPercent(p float64) float64      { return p * 0.9 }
func FestivalOffer(p float64) float64   { return p*0.75 - 50 }

// Context – strategy ని field గా hold చేస్తుంది
type Cart struct {
        items    float64
        discount DiscountStrategy // swappable
}

func (c Cart) Total() float64 {
        if c.discount == nil {
                return c.items
        }
        return c.discount(c.items) // strategy invoke – if/else లేదు
}

func main() {
        base := 1000.0
        fmt.Println(Cart{base, NoDiscount}.Total())    // 1000
        fmt.Println(Cart{base, TenPercent}.Total())    // 900
        fmt.Println(Cart{base, FestivalOffer}.Total()) // 700

        // inline strategy (closure) – Go's flexibility
        loyalty := func(p float64) float64 { return p - 100 }
        fmt.Println(Cart{base, loyalty}.Total()) // 900
}
```

Code – Interface-based strategy (when strategy has state/multiple methods)

```
// strategy కి state లేదా multiple methods ఉంటే interface వాడు
type PaymentStrategy interface {
        Pay(amount int) error
        Name() string
}

type CreditCard struct{ number string }
func (c CreditCard) Pay(a int) error { fmt.Println("CC pay", a); return nil }
func (c CreditCard) Name() string    { return "CreditCard" }

type UPI struct{ vpa string }
func (u UPI) Pay(a int) error { fmt.Println("UPI pay", a); return nil }
func (u UPI) Name() string    { return "UPI" }

// context strategy interface hold చేస్తుంది
// type Checkout struct { strategy PaymentStrategy }
```

Func-type vs Interface strategy

అంశం	Func type `func(...)`	Interface
Strategy state	లేదు (stateless) / closure	✅ struct fields
Methods count	1 (single behavior)	multiple
Boilerplate	తక్కువ (plain func)	ఎక్కువ (struct + methods)
Inline definition	✅ closures	❌ (named type అవసరం)
Go idiomatic	✅✅ (single behavior కి default)	multiple methods/state ఉంటే

Go idiom

> **Classic vs Go:** GoF Strategy = Strategy interface + concrete strategy classes + context. **Go లో single-behavior strategy కి interface అనవసరం** – `func` type చాలు. ఇది **Go's biggest simplification** over Java patterns.
>
> **నియమం:**
>
> - Strategy కి **ఒకే behavior + no state** → **`func` type** (`sort.Slice` దీనికి perfect example – comparison func).
> - Strategy కి **state లేదా multiple methods** → **interface**.
>
> Real Go strategy: `sort.Slice(s, func(i, j int) bool {...})` – comparison strategy as func. `http.HandlerFunc`, `filepath.WalkFunc` – అన్నీ func-type strategies.
>
> Strategy Go లో "if/else grows" problem కి #1 answer – Section 10 "encapsulate what varies" యొక్క implementation.

ఎప్పుడు వాడాలి / వద్దు

- **వాడు:** ఒకే operation కి multiple interchangeable algorithms (pricing, sorting, compression, routing, payment); runtime selection; "switch/if grows" smell.
- **వాడు (func type):** algorithm stateless & single – plain func.
- **వద్దు:** ఒకటే algorithm ఎప్పటికీ ఉంటే – strategy అనవసరం (YAGNI).
- **వద్దు (over-abstraction):** trivial variation కి strategy – inline చాలు.

Gotchas

- **nil strategy:** func-type strategy nil అయితే invoke చేస్తే panic – nil check (above `Total()` లో చూపినట్లు) లేదా default strategy.
- **Closures capture variables:** loop లో closure strategy create చేస్తే variable capture bug (Go 1.22 కి ముందు) – జాగ్రత్త.
- **Interface for single method = over-engineering:** Go లో single-method strategy కి interface రాయకు, func type వాడు.
- **Strategy state leak:** shared mutable strategy ని concurrent గా వాడితే race.

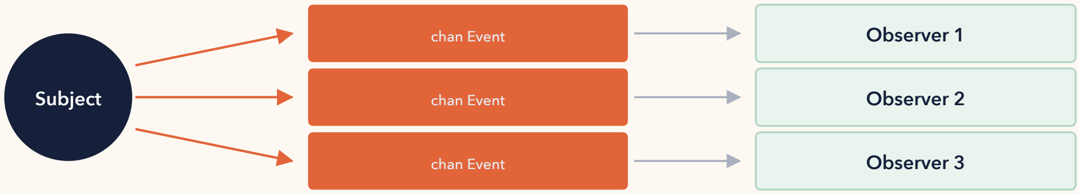
Key Points

- **Strategy** = swappable algorithms, runtime selection; "if/else grows" కి #1 answer.
- Go idiom: **single behavior** → `func` **type** (interface అనవసరం); **state/multiple methods** → **interface**.
- Real Go: `sort.Slice(..., func...)`, `http.HandlerFunc` — func-type strategies everywhere.
- nil strategy panic, closure capture bugs జాగ్రత్త; trivial variation కి over-abstract వద్దు.

25. Observer (Channels & Callbacks)

OBSERVER IN GO · CHANNELS VS CALLBACKS

OBSERVER — channels తో



Channel తో ఒక ముఖ్యమైన ప్రమాదం

ఒక observer నెమ్మదిగా ఉంటే — unbuffered channel lo publish **block** అవుతుంది.
పరిష్కారాలు: buffered channel · select తో default (drop) · ఒక్కో observer కి goroutine.
Callback శైలి అయితే ఈ సమస్య లేదు కానీ — ఒక callback panic అయితే publisher పడిపోతుంది.

వివరణ

Observer = ఒక subject యొక్క state మారినప్పుడు, దానిపై ఆధారపడిన observers అందరికీ **automatic notify** చేయడం. Subject observers list maintain చేస్తుంది; event జరిగినప్పుడు అందరికీ చెప్తుంది. Loose coupling — subject కి observers ఎవరో concrete గా తెలియనవసరం లేదు.

Go లో observer రెండు రూపాలు:

1. **Callback/interface-based** (classic) — observers ఒక interface satisfy చేస్తారు, subject list ని iterate చేసి notify.
2. **Channel-based** (idiomatic Go concurrency) — observers channels subscribe చేస్తారు, subject events channels లోకి పంపుతుంది. Goroutines తో async.

Real-life Scenario

> **YouTube subscribe** analogy: ఒక channel (subject) కి subscribers (observers). Creator కొత్త video పెట్టగానే (state change), అందరు subscribers కి notification వెళ్తుంది — creator ఒక్కొక్కరికీ manually చెప్పడు. కొత్తవాళ్ళు subscribe/unsubscribe అవ్వొచ్చు. Creator కి subscribers ఎవరో వ్యక్తిగతంగా తెలియదు.

Code – Interface-based observer

```
package main

import "fmt"

// Observer interface – notify అందుకునేవాళ్ళు
type Observer interface {
	Update(event string)
}

// Subject – observers list maintain చేస్తుంది
type Subject struct {
	observers []Observer
}

func (s *Subject) Subscribe(o Observer)   { s.observers = append(s.observers, o) }
func (s *Subject) notifyAll(event string) {
	for _, o := range s.observers {
		o.Update(event) // అందరికీ notify
	}
}
func (s *Subject) StateChanged(event string) { s.notifyAll(event) }

// ---- concrete observers ----
type EmailObserver struct{ addr string }
func (e EmailObserver) Update(event string) { fmt.Printf("Email→%s: %s\n", e.addr,
event) }

type SMSObserver struct{ phone string }
func (s SMSObserver) Update(event string) { fmt.Printf("SMS→%s: %s\n", s.phone,
event) }

func main() {
	subject := &Subject{}
	subject.Subscribe(EmailObserver{"ravi@x.com"})
	subject.Subscribe(SMSObserver{"99999"})
	subject.StateChanged("Order shipped") // అందరికీ ఒకేసారి
}
```

Code – Channel-based observer (idiomatic Go, async)

```
package main

import (
	"fmt"
	"sync"
)

type EventBus struct {
	mu   sync.RWMutex
	subs []chan string // ప్రతి observer ఒక channel
}

// Subscribe – కొత్త channel return చేస్తుంది (observer దీన్ని range చేస్తాడు)
func (b *EventBus) Subscribe() <-chan string {
	b.mu.Lock()
```

```
        defer b.mu.Unlock()
        ch := make(chan string, 10) // buffered — slow observer అందరినీ block
చేయకూడదు
        b.subs = append(b.subs, ch)
        return ch
}

// Publish — అన్ని channels లోకి event పంపుతుంది
func (b *EventBus) Publish(event string) {
        b.mu.RLock()
        defer b.mu.RUnlock()
        for _, ch := range b.subs {
                select {
                case ch <- event: // non-blocking send
                default:          // buffer full → drop (backpressure policy)
                }
        }
}

func main() {
        bus := &EventBus{}
        ch1 := bus.Subscribe()
        ch2 := bus.Subscribe()

        var wg sync.WaitGroup
        wg.Add(2)
        go func() { defer wg.Done(); fmt.Println("Observer1:", <-ch1) }()
        go func() { defer wg.Done(); fmt.Println("Observer2:", <-ch2) }()

        bus.Publish("user signed up")
        wg.Wait()
}
```

Interface vs Channel observer

అంశం	Interface/callback	Channel-based
Sync/Async	synchronous (caller blocks)	asynchronous (goroutines)
Coupling	subject holds Observer interface	subject holds channels
Backpressure	manual	buffered channel + select/default
Concurrency-safe	mutex అవసరం	channels + mutex on subs list
Go idiom	simple cases	concurrent/decoupled (idiomatic)

Go idiom

> **Classic vs Go:** GoF Observer = Observer interface + Subject with attach/detach/notify. Go లో ఇది works, కానీ **channels observer యొక్క Go-native form** – publish/subscribe with goroutines. Section 43 (Pub/Sub) దీని full case study.
>
> **నియమం:**
>
> - **Synchronous, in-process, simple** → interface-based (callbacks). `container/list` కూడా అవసరం లేదు, slice చాలు.
> - **Async, decoupled, concurrent** → channel-based event bus.
>
> **Go warning:** channel observer లో **goroutine leaks** (unsubscribe మర్చిపోతే), **slow consumer blocking** (unbuffered channel), **closed channel panic** – ఇవి common bugs. Buffered channels + `select` / `default` + proper unsubscribe అవసరం.

ఎప్పుడు వాడాలి / వద్దు

- **వాడు:** ఒక event కి multiple reactions (order placed → email + SMS + analytics + inventory); publish-subscribe; UI event handling; state change propagation.
- **వాడు (channels):** async, concurrent, decoupled subsystems.
- **వద్దు:** ఒకటే observer ఎప్పుడూ ఉంటే – direct call చాలు.
- **వద్దు (over-engineering):** simple case కి full event bus – YAGNI.

Gotchas

- **Goroutine/memory leak:** unsubscribe లేకపోతే observers/channels accumulate అవుతాయి. Explicit `Unsubscribe()` + channel close.
- **Slow observer blocks all:** unbuffered channel లో ఒక slow observer publish ని block చేస్తుంది. Buffered + `select default` (drop policy) లేదా per-observer goroutine.
- **Notification order:** slice iterate order = subscription order, కానీ దీనిపై depend చేయకు.
- **Concurrent modify:** notify చేస్తూ subscribe/unsubscribe → race. RWMutex లేదా copy-then-notify.
- **Reentrant notify:** observer తిరిగి subject ని modify చేస్తే deadlock/infinite loop.

Key Points

- **Observer** = subject state మారితే observers అందరికీ auto-notify; loose coupling.
- Go: **interface/callback** (sync, simple) లేదా **channel-based** (async, concurrent, idiomatic).
- Channel observers: buffered + `select default` (slow consumer), explicit unsubscribe (leak), RWMutex (concurrent).
- Section 43 (Pub/Sub) = full channel-based case study.

26. Command

వివరణ

Command = ఒక request/action ని **object గా** encapsulate చేయడం. అప్పుడు actions ని queue చేయవచ్చు, log చేయవచ్చు, undo/redo చేయవచ్చు, parameterize చేయవచ్చు. "పని" ని data గా మార్చి – తర్వాత execute చేయవచ్చు.

Go లో command రెండు రూపాలు:

1. **Interface-based** – `Command` interface with `Execute()` (+ `Undo()`).
2. **Func-based** – command కేవలం `func()` (closures state capture చేస్తాయి). Undo అవసరం లేకపోతే idiomatic.

Real-life Scenario

> **Restaurant order slip** analogy: waiter నీ order ని ఒక slip (command object) మీద రాస్తాడు. ఆ slip queue లోకి వెళ్తుంది, kitchen దాన్ని execute చేస్తుంది. Slip ఉంది కాబట్టి – order log చేయవచ్చు, cancel (undo) చేయవచ్చు, ఎవరైనా cook execute చేయవచ్చు. Waiter (invoker) కి వంట ఎలా చేయాలో తెలియనవసరం లేదు – slip పంపితే చాలు.

Code – Interface-based with undo

```
package main

import "fmt"

// Command — Execute + Undo
type Command interface {
        Execute()
        Undo()
}

// Receiver — actual పని చేసేది
type Light struct{ on bool }
func (l *Light) TurnOn()  { l.on = true; fmt.Println("Light ON") }
func (l *Light) TurnOff() { l.on = false; fmt.Println("Light OFF") }

// Concrete command — action ని object గా wrap చేస్తుంది
type TurnOnCommand struct{ light *Light }
func (c TurnOnCommand) Execute() { c.light.TurnOn() }
func (c TurnOnCommand) Undo()    { c.light.TurnOff() }

// Invoker — commands ని execute + history (undo కోసం)
type RemoteControl struct{ history []Command }

func (r *RemoteControl) Press(cmd Command) {
        cmd.Execute()
        r.history = append(r.history, cmd) // undo కోసం store
}
func (r *RemoteControl) UndoLast() {
```

```go
		if len(r.history) == 0 {
			return
		}
		last := r.history[len(r.history)-1]
		r.history = r.history[:len(r.history)-1]
		last.Undo()
}

func main() {
		light := &Light{}
		remote := &RemoteControl{}
		remote.Press(TurnOnCommand{light}) // Light ON
		remote.UndoLast()                  // Light OFF (undo)
}
```

Code – Func-based command (idiomatic when no undo)

```go
// command = func(). Queue of tasks — job queue, worker pool కి perfect.
type Task func()

type Queue struct{ tasks []Task }

func (q *Queue) Add(t Task)  { q.tasks = append(q.tasks, t) }
func (q *Queue) RunAll() {
		for _, t := range q.tasks {
			t() // execute
		}
}

// వాడకం:
// q.Add(func() { fmt.Println("send email") })
// q.Add(func() { fmt.Println("update DB") })
// q.RunAll()
```

Interface vs Func command

అంశం	Interface command	Func command (`func()`)
Undo/redo	✅ easy (`Undo()` method)	కష్టం (extra state అవసరం)
Serialization	✅ (struct fields marshal)	❌ (funcs serialize అవ్వవు)
Boilerplate	ఎక్కువ	తక్కువ (closures)
State capture	explicit fields	closure automatic
Go idiom	undo/logging/persistence అవసరమైతే	simple deferred execution (default)

Go idiom

> **Classic vs Go:** GoF Command = Command interface + ConcreteCommand + Invoker + Receiver (class-heavy). **Go లో undo/serialization అవసరం లేకపోతే `func()` చాలు** – closures state ని capture చేస్తాయి. ఇది Go job queues, worker pools, task schedulers లో ubiquitous.
>
> **నియమం:**
>
> - **Undo/redo, logging, serialization (persistent queue), replay** అవసరం → **interface command** (struct – fields inspect/marshal చేయవచ్చు).
> - **Simple deferred/queued execution** → `func()` (worker pool tasks).
>
> Real Go: worker pool `jobs chan func()`, `time.AfterFunc(d, func(){...})`, HTTP handler registration – అన్నీ func-commands. `sync.Once`, `defer` కూడా command spirit.

ఎప్పుడు వాడాలి / వద్దు

- **వాడు:** undo/redo (editors), job/task queues, transaction logs, macro recording, request scheduling, decoupling invoker from receiver.
- **వాడు (func):** worker pools, deferred tasks, callbacks.
- **వద్దు:** direct method call చాలు అయితే – command object అనవసరం (over-engineering).
- **వద్దు:** undo/queue/log అవసరం లేకపోతే interface command వద్దు – func చాలు.

Gotchas

- **Undo state:** undo కి command ప్రతిక్రియకి కావలసిన state (previous value) capture చేయాలి – లేకపోతే undo సరిగా పని చేయదు.
- **Closure capture bug:** loop లో `func()` commands create చేస్తే loop variable capture (Go 1.22 కి ముందు) – local copy తీసుకో.
- **Func not serializable:** persistent queue (DB/disk) కావాలంటే func command పని చేయదు – interface + marshalable fields.
- **Memory in history:** unlimited undo history = memory growth. Bounded history/circular buffer.
- **Concurrent execution:** commands ని concurrent గా run చేస్తే shared receiver race – sync అవసరం.

Key Points

- **Command** = action ని object గా encapsulate – queue, log, undo/redo, parameterize.
- Go: **interface command** (undo/serialize/log), **`func()` command** (simple queued execution – idiomatic).
- Real Go: worker pool `chan func()`, `time.AfterFunc`, task schedulers.
- Undo needs previous-state capture; func commands not serializable; closure capture bug జాగ్రత్త.

27. State (State Machine)

వివరణ

State = ఒక object యొక్క behavior దాని **internal state బట్టి** మారాలి – అది "class మార్చుకున్నట్టు" కనిపించాలి. State-dependent behavior ని పెద్ద `switch(state)` గా రాయకుండా, ఒక్కో state ని separate type గా encapsulate చేసి, object ప్రస్తుత state కి delegate చేస్తుంది.

Go లో state రెండు రూపాలు:

1. **Interface-based** (classic OOP state) – `State` interface, ఒక్కో state ఒక struct, transitions state objects మారుస్తాయి.
2. **Enum + transition table** (idiomatic for simple FSMs) – `iota` states + `map[state]map[event]state`.

Real-life Scenario

> **Traffic signal** analogy: signal Red → Green → Yellow → Red. ప్రతి state లో behavior వేరు (Red = ఆగు, Green = వెళ్ళు). Next state ప్రస్తుత state బట్టి determine అవుతుంది (Red తర్వాత Green మాత్రమే, నేరుగా Yellow కాదు). Signal object ఒకటే, కానీ దాని behavior state బట్టి మారుతుంది.

Code – Interface-based state (order lifecycle)

```
package main

import "fmt"

// State interface — ఒక్కో state ఇది satisfy చేస్తుంది
type OrderState interface {
	Next(o *Order)
	Name() string
}

type Order struct{ state OrderState }
func (o *Order) SetState(s OrderState) { o.state = s }
func (o *Order) Next()                 { o.state.Next(o) }
func (o *Order) Status() string        { return o.state.Name() }

// ---- concrete states — ఒక్కోటి తర్వాతి state కి transition define చేస్తుంది ----
type PendingState struct{}
func (PendingState) Name() string { return "Pending" }
func (PendingState) Next(o *Order) { o.SetState(ShippedState{}) }

type ShippedState struct{}
func (ShippedState) Name() string { return "Shipped" }
func (ShippedState) Next(o *Order) { o.SetState(DeliveredState{}) }

type DeliveredState struct{}
func (DeliveredState) Name() string { return "Delivered" }
func (DeliveredState) Next(o *Order) { fmt.Println("Already delivered — terminal") }
```

```
func main() {
        order := &Order{state: PendingState{}}
        fmt.Println(order.Status()) // Pending
        order.Next()
        fmt.Println(order.Status()) // Shipped
        order.Next()
        fmt.Println(order.Status()) // Delivered
        order.Next()                // Already delivered — terminal
}
```

Code – Enum + transition table (idiomatic simple FSM)

```
type State int
const (
        Pending State = iota
        Shipped
        Delivered
)

// transition table — clean, data-driven, easy to visualize
var transitions = map[State]State{
        Pending:   Shipped,
        Shipped:   Delivered,
        Delivered: Delivered, // terminal
}

func next(s State) State { return transitions[s] }
// event-based అయితే: map[State]map[Event]State
```

Interface-based vs Enum+table state

అంశం	Interface-based	Enum + transition table
State-specific behavior	✅ rich (methods per state)	limited (data only)
Transitions	scattered in state types	centralized table (easy to see)
Boilerplate	ఎక్కువ (struct per state)	తక్కువ
Complex logic per state	✅ good	awkward
Go idiom	rich behavior states	simple FSMs (default for simple)

Go idiom

> **Classic vs Go:** GoF State = State interface + ConcreteState classes + Context (delegates). Go లో interface-based works బాగా – కానీ Go embedding **no virtual dispatch** వల్ల state types base state ని embed చేసినా tricky. అందుకే states ని standalone structs గా ఉంచు.
>
> **Go's practical split:**
>
> - **Rich per-state behavior** (each state does different work: vending machine, TCP connection, game character) → **interface-based**.
> - **Simple transitions, little per-state behavior** (order status, workflow steps) → **enum + `map` transition table** (data-driven, chk in one place).
>
> Real Go: Section 44 (Vending Machine) full interface-based state case study. `net` package connection states, workflow engines.

ఎప్పుడు వాడాలి / వద్దు

- **వాడు:** object behavior దాని state బట్టి గణనీయంగా మారుతుంది; పెద్ద `switch(state)` scattered గా పెరుగుతోంది; finite states with clear transitions (vending machine, order, TCP, game AI).
- **వాడు (enum table):** simple workflows, status progression.
- **వద్దు:** states 2-3 మాత్రమే, behavior simple అయితే – bool/enum + if చాలు.
- **వద్దు (over-engineering):** trivial state కి full state pattern – YAGNI.

Gotchas

- **Invalid transitions:** state pattern valid transitions define చేయాలి – invalid ని silently allow చేయవద్దు (error/panic). Table approach ఇది clearer చేస్తుంది.
- **State explosion:** చాలా states × events = పెద్ద table/many structs. Hierarchical states ఆలోచించు.
- **Shared mutable state:** context మరియు state objects state share చేస్తే concurrency race – mutex.
- **No virtual dispatch (embedding):** base state embed చేసి override చేయాలనుకుంటే Go లో పని చేయదు – standalone state structs.
- **Terminal states:** terminal state లో `Next()` idempotent/error return చేయాలి – infinite loop వద్దు.

Key Points

- **State** = behavior state బట్టి మారుతుంది; scattered `switch(state)` ని state objects గా encapsulate.
- Go: **interface-based** (rich per-state behavior) లేదా **enum + `map` transition table** (simple FSMs – idiomatic).
- Valid transitions enforce చెయ్యి; terminal states idempotent; state explosion & concurrency జాగ్రత్త.

- Section 44 (Vending Machine) = full state case study.

28. Template Method (via Embedding + Interface)

వివరణ

Template Method = ఒక algorithm యొక్క **skeleton (steps order)** ని fix చేసి, కొన్ని steps ని subclasses/implementations override చేయనివ్వడం. "ఏమి చేయాలో" (overall flow) parent decide చేస్తుంది, "ఎలా చేయాలో" (specific steps) children.

ఇక్కడ Go's weakness బయటపడుతుంది: Java లో template method inheritance + virtual dispatch మీద ఆధారపడుతుంది – parent method child యొక్క overridden step ని call చేస్తుంది. **Go embedding లో virtual dispatch లేదు** (Section 4 gotcha) – embedded parent method child ని "చూడలేదు". కాబట్టి Go లో template method **interface injection** తో implement చేస్తాం, pure embedding తో కాదు.

Real-life Scenario

> **Tea/Coffee recipe** analogy: hot beverage recipe skeleton – (1) నీళ్ళు మరిగించు (2) పదార్థం కలుపు (3) కప్పులో పోయి (4) toppings. Steps 1, 3 అన్నిటికీ same (fixed). Step 2 tea అయితే tea leaves, coffee అయితే coffee powder (varies). Recipe order మారదు, కొన్ని steps మాత్రమే మారతాయి.

Code – Template Method via interface injection (Go-correct way)

```
package main

import "fmt"

// steps interface – varying steps మాత్రమే
type BeverageSteps interface {
	Brew()
	AddCondiments()
}

// template – algorithm skeleton (fixed order), varying steps ని inject చేస్తుంది
func MakeBeverage(steps BeverageSteps) {
	boilWater()          // fixed step
	steps.Brew()         // varying – injected
	pourInCup()          // fixed step
	steps.AddCondiments() // varying – injected
}

func boilWater() { fmt.Println("నీళ్ళు మరిగించు") }
func pourInCup() { fmt.Println("కప్పులో పోయి") }

// ---- concrete implementations ----
type Tea struct{}
func (Tea) Brew()          { fmt.Println("tea leaves steep చెయ్యి") }
```

```
func (Tea) AddCondiments() { fmt.Println("నిమ్మకాయ కలుపు") }

type Coffee struct{}
func (Coffee) Brew()          { fmt.Println("coffee powder brew చెయ్యి") }
func (Coffee) AddCondiments() { fmt.Println("పాలు + చక్కెర") }

func main() {
	fmt.Println("--- Tea ---")
	MakeBeverage(Tea{})
	fmt.Println("--- Coffee ---")
	MakeBeverage(Coffee{})
}
```

Why pure embedding FAILS (the Go gotcha)

```
// ❌ ఇలా Java-style చేస్తే పని చేయదు
type Base struct{}
func (b Base) Run()  { b.Step() }              // Base.Step() ని call చేస్తుంది
func (b Base) Step() { fmt.Println("base step") }

type Derived struct{ Base } // embed
func (d Derived) Step() { fmt.Println("derived step") } // "override"?

// Derived{}.Run() → "base step" prints! (NOT "derived step")
// ఎందుకు: Base.Run() లోని b.Step() ఎప్పుడూ Base.Step() ని call చేస్తుంది.
// Go కి virtual dispatch లేదు — embedded method outer override ని చూడలేదు.
```

Interface injection vs Embedding (for template method)

అంశం	Interface injection (✅ Go-correct)	Pure embedding (❌ broken)
Virtual dispatch	✅ (interface calls dynamic)	❌ (static, embedded wins)
"Override" works?	✅ yes	❌ no (base step always runs)
Go idiom	✅ preferred	avoid for template method
Structure	template func + steps interface	base struct + embed

Go idiom

> **Classic vs Go:** GoF Template Method = abstract class with `templateMethod()` (final) calling abstract `step()` methods (overridden by subclasses). **Go లో ఇది directly translate అవ్వదు** – no inheritance, no virtual dispatch.
>
> **Go's answer:** Template Method ని **Strategy తో replace చెయ్యి**. Fixed skeleton ఒక function/method గా, varying steps ఒక **interface parameter** గా (లేదా func types). ఇది "hollywood principle" (don't call us, we'll call you) ని interface injection తో achieve చేస్తుంది.
>
> చాలామంది Go veterans అంటారు: **"Template Method Go లో అవసరం లేదు – Strategy/composition చాలు."** అంటే ఇది patterns లో Go కి least natural ఒకటి. దీన్ని force చేయకు – steps ని interface గా inject చేయడమే idiomatic.

ఎప్పుడు వాడాలి / వద్దు

- **వాడు:** algorithm skeleton fixed, కొన్ని steps మాత్రమే vary (data processing pipelines: parse → validate → transform → save; test setup/teardown; report generation).
- **వాడు (Go way):** ఎప్పుడూ **interface injection** గా, pure embedding గా కాదు.
- **వద్దు:** steps అన్నీ vary అయితే – Strategy మొత్తం behavior కి.
- **వద్దు (embedding):** template method కి embedding "override" ని నమ్మకు – silent bug.

Gotchas

- **No virtual dispatch (THE big one):** embedded base method child's overridden step ని call చేయదు – always base ది. Interface injection తప్పనిసరి.
- **Interface satisfaction:** injected type అన్ని steps implement చేయాలి – లేకపోతే compile error (ఇది good, Java's runtime failure కంటే).
- **Over-abstraction:** 2-step template కి interface + implementations = over-engineering. Simple అయితే inline.
- **Hook methods:** optional steps (default behavior) కావాలంటే interface లో default provide చేయడం Go లో కష్టం (embed a default struct).

Key Points

- **Template Method** = algorithm skeleton fixed, కొన్ని steps override.
- **Go gotcha:** pure embedding template method **broken** (no virtual dispatch – embedded base always wins).
- **Go-correct: interface injection** – skeleton func + varying steps as interface parameter (≈ Strategy).
- Go veterans: "Template Method అవసరం లేదు, Strategy/composition చాలు" – force చేయకు.

29. Iterator (Channels, Go 1.23 Range-over-func)

వివరణ

Iterator = ఒక collection యొక్క internal structure ని expose చేయకుండా, దాని elements ని sequence గా traverse చేయడం. Client "next element ఏమిటి?" అని అడుగుతాడు, collection ఎలా stored అయిందో తెలియనవసరం లేదు.

Go లో iteration మూడు రూపాలు:

1. **Built-in `range`** — slices, maps, channels, strings కి native. చాలా cases దీంతోనే solve అవుతాయి — custom iterator అనవసరం.
2. **Channel-based iterator** — goroutine values ని channel లోకి పంపుతుంది, client `range` చేస్తుంది. Lazy, కానీ goroutine cost.
3. **Go 1.23 range-over-func (`iter.Seq`) — కొత్త idiomatic way**. Iterator ఒక function గా, `for x := range myIterator` నేరుగా వాడవచ్చు.

Real-life Scenario

> **TV remote channel up** analogy: నువ్వు "next channel" button నొక్కుతావు — TV internally channels ఎలా stored అయ్యాయో (frequency table, list) తెలియనవసరం లేదు. Remote ఒక uniform "next" interface ఇస్తుంది. Iterator అలాగే — collection internals దాచి, "next" ఇస్తుంది.

Code — Channel-based iterator (pre-1.23 idiom)

```
package main

import "fmt"

type Tree struct {
	value       int
	left, right *Tree
}

// InOrder — channel-based iterator. goroutine values push చేస్తుంది.
func (t *Tree) InOrder() <-chan int {
	ch := make(chan int)
	go func() {
		defer close(ch) // ముఖ్యం — లేకపోతే range forever block
		var walk func(*Tree)
		walk = func(n *Tree) {
			if n == nil {
				return
			}
			walk(n.left)
			ch <- n.value // lazy — consumer receive అయ్యేదాకా block
			walk(n.right)
		}
		walk(t)
```

```
	}()
	return ch
}

func main() {
	root := &Tree{2, &Tree{1, nil, nil}, &Tree{3, nil, nil}}
	for v := range root.InOrder() { // client కి tree structure తెలియదు
		fmt.Print(v, " ") // 1 2 3
	}
	fmt.Println()
}
```

Code – Go 1.23 range-over-func (THE new idiom)

```
package main

import (
	"fmt"
	"iter" // Go 1.23+
)

type Stack[T any] struct{ items []T }

func (s *Stack[T]) Push(v T) { s.items = append(s.items, v) }

// All — iter.Seq[T] return చేస్తుంది. range-over-func తో వాడవచ్చు.
// yield(v) false return చేస్తే consumer early break చేసినట్టు — iteration ఆగుతుంది.
func (s *Stack[T]) All() iter.Seq[T] {
	return func(yield func(T) bool) {
		for i := len(s.items) - 1; i >= 0; i-- { // top నుండి
			if !yield(s.items[i]) {
				return // consumer break చేశాడు — cleanup ఇక్కడ
			}
		}
	}
}

func main() {
	s := &Stack[int]{}
	s.Push(1); s.Push(2); s.Push(3)

	// range-over-func — no goroutine, no channel, lazy, breakable
	for v := range s.All() {
		fmt.Print(v, " ") // 3 2 1
		if v == 2 {
			break // yield false → iterator cleanly stops
		}
	}
	fmt.Println() // 3 2
}
```

Three iteration approaches

Approach	Lazy?	Cost	Break-safe	Go version
Built-in `range`	–	zero	✅	always
Channel iterator	✅	goroutine + chan	⚠️ leak risk	always
range-over-func (`iter.Seq`)	✅	closure (cheap)	✅ (yield false)	**1.23+**

Go idiom

> **Classic vs Go:** GoF Iterator = Iterator interface (`hasNext()` , `next()`) + concrete iterators. **Go లో దాదాపు ఎప్పుడూ `range` చాలు** – custom iterator అరుదు.
>
> **Evolution:**
>
> - **మామూలు collections** (slice, map) → just `range` . Custom iterator అనవసరం.
> - **Pre-1.23 custom iteration** → channel-based (కానీ goroutine leak risk if consumer breaks early – channel never drained).
> - **Go 1.23+ → range-over-func** (`iter.Seq[T]` , `iter.Seq2[K,V]`). Channel goroutine cost లేకుండా, break-safe, lazy. ఇది **కొత్త idiomatic iterator**. Standard library `maps.Keys()` , `slices.All()` ఇలానే return చేస్తాయి.
>
> **నియమం:** కొత్త code లో custom iteration కావాలంటే **range-over-func** (1.23+); channel iterator ని fan-out/concurrency అవసరమైతేనే.

ఎప్పుడు వాడాలి / వద్దు

- **వాడు (`range`):** ఏ built-in collection అయినా – default.
- **వాడు (range-over-func):** custom collections (trees, graphs, paginated APIs, generators), lazy sequences, Go 1.23+.
- **వాడు (channel):** concurrent producers, పని generation తో overlap అవ్వాలి, fan-out.
- **వద్దు:** slice ఉంటే custom iterator రాయకు – `range` చాలు.

Gotchas

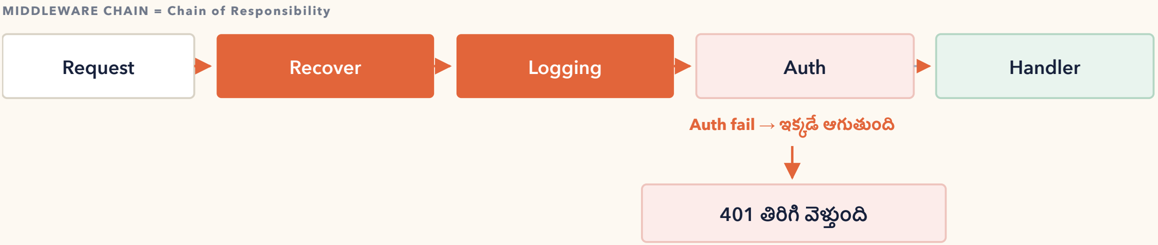
- **Channel iterator leak (BIG):** consumer early `break` చేస్తే, goroutine `ch <- v` దగ్గర **forever block** (leak). range-over-func దీన్ని solve చేస్తుంది (yield false → cleanup).
- **`close(ch)` మర్చిపోవడం:** channel close చేయకపోతే `range` forever block. `defer close(ch)` .
- **range-over-func requires Go 1.23:** older versions లో available కాదు.
- **yield return value:** range-over-func లో `yield` return false ని ignore చేస్తే break పని చేయదు – always check.
- **Map iteration order random:** `range` over map order guaranteed కాదు – sorted కావాలంటే keys extract + sort.

Key Points

- Go: **built-in** `range` చాలా cases కి చాలు – custom iterator అరుదు.
- **Channel iterator** = lazy కానీ goroutine leak risk (consumer early break → blocked goroutine).
- **Go 1.23 range-over-func (** `iter.Seq[T]` **)** = కొత్త idiomatic iterator – cheap, break-safe, `for x := range myIter`.
- Map `range` order random; channel iterator `defer close(ch)` తప్పనిసరి.

30. Chain of Responsibility (Middleware Chains)

CHAIN OF RESPONSIBILITY · MIDDLEWARE CHAINS



ఇది నిజమైన CoR – ఎందుకంటే

ప్రతి middleware "నేను handle చేసి ఆపేయాలా, లేక next కి పంపాలా" అని నిర్ణయిస్తుంది.
Auth fail అయితే – next ని పిలవకుండా response రాసి ఆగిపోతుంది.
Logging lo ఇది fan-out (అందరూ చూడాలి) – అది chain కాదు. ఈ తేడా ముఖ్యం.

వివరణ

Chain of Responsibility (CoR) = ఒక request ని **handlers గొలుసు** గుండా పంపడం. ప్రతి handler "నేను దీన్ని handle చేయగలనా?" అని చూసి, చేస్తే చేస్తుంది, లేదా next handler కి pass చేస్తుంది. Sender కి ఏ handler process చేస్తుందో తెలియనవసరం లేదు.

Go లో CoR యొక్క అత్యంత common form = **HTTP middleware chains**. ప్రతి middleware request ని process చేసి (auth, logging, rate limit), next కి pass చేస్తుంది. ఇది Go web frameworks అన్నిటి గుండె.

Real-life Scenario

> **Customer support escalation** analogy: నీ problem L1 support కి వెళ్తుంది. L1 solve చేయగలిగితే చేస్తాడు; లేకపోతే Manager (L2) కి pass. Manager కూడా చేయలేకపోతే Director (L3). ప్రతి level "నేను handle చేయగలనా?" అని చూసి, లేకపోతే పైకి pass. నువ్వు (sender) ఎవరు solve చేస్తారో తెలియనవసరం లేదు.

Code – Classic CoR (approval chain)

```
package main

import "fmt"

// Handler — next కి reference + handle logic
type Approver interface {
```

```go
	SetNext(Approver)
	Approve(amount int)
}

type base struct{ next Approver }
func (b *base) SetNext(n Approver) { b.next = n }
func (b *base) passToNext(amount int) {
	if b.next != nil {
		b.next.Approve(amount)
	} else {
		fmt.Println("ఎవరూ approve చేయలేకపోయారు:", amount)
	}
}

type TeamLead struct{ base }
func (t *TeamLead) Approve(amount int) {
	if amount <= 1000 {
		fmt.Println("TeamLead approved:", amount)
		return
	}
	t.passToNext(amount) // నా limit దాటింది → pass
}

type Manager struct{ base }
func (m *Manager) Approve(amount int) {
	if amount <= 10000 {
		fmt.Println("Manager approved:", amount)
		return
	}
	m.passToNext(amount)
}

type Director struct{ base }
func (d *Director) Approve(amount int) {
	fmt.Println("Director approved:", amount)
}

func main() {
	tl, mgr, dir := &TeamLead{}, &Manager{}, &Director{}
	tl.SetNext(mgr)
	mgr.SetNext(dir)

	tl.Approve(500)    // TeamLead
	tl.Approve(5000)   // Manager
	tl.Approve(100000) // Director
}
```

Code — Middleware chain (idiomatic Go)

```go
// middleware = func(next Handler) Handler — CoR యొక్క Go form
type Handler func(req string) string

func chain(h Handler, middlewares ...func(Handler) Handler) Handler {
	// reverse order — first middleware outermost
	for i := len(middlewares) - 1; i >= 0; i-- {
		h = middlewares[i](h)
	}
```

```
        return h
}

func Logging(next Handler) Handler {
        return func(req string) string {
                fmt.Println("log:", req)
                return next(req)
        }
}
func Auth(next Handler) Handler {
        return func(req string) string {
                if req == "" {
                        return "401" // chain break — next కి వెళ్ళదు
                }
                return next(req)
        }
}

// final := chain(baseHandler, Logging, Auth)
// final("GET /") → log → auth → baseHandler
```

CoR (classic) vs Middleware chain

అంశం	Classic CoR	Middleware chain
Structure	handlers with `next` field	`func(next) next` closures
Who handles	first capable handler stops	అందరూ process చేస్తారు (usually)
Go idiom	rare	✅✅ ubiquitous (HTTP/gRPC)
Short-circuit	handle & return	early return (auth fail)

Go idiom

> **Classic vs Go:** GoF CoR = Handler base class + successor link. Go లో classic form అరుదు — బదులుగా **middleware pattern** (`func(next Handler) Handler`) prevails. ఇది decorator (Section 18) కి చాలా దగ్గర — తేడా intent: CoR = request pipeline / conditional handling; decorator = behavior layering. Go లో రెండూ same middleware form.
>
> Real Go: `net/http` middleware, gRPC interceptors, chi/gin/echo middleware, `io` pipelines. ప్రతి Go web app CoR వాడుతుంది (తెలియకుండానే).
>
> **నియమం:** request pipeline (auth → log → rate-limit → handler) → **middleware chain**. "first-capable-handler" semantics (ఒక్కడే handle) → classic CoR with early return.

ఎప్పుడు వాడాలి / వద్దు

- **వాడు:** request processing pipeline (HTTP/gRPC middleware); escalation/approval chains; event handling where multiple potential handlers; filters.
- **వాడు (middleware):** cross-cutting concerns in a pipeline.
- **వద్దు:** ఒకటే handler ఎప్పుడూ — direct call.

- **వద్దు:** chain order fragile & hard to reason → simplify.

Gotchas

- **Chain order matters:** auth ముందా, rate-limit ముందా – order behavior మారుస్తుంది. Explicit గా define చెయ్యి.
- **Unhandled request:** ఏ handler handle చేయకపోతే? Default/terminal handler పెట్టు (silent drop వద్దు).
- **Middleware forgetting `next()`:** middleware `next(req)` call చేయకపోతే chain ఆగిపోతుంది (silent bug) – intentional short-circuit మాత్రమే.
- **Nil next:** classic CoR లో last handler `next` nil – nil check అవసరం (panic వద్దు).
- **Context propagation:** Go middleware లో `context.Context` ని next కి pass చేయాలి (cancellation, values).

Key Points

- **CoR** = request ని handlers గొలుసు గుండా; ప్రతి handler process చేస్తుంది లేదా next కి pass.
- Go idiom = **middleware chain** (`func(next Handler) Handler`) – HTTP/gRPC ubiquitous.
- Decorator కి దగ్గర; తేడా intent (pipeline/conditional vs behavior layering).
- Chain order matters; unhandled request కి terminal handler; middleware `next()` call మర్చిపోవద్దు; `context` propagate చెయ్యి.

31. Mediator

వివరణ

Mediator = అనేక objects ఒకదానితో ఒకటి **నేరుగా** మాట్లాడకుండా, ఒక central **mediator** గుండా communicate చేయడం. Many-to-many "గజిబిజి spaghetti" communication ని, mediator ద్వారా star topology కి మారుస్తుంది. Objects mediator ని మాత్రమే తెలుసుకుంటాయి, ఒకదానికొకటి కాదు.

Go లో mediator = ఒక struct/interface ఇది participants ని register చేసి, వాటి మధ్య messages route చేస్తుంది.

Real-life Scenario

> **Air Traffic Control (ATC) tower** analogy: విమానాలు (colleagues) ఒకదానితో ఒకటి నేరుగా మాట్లాడవు ("నువ్వు ఎక్కడ ఉన్నావు?" అని pilot pilot ని అడగడు) – ఇది chaos అవుతుంది. అన్నీ ATC tower (mediator) తో మాట్లాడతాయి. Tower అందరి positions తెలుసుకుని coordinate చేస్తుంది. కొత్త విమానం వస్తే tower తో register అవుతుంది, మిగతా విమానాలతో నేరుగా కాదు.

Code

```
package main

import "fmt"

// Mediator — participants మధ్య messages route చేస్తుంది
type ChatRoom interface {
	Broadcast(from string, msg string)
	Register(u *User)
}

// concrete mediator
type ChatRoomImpl struct {
	users map[string]*User
}

func NewChatRoom() *ChatRoomImpl { return &ChatRoomImpl{users: map[string]*User{}} }

func (c *ChatRoomImpl) Register(u *User) {
	c.users[u.name] = u
	u.room = c // user కి mediator reference
}

func (c *ChatRoomImpl) Broadcast(from, msg string) {
	for name, u := range c.users {
		if name != from { // sender కి తప్ప అందరికీ
			u.receive(from, msg)
		}
	}
}

// Colleague — mediator తో మాత్రమే మాట్లాడుతుంది, ఇతర users తో నేరుగా కాదు
type User struct {
	name string
	room ChatRoom
}

func (u *User) Send(msg string) { u.room.Broadcast(u.name, msg) } // mediator కి
func (u *User) receive(from, msg string) {
	fmt.Printf("[%s's screen] %s: %s\n", u.name, from, msg)
}

func main() {
	room := NewChatRoom()
	ravi := &User{name: "Ravi"}
	kiran := &User{name: "Kiran"}
	anu := &User{name: "Anu"}
	room.Register(ravi)
	room.Register(kiran)
	room.Register(anu)

	ravi.Send("అందరికీ హాలో!") // Ravi mediator కి పంపాడు; అది Kiran, Anu కి route
చేసింది
}
```

Without mediator vs With mediator

అంశం	Direct (no mediator)	With mediator
Connections	N objects → N×(N-1) links	N objects → N links (to mediator)
Coupling	tight (everyone knows everyone)	loose (only mediator)
కొత్త object	అందరినీ update చేయాలి	mediator తో register చాలు
Complexity	objects లో scattered	centralized in mediator

Mediator vs Observer vs Facade

అంశం	Mediator	Observer	Facade
Direction	many-to-many (bidirectional)	one-to-many (subject→obs)	client→subsystem
Purpose	coordinate peers	notify dependents	simplify access
Who knows	peers know mediator only	subject knows observers	client knows facade

Go idiom

> **Classic vs Go:** structure same. Go లో mediator తరచూ **channels తో** implement అవుతుంది – central goroutine (mediator) messages route చేస్తుంది, participants channels తో దానితో మాట్లాడతాయి. ఇది "sharing memory by communicating" Go proverb కి సరిపోతుంది.
>
> **Go forms:**
>
> - **Interface/struct mediator** (above) – synchronous, simple.
> - **Channel-based hub** – central goroutine `select` తో అన్ని participant channels నుండి receive చేసి route చేస్తుంది (WebSocket chat servers ఇలానే).
>
> **Go caution:** mediator **god object** అవ్వొచ్చు – అన్ని logic దానిలో centralize అయితే SRP break. Coordination మాత్రమే mediator లో, business logic participants లో.

ఎప్పుడు వాడాలి / వద్దు

- **వాడు:** many objects complex గా interact అవుతాయి (chat rooms, UI dialogs with interdependent widgets, game entity coordination, workflow orchestration); N×N coupling తగ్గించాలి.
- **వాడు (channels):** concurrent participants (WebSocket hub, actor-like systems).
- **వద్దు:** objects తక్కువ / simple interaction – direct calls చాలు.
- **వద్దు:** mediator god object అవుతుంటే – responsibilities split చెయ్యి.

Gotchas

- **God object:** mediator లో అన్ని logic కూరితే unmaintainable. Coordination మాత్రమే, business logic కాదు.
- **Single point of failure/bottleneck:** అన్ని communication mediator గుండా – channel-based లో ఇది throughput bottleneck అవ్వచ్చు.
- **Concurrent access:** interface mediator లో participants map ని concurrent access చేస్తే race – mutex లేదా channel-based hub.
- **Tight mediator coupling:** participants mediator concrete type మీద depend అయితే – interface వాడు.

Key Points

- **Mediator** = objects నేరుగా కాకుండా central mediator గుండా communicate; N×N → N coupling.
- Go: interface/struct mediator (sync) లేదా **channel-based hub** (concurrent – WebSocket chat).
- Mediator = coordinate peers (bidirectional); Observer = notify dependents (one-way).
- **God object** risk – coordination మాత్రమే mediator లో; concurrent map access → mutex/channels.

32. Memento

వివరణ

Memento = ఒక object యొక్క internal state ని **snapshot** గా capture చేసి, encapsulation break చేయకుండా, తర్వాత ఆ state కి **restore** చేయడం. Undo/redo, checkpoints, save/load కి. కీలకం: state snapshot object's internals ని బయటికి leak చేయకూడదు.

Go లో memento = ఒక struct (snapshot), originator `Save()` (memento create) మరియు `Restore(memento)` methods, caretaker mementos list maintain చేస్తుంది.

Real-life Scenario

> **Game save point** analogy: పెద్ద boss fight ముందు నువ్వు game save చేస్తావు (memento). చనిపోతే, ఆ save point కి తిరిగి వెళ్తావు (restore). Save file లో నీ health, position, inventory అన్నీ ఉంటాయి – కానీ నువ్వు ఆ file ని edit చేయవు (encapsulation preserved), game మాత్రమే దాన్ని create/restore చేస్తుంది.

Code

```
package main

import "fmt"
```

```go
// Memento — state snapshot. fields unexported → outside modify చేయలేదు
(encapsulation).
type Memento struct {
	content string
}

// Originator — state ఉన్న object
type Editor struct {
	content string
}

func (e *Editor) Type(text string) { e.content += text }
func (e *Editor) Content() string   { return e.content }

// Save — ప్రస్తుత state ని memento గా capture
func (e *Editor) Save() Memento { return Memento{content: e.content} }

// Restore — memento నుండి state తిరిగి పెట్టు
func (e *Editor) Restore(m Memento) { e.content = m.content }

// Caretaker — mementos history maintain చేస్తుంది (memento internals చూడదు)
type History struct {
	mementos []Memento
}

func (h *History) Push(m Memento) { h.mementos = append(h.mementos, m) }
func (h *History) Pop() (Memento, bool) {
	if len(h.mementos) == 0 {
		return Memento{}, false
	}
	last := h.mementos[len(h.mementos)-1]
	h.mementos = h.mementos[:len(h.mementos)-1]
	return last, true
}

func main() {
	editor := &Editor{}
	history := &History{}

	editor.Type("Hello ")
	history.Push(editor.Save()) // checkpoint 1
	editor.Type("World")
	history.Push(editor.Save()) // checkpoint 2
	editor.Type("!!!")
	fmt.Println(editor.Content()) // Hello World!!!

	// undo (last checkpoint కి restore)
	if m, ok := history.Pop(); ok {
		editor.Restore(m)
	}
	fmt.Println(editor.Content()) // Hello World
}
```

Roles in Memento

Role	ఏం చేస్తుంది	Go
Originator	state ఉన్న object; Save/Restore	`Editor`
Memento	immutable state snapshot	`Memento` struct (unexported fields)
Caretaker	mementos store చేస్తుంది (internals చూడదు)	`History`

Go idiom

> **Classic vs Go:** GoF Memento Java లో access control tricks (nested classes, package-private) తో originator మాత్రమే memento internals చూడగలిగేలా చేస్తుంది. **Go లో ఇది packages + unexported fields తో natural** – memento struct ని originator package లో పెట్టి fields unexported చేస్తే, caretaker (వేరే package) memento ని hold చేయగలడు కానీ దాని internals చూడలేడు. Perfect encapsulation.
>
> **Go-specific:**
>
> - **Deep copy జాగ్రత్త (Section 15):** state లో slice/map/pointer ఉంటే memento shallow copy – original మారితే memento కూడా మారుతుంది (bug). `Save()` లో deep copy చెయ్యి.
> - **Value memento immutable:** memento ని value గా store చేస్తే (pointer కాదు) accidental mutation తప్పుతుంది.
> - Simple cases లో pattern name overkill – just "snapshot the state" (struct copy).

ఎప్పుడు వాడాలి / వద్దు

- **వాడు:** undo/redo (editors, drawing apps), transaction rollback, game save/load, checkpoints, "cancel changes" dialogs.
- **వద్దు:** state చాలా పెద్దది & snapshots తరచూ → memory blowup (command pattern with reverse ops ఆలోచించు).
- **వద్దు:** simple undo (ఒక్క value) కి full memento – variable లో పాత value store చాలు.

Gotchas

- **Shallow copy bug (BIG):** state లో reference fields (slice/map/pointer) ఉంటే memento shared – original మారితే memento corrupt. **Deep copy in** `Save()`.
- **Memory blowup:** ప్రతి change కి full state snapshot = memory పేలుతుంది. Bounded history, diffs, లేదా command-reverse.
- **Encapsulation leak:** memento fields exported అయితే caretaker/others మార్చొచ్చు – unexported + originator package లో.
- **Restore side effects:** restore చేసినప్పుడు derived state (caches, observers) కూడా reset అవ్వాలి – miss అయితే inconsistency.

Key Points

- **Memento** = state snapshot capture & restore, encapsulation break చేయకుండా (undo/redo, checkpoints).
- Go: memento struct **unexported fields** in originator's package = perfect encapsulation (Java tricks అవసరం లేదు).
- **Deep copy in `Save()`** (reference fields shallow copy bug); value memento immutable.
- Memory blowup – bounded history/diffs; restore derived state కూడా reset చెయ్యి.

33. Visitor (మరియు Go లో ఎందుకు awkward)

వివరణ

Visitor = ఒక object structure (types) ని మార్చకుండా, వాటిపై కొత్త **operations** జోడించడం. Operation logic ని ఒక "visitor" object లో పెట్టి, structure elements దాన్ని "accept" చేస్తాయి. కొత్త operation = కొత్త visitor, existing types touch చేయకుండా.

ఇక్కడ Go's weakness మళ్ళీ కనిపిస్తుంది: Visitor **double dispatch** మీద ఆధారపడుతుంది (element type × visitor type). Go కి method overloading లేదు, generics limited – కాబట్టి visitor **verbose మరియు awkward**. అందుకే Go community తరచూ visitor ని avoid చేసి, **type switch** వాడుతుంది.

Real-life Scenario

> **Tax auditor** analogy: auditor (visitor) ఇంటింటికీ (elements) వెళ్తాడు. ప్రతి ఇల్లు auditor ని "accept" చేసి, తన financial details చూపిస్తుంది. Auditor ఒక్కో type ఇంటికి (apartment, villa, shop) వేరే tax లెక్కిస్తాడు. కొత్త audit rule వస్తే – కొత్త auditor, ఇళ్ళు మార్చనవసరం లేదు. కానీ కొత్త ఇంటి రకం వస్తే – ప్రతి auditor ని update చేయాలి (visitor's weakness).

Code – Classic Visitor (verbose in Go)

```go
package main

import "fmt"

// Visitor – ఒక్కో element type కి ఒక method (double dispatch)
type Visitor interface {
        VisitCircle(c *Circle)
        VisitSquare(s *Square)
}

// Element – accept(visitor)
type Shape interface {
        Accept(v Visitor)
}

type Circle struct{ Radius float64 }
func (c *Circle) Accept(v Visitor) { v.VisitCircle(c) } // dispatch 1: element type
```

```go
type Square struct{ Side float64 }
func (s *Square) Accept(v Visitor) { v.VisitSquare(s) }

// ---- concrete visitor: area operation ----
type AreaVisitor struct{ Total float64 }
func (a *AreaVisitor) VisitCircle(c *Circle) { a.Total += 3.14 * c.Radius * c.Radius
}
func (a *AreaVisitor) VisitSquare(s *Square) { a.Total += s.Side * s.Side }

// ---- కొత్త operation = కొత్త visitor (existing shapes touch చేయకుండా) ----
type PerimeterVisitor struct{ Total float64 }
func (p *PerimeterVisitor) VisitCircle(c *Circle) { p.Total += 2 * 3.14 * c.Radius }
func (p *PerimeterVisitor) VisitSquare(s *Square) { p.Total += 4 * s.Side }

func main() {
	shapes := []Shape{&Circle{Radius: 5}, &Square{Side: 4}}

	area := &AreaVisitor{}
	for _, s := range shapes {
		s.Accept(area) // dispatch 2: visitor method
	}
	fmt.Printf("Total area: %.2f\n", area.Total) // 94.50
}
```

Code – type switch (idiomatic Go alternative)

```go
// Go లో visitor కి బదులు — type switch. చాలా simpler.
func Area(shapes []Shape2) float64 {
	total := 0.0
	for _, s := range shapes {
		switch v := s.(type) { // type switch = single dispatch, Go-native
		case Circle2:
			total += 3.14 * v.Radius * v.Radius
		case Square2:
			total += v.Side * v.Side
		}
	}
	return total
}

type Shape2 interface{ isShape() }
type Circle2 struct{ Radius float64 }
func (Circle2) isShape() {}
type Square2 struct{ Side float64 }
func (Square2) isShape() {}
```

Visitor vs type switch (Go)

అంశం	Visitor pattern	Type switch
Add new operation	✅ new visitor (types unchanged)	new function with switch
Add new type	❌ update all visitors	update all switches (same problem)
Boilerplate	ఎక్కువ (Accept + Visit methods)	తక్కువ

అంశం	Visitor pattern	Type switch
Go idiomatic	❌ awkward, verbose	✅ preferred for most cases
Compile-time safety	✅ (miss method = compile error)	⚠️ (miss case = silent, no default)

Go idiom

> **ఎందుకు Go లో awkward:** Visitor's power = **double dispatch** (Java method overloading `visit(Circle)` vs `visit(Square)`). **Go కి method overloading లేదు** – కాబట్టి `VisitCircle`, `VisitSquare` అని different names రాయాలి (verbose). కొత్త type add చేస్తే visitor interface కి కొత్త method → అన్ని visitors break.
>
> **Go's answer: type switch.** చాలా cases లో type switch visitor కంటే simpler మరియు enough. Downside – కొత్త type add చేస్తే switch cases miss అవ్వచ్చు (compiler warn చేయదు; visitor compile error ఇస్తుంది).
>
> **ఎప్పుడు visitor Go లో worth:** stable element hierarchy + తరచూ కొత్త operations (compilers/AST – `go/ast` కి visitor-ish `ast.Walk`), compile-time completeness ముఖ్యం అయితే. లేదంటే **type switch**.
>
> **Go proverb spirit:** "Go లో visitor రాసేముందు, type switch సరిపోతుందా అని ఆలోచించు – 90% సరిపోతుంది."

ఎప్పుడు వాడాలి / వద్దు

- **వాడు:** stable set of types + తరచూ కొత్త operations (AST traversal, document processing, compilers); compile-time completeness అవసరం.
- **వద్దు (Go default):** చాలా cases – **type switch** వాడు (simpler, idiomatic).
- **వద్దు:** types తరచూ మారుతుంటే – visitor అన్ని visitors break చేస్తుంది.

Gotchas

- **New type breaks all visitors:** element types కి కొత్తది add చేస్తే, visitor interface కి method add → అన్ని concrete visitors update. Visitor's inherent weakness.
- **Type switch missing case:** type switch లో కొత్త type కి case మర్చిపోతే silent (no compile error) – `default` panic పెట్టి catch చెయ్యి.
- **Verbose:** Go visitor boilerplate ఎక్కువ – worth ఉందా ఆలోచించు.
- **Pointer vs value in switch:** `case Circle` vs `case *Circle` – type switch లో సరిగ్గా match చెయ్యి.

Key Points

- **Visitor** = types మార్చకుండా కొత్త operations; double dispatch మీద ఆధారపడుతుంది.
- **Go లో awkward** – method overloading లేదు → verbose `VisitX` methods; కొత్త type అన్ని visitors break.
- **Go idiom = type switch** – 90% cases లో simpler & enough (missing case కి `default` panic).

- Visitor Go లో worth: stable types + తరచూ operations + compile-time completeness (AST/compilers).

34. Interpreter

వివరణ

Interpreter = ఒక చిన్న భాష (language/grammar) కోసం, ప్రతి grammar rule ని ఒక **type** గా represent చేసి, expressions ని evaluate చేయడం. Expression tree (AST) నిర్మించి, recursively interpret చేస్తాం. Simple DSLs – arithmetic, boolean rules, query filters, config expressions.

Go లో interpreter = ఒక `Expression` interface (`Interpret()` / `Eval()`), terminal expressions (numbers, variables) + non-terminal expressions (operators – children hold చేస్తాయి), tree recursively evaluate.

Real-life Scenario

> **Calculator** analogy: నువ్వు "5 + 3 * 2" type చేస్తావు. Calculator దీన్ని ఒక tree గా parse చేస్తుంది: `+(5, *(3, 2))`. తర్వాత tree ని bottom-up evaluate – `*(3,2)=6`, తర్వాత `+(5,6)=11`. ప్రతి operation (+, *) ఒక rule; numbers = terminals. Interpreter ఆ tree ని "అర్థం చేసుకుని" (interpret) answer ఇస్తుంది.

Code – Arithmetic interpreter

```
package main

import "fmt"

// Expression — grammar rule. అన్ని expressions ఇది satisfy చేస్తాయి.
type Expression interface {
	Eval() int
}

// ---- Terminal expression: number ----
type Number struct{ value int }
func (n Number) Eval() int { return n.value }

// ---- Non-terminal expressions: operators (children hold చేస్తాయి) ----
type Add struct{ left, right Expression }
func (a Add) Eval() int { return a.left.Eval() + a.right.Eval() } // recursive

type Multiply struct{ left, right Expression }
func (m Multiply) Eval() int { return m.left.Eval() * m.right.Eval() }

func main() {
	// "5 + 3 * 2" → Add(5, Multiply(3, 2)) — expression tree (AST)
	expr := Add{
		left:  Number{5},
		right: Multiply{left: Number{3}, right: Number{2}},
	}
```

```
	fmt.Println("Result:", expr.Eval()) // 11
}
```

Code – Boolean rule interpreter (real-world DSL)

```
// feature flag / access rule: "isAdmin AND (isPremium OR isTrial)"
type BoolExpr interface {
	Eval(ctx map[string]bool) bool
}

type Var struct{ name string }
func (v Var) Eval(ctx map[string]bool) bool { return ctx[v.name] }

type And struct{ l, r BoolExpr }
func (a And) Eval(ctx map[string]bool) bool { return a.l.Eval(ctx) && a.r.Eval(ctx) }

type Or struct{ l, r BoolExpr }
func (o Or) Eval(ctx map[string]bool) bool { return o.l.Eval(ctx) || o.r.Eval(ctx) }

// rule := And{Var{"isAdmin"}, Or{Var{"isPremium"}, Var{"isTrial"}}}
// rule.Eval(map[string]bool{"isAdmin": true, "isTrial": true}) → true
```

Terminal vs Non-terminal expressions

అంశం	Terminal (leaf)	Non-terminal (composite)
Children	లేవు	ఉన్నాయి (sub-expressions)
Example	Number, Variable	Add, Multiply, And, Or
Eval	returns own value	children evaluate + combine (recursive)
Composite pattern	← ఇది Composite (Sec 21) కి దగ్గర	

Go idiom

> **Classic vs Go:** structure same (interface + expression types). Interpreter ≈ **Composite pattern** (Section 21) applied to grammar – expression tree = composite tree.
>
> **Go reality check:**
>
> - **Simple DSLs** (arithmetic, boolean rules, filters) → interpreter pattern fine.
> - **Real languages/complex grammars** → interpreter pattern **doesn't scale** – hand-written interpreter అనవసర object explosion. బదులుగా Go tools: `text/scanner`, `go/parser` (Go itself), లేదా parser generators (`goyacc`, participle, ANTLR-Go). Production లో ఎవరూ complex language ని GoF interpreter pattern తో రాయరు.
> - **Go's own AST** (`go/ast`) = large-scale "interpreter"-ish structure – కానీ visitor/walk తో traverse.
>
> **నియమం:** చిన్న embedded rule engine (< ~10 rule types) → interpreter pattern OK. అంతకంటే పెద్దది → proper parser/AST tooling.

ఎప్పుడు వాడాలి / వద్దు

- **వాడు:** simple, stable grammar – arithmetic expressions, boolean/business rule engines, query filters, feature-flag conditions, template expressions.
- **వద్దు:** complex/evolving language – parser generators, `go/parser`-style tooling వాడు.
- **వద్దు:** grammar ఒక్క expression మాత్రమే – regex/simple parse చాలు (over-engineering).

Gotchas

- **Class/type explosion:** ప్రతి grammar rule = ఒక type. Grammar పెరిగితే types పేలుతాయి – maintainability సమస్య.
- **No parsing:** interpreter pattern **evaluation** మాత్రమే – string → tree (parsing) separate పని. దాన్ని కూడా రాయాలి (lexer/parser).
- **Deep recursion:** deeply nested expression → stack overflow risk. Extreme cases లో iterative eval.
- **Error handling:** `Eval() int` errors handle చేయదు – division by zero, undefined var → `Eval() (int, error)` variant అవసరం.
- **Performance:** tree-walking interpreter slow – hot paths లో bytecode/compilation ఆలోచించు.

Key Points

- **Interpreter** = చిన్న grammar; ప్రతి rule ఒక type; expression tree (AST) recursively evaluate.
- Composite pattern (Sec 21) ని grammar కి apply చేసినట్లు – terminal + non-terminal expressions.
- **Simple DSLs** కి Go లో fine; **complex languages** కి parser generators/`go/parser` (interpreter pattern scale అవదు).
- Type explosion, parsing separate, error handling (`Eval() (T, error)`), deep recursion జాగ్రత్త.

Part 6 — Go-specific Design

> ఇప్పటిదాకా classic GoF patterns Go కి map చేశాం. ఇప్పుడు **Go-native design techniques** — functional options (deep), dependency injection, concurrency patterns (worker pool/pub-sub/pipeline), error-handling design. ఇవి Go LLD యొక్క నిజమైన గుండె — interviews లో "idiomatic Go" అని దీనినే అంటారు.

---

35. Functional Options Pattern (Full Deep Dive)

వివరణ

Functional Options = Go లో configurable objects create చేసే **signature idiom**. Section 14 లో పరిచయం చూశాం — ఇక్కడ full depth: error handling, validation, required vs optional, option grouping, defaults, testing. Rob Pike ప్రవేశపెట్టి, Dave Cheney popularize చేసిన pattern. gRPC, Kubernetes, Uber, ప్రతి major Go library వాడుతుంది.

Core idea: `func New(required..., opts ...Option) (*T, error)` — required args positional, optional configs `Option func(*T)` closures గా. Defaults constructor లో, options వాటిని override.

Real-life Scenario

> **Pizza order** analogy: base pizza (required: size) order చేస్తావు. తర్వాత "extra cheese", "no onion", "thin crust" — optional toppings (options). నువ్వు చెప్పనివి defaults (regular crust, standard cheese). నువ్వు కావలసిన toppings మాత్రమే చెప్తావు, positional గా "medium, yes, no, thin, ..." అని 10 args చెప్పవు. కొత్త topping (paneer) menu కి add అయితే — పాత orders break అవ్వవు.

Code — Full-featured functional options

```
package main

import (
	"errors"
	"fmt"
	"time"
)

type Server struct {
	host    string
	port    int
	timeout time.Duration
	maxConn int
	tls     bool
```

```
}

// Option — error return చేసే variant (validation కోసం powerful)
type Option func(*Server) error

// ---- option constructors (with validation) ----
func WithPort(p int) Option {
        return func(s *Server) error {
                if p < 1 || p > 65535 {
                        return fmt.Errorf("invalid port: %d", p)
                }
                s.port = p
                return nil
        }
}

func WithTimeout(d time.Duration) Option {
        return func(s *Server) error {
                if d <= 0 {
                        return errors.New("timeout positive గా ఉండాలి")
                }
                s.timeout = d
                return nil
        }
}

func WithTLS() Option {
        return func(s *Server) error { s.tls = true; return nil }
}

// option grouping — related options ఒకచోట (composition of options)
func WithProductionDefaults() Option {
        return func(s *Server) error {
                s.timeout = 60 * time.Second
                s.maxConn = 1000
                s.tls = true
                return nil
        }
}

// constructor — required (host) positional, optional variadic
func NewServer(host string, opts ...Option) (*Server, error) {
        if host == "" {
                return nil, errors.New("host required")
        }
        // 1. sensible defaults
        s := &Server{
                host:    host,
                port:    8080,
                timeout: 30 * time.Second,
                maxConn: 100,
        }
        // 2. options apply + validate (fail fast)
        for _, opt := range opts {
                if err := opt(s); err != nil {
                        return nil, fmt.Errorf("option error: %w", err)
                }
        }
        // 3. cross-field validation (options అయ్యాక)
```

```
        if s.tls && s.port == 80 {
                return nil, errors.New("TLS తో port 80 వద్దు")
        }
        return s, nil
}

func main() {
        s, err := NewServer("api.x.com",
                WithProductionDefaults(),
                WithPort(443),
                WithTimeout(10*time.Second),
        )
        if err != nil {
                fmt.Println("error:", err)
                return
        }
        fmt.Printf("%+v\n", *s)

        // invalid — validation catches
        _, err = NewServer("x", WithPort(99999))
        fmt.Println("expected error:", err)
}
```

Option type variants

Variant	Signature	ఎప్పుడు
Simple	`func(*T)`	validation అవసరం లేదు (Section 14)
Error-returning	`func(*T) error`	validation అవసరం (above)
Interface option	`type Option interface{ apply(*T) }`	complex – options need identity/comparison
Config struct + options	options mutate a `Config`	large config, separate from object

Functional options vs Config struct

అంశం	Functional options	Config struct param
Backward compatible	✅ (new option = no break)	⚠️ (new field OK, but zero-value ambiguity)
Defaults	constructor లో	caller must know / separate defaults
"unset vs zero"	✅ clear (option missing)	❌ ambiguous (`0` = unset or intended?)
Verbosity	slightly more	less
Discoverability	`WithX` autocomplete	struct fields
ఎప్పుడు	public API, many optionals	internal, simple, all-known config

Go idiom

> **ఎందుకు functional options Go-idiomatic:**
>
> - **Go కి default parameters / overloading లేదు** – ఈ gap ని functional options fill చేస్తాయి.
> - **Backward compatible:** library maintainers కొత్త option add చేయవచ్చు, existing callers break అవ్వకుండా. ఇది **public library API design** కి critical.
> - **"unset vs zero value" problem solve:** config struct లో `Port: 0` – "unset" నా "intentionally 0" నా తెలియదు. Option missing అయితే స్పష్టంగా "unset" (default applies).
> - **Self-documenting:** `WithTLS()` positional bool `true` కంటే readable.
>
> **గుర్తుంచుకో:** functional options **public/library APIs** కి best. Internal, simple, all-required config కి **plain struct/positional args** చాలు – options ప్రతిచోటా over-kill.

ఎప్పుడు వాడాలి / వద్దు

- **వాడు:** public library/package APIs; 3+ optional parameters; future extensibility ముఖ్యం; "unset vs zero" ambiguity ఉంది; defaults ఉన్నాయి.
- **వద్దు:** 1-2 required params, no optionals – `New(a, b)` చాలు.
- **వద్దు:** internal-only, all fields always set – plain config struct.
- **వద్దు (over):** ప్రతి struct కి options – boilerplate పెరుగుతుంది.

Gotchas

- **Options order dependency:** options sequential apply – ఒకటి ఇంకోదాన్ని override చేయొచ్చు (`WithProductionDefaults()` ముందు, `WithPort(443)` తర్వాత → 443 wins). Order-sensitive options design smell.
- **Required as option (anti-pattern):** required fields ని option గా పెట్టకు – positional args (compiler enforce). Option missing అయితే required silently unset అవుతుంది.
- **Validation timing:** options అయ్యాక cross-field validation చెయ్యి (individual option లో partial state చూడొచ్చు).
- **nil option in variadic:** nil `Option` pass అయితే apply loop panic – nil check అవసరమైతే.
- **Testing options:** ప్రతి option ని విడిగా test చేయవచ్చు (`opt(s)` call చేసి field check) – functional options testable.

Key Points

- **Functional Options** = Go's idiom for configurable creation (`New(required, opts ...Option)`) – default params/overloading లేని gap ని fill చేస్తుంది.
- **Error-returning** `Option func(*T) error` = validation కి; defaults constructor లో, cross-field validation options అయ్యాక.
- Solves **"unset vs zero value"** + **backward compatible** + self-documenting → **public library APIs** కి best.

- Required as option వద్దు (positional); order dependency & nil option జాగ్రత్త; options individually testable.

36. Dependency Injection in Go (Manual, wire, fx)

వివరణ

Dependency Injection (DI) = ఒక object తన dependencies ని **తనే create చేసుకోకుండా**, బయట నుండి (constructor/parameter) pass చేయడం. దీంతో loose coupling, testability (mock inject), flexibility. ఇది DIP (Section 8) యొక్క practical implementation.

Go లో DI చాలా simple – **manual constructor injection** default. Java's Spring లాంటి heavy DI containers అవసరం లేదు (చాలా Go devs వాటిని avoid చేస్తారు). పెద్ద projects కి `google/wire` (compile-time) లేదా `uber-go/fx` (runtime) ఉన్నాయి, కానీ manual DI 90% cases కి సరిపోతుంది.

Real-life Scenario

> **కారులో engine** analogy: కారు factory లో engine ని కారు లోపలే build చేయదు – engine విడిగా తయారవుతుంది, తర్వాత కారులో **అమర్చుతారు** (inject). దీంతో – same కారు body కి petrol/diesel/electric engine అమర్చవచ్చు (flexibility). Testing లో "test engine" అమర్చవచ్చు (mock). కారు engine ని hard-wire చేస్తే – ఈ flexibility పోతుంది.

Code – Manual constructor injection (idiomatic Go)

```
package main

import "fmt"

// dependencies as interfaces (accept interfaces)
type UserRepo interface {
	FindName(id int) string
}
type Mailer interface {
	Send(to, msg string) error
}

// ---- concrete dependencies ----
type PostgresRepo struct{}
func (PostgresRepo) FindName(id int) string { return fmt.Sprintf("User%d", id) }

type SMTPMailer struct{}
func (SMTPMailer) Send(to, msg string) error { fmt.Printf("mail→%s: %s\n", to, msg);
return nil }

// ---- Service: dependencies ని constructor లోకి inject ----
type WelcomeService struct {
	repo   UserRepo // injected – concrete కాదు
	mailer Mailer
}
```

```
func NewWelcomeService(repo UserRepo, mailer Mailer) *WelcomeService {
        return &WelcomeService{repo: repo, mailer: mailer}
}

func (s *WelcomeService) Welcome(id int) error {
        name := s.repo.FindName(id)
        return s.mailer.Send(name, "Welcome "+name+"!")
}

// ---- composition root: main() లో dependencies wire చేస్తాం ----
func main() {
        // అన్ని concrete dependencies ఇక్కడ (ఒకేచోట) create + wire
        repo := PostgresRepo{}
        mailer := SMTPMailer{}
        svc := NewWelcomeService(repo, mailer)
        svc.Welcome(42)
}
```

Code – Testing benefit (mock injection)

```
// test లో — real DB/SMTP బదులు fakes inject. Framework అవసరం లేదు.
type fakeRepo struct{}
func (fakeRepo) FindName(id int) string { return "TestUser" }

type spyMailer struct{ sent string }
func (m *spyMailer) Send(to, msg string) error { m.sent = msg; return nil }

// func TestWelcome(t *testing.T) {
//     spy := &spyMailer{}
//     svc := NewWelcomeService(fakeRepo{}, spy)
//     svc.Welcome(1)
//     if spy.sent != "Welcome TestUser!" { t.Fail() }
// }
```

Manual DI vs wire vs fx

అంశం	Manual (constructor)	google/wire	uber-go/fx
Wiring	hand-written in `main()`	**code-generated** (compile-time)	reflection (runtime)
Compile safety	✅	✅ (generated code)	⚠️ (runtime errors)
Boilerplate	grows with app size	wire.go + generated	provider functions
Learning curve	zero	moderate	higher
Startup cost	zero	zero (compiled)	reflection overhead
ఎప్పుడు	most apps (default)	large app, wiring tedious	large app, lifecycle mgmt (start/stop hooks)

Go idiom

> **Go's DI philosophy: "DI = just passing arguments."** Go లో fancy container అవసరం లేదు – dependencies ని interfaces గా accept చేసి, constructor లో pass చేయడమే. **Composition root** (`main()` లేదా ఒక `setup()` func) లో అన్ని concrete types ఒకచోట wire.
>
> **నియమాలు:**
>
> - **Accept interfaces** (dependencies), **return structs** (Section 5).
> - **Constructor injection** default (`NewX(deps...)`). Field/setter injection Go లో అరుదు.
> - **Composition root ఒకటే** – `main()` లో అన్నీ wire (dependencies graph ఒకచోట కనిపిస్తుంది).
> - Manual DI tedious అయ్యేంత పెద్ద app అయితేనే `wire` (compile-time, Go-idiomatic choice) – reflection-heavy fx కంటే wire ని చాలా Go devs prefer చేస్తారు.
>
> **anti-pattern:** global singletons / `init()` లో dependencies create చేయడం = hidden dependencies, untestable. DI దీన్ని fix చేస్తుంది.

ఎప్పుడు వాడాలి / వద్దు

- **వాడు:** ఎప్పుడూ dependencies కి (DB, HTTP clients, loggers, mailers, clocks) – testability + flexibility.
- **వాడు (manual):** default – 90% Go apps.
- **వాడు (wire):** dependency graph చాలా పెద్దది, manual wiring error-prone అయితే.
- **వద్దు (DI container):** small/medium apps కి fx/heavy container – over-engineering. Manual చాలు.
- **వద్దు:** pure functions / stateless utilities కి DI అనవసరం.

Gotchas

- **Over-injection:** అన్నిటినీ inject చేయకు – `time.Now()` , `rand` లాంటివి testability అవసరమైతేనే interface వెనుక (clock injection). లేదంటే direct.
- **Interface pollution:** ప్రతి dependency కి interface = premature. Concrete pass చేసి, test కి mock అవసరమైనప్పుడే interface extract.
- **Constructor param explosion:** చాలా dependencies (7+) = struct too big (SRP smell). Split the service.
- **Circular dependencies:** A needs B, B needs A → wire చేయలేం. Redesign (interface/mediator/event).
- **Global state leak:** singleton + DI mix చేస్తే confusing. Consistent గా DI వాడు.

Key Points

- **DI** = dependencies ని బయట నుండి inject (self-create కాదు); DIP యొక్క practical form; testability + flexibility.

- Go: **manual constructor injection** default – "DI = just passing arguments", container అవసరం లేదు.
- **Composition root** (`main()`) లో అన్ని concrete types ఒకేచోట wire.
- Large apps → `wire` (compile-time, preferred) / `fx` (runtime). Over-injection & interface pollution జాగ్రత్త.

37. Concurrency Design Patterns as LLD Building Blocks

వివరణ

Go లో **concurrency = first-class LLD building block**. Goroutines + channels తో worker pool, pub-sub, pipeline లాంటి patterns design చేయడం Go LLD యొక్క ప్రత్యేకత. Java లో ఇవి threads + locks + queues తో painful; Go లో channels తో natural. Go proverb: **"Don't communicate by sharing memory; share memory by communicating."**

మూడు core patterns:

1. **Worker Pool** – fixed goroutines ఒక jobs channel నుండి పని తీసుకుంటాయి (bounded parallelism).
2. **Pipeline** – stages, ప్రతిదీ channel ద్వారా next కి data forward (stream processing).
3. **Fan-out / Fan-in** – ఒక source ని multiple workers (fan-out), results ని ఒక channel లోకి (fan-in).

Real-life Scenario

> **Restaurant kitchen** analogy: **Worker pool** = 5 cooks (fixed), orders queue (channel) నుండి తీసుకుని వండుతారు – orders ఎక్కువ వచ్చినా cooks 5 మాత్రమే (bounded). **Pipeline** = assembly line – ఒకరు కూరగాయలు కోస్తారు → next వండుతారు → next plate చేస్తారు, ప్రతి stage తర్వాతి వాడికి pass. **Fan-out/in** = ఒక పెద్ద order ని 5 cooks కి పంచి (fan-out), అన్నీ ready అయ్యాక ఒక plate లో కలపడం (fan-in).

Code – Worker Pool (bounded parallelism)

```
package main

import (
	"fmt"
	"sync"
)

func worker(id int, jobs <-chan int, results chan<- int, wg *sync.WaitGroup) {
	defer wg.Done()
	for job := range jobs { // jobs channel range — closed అయ్యేదాకా
		results <- job * job // process
	}
}

func main() {
```

```go
	const numWorkers = 3
	jobs := make(chan int, 10)
	results := make(chan int, 10)

	var wg sync.WaitGroup
	// fixed workers start (bounded parallelism — 1000 goroutines కాదు)
	for w := 1; w <= numWorkers; w++ {
		wg.Add(1)
		go worker(w, jobs, results, &wg)
	}

	// jobs push
	for j := 1; j <= 5; j++ {
		jobs <- j
	}
	close(jobs) // ముఖ్యం — workers range ఆగడానికి

	// results collector goroutine
	go func() { wg.Wait(); close(results) }()

	sum := 0
	for r := range results {
		sum += r
	}
	fmt.Println("sum of squares:", sum) // 1+4+9+16+25 = 55
}
```

Code — Pipeline (stream processing)

```go
// stage 1: generate → stage 2: square → stage 3: consume
func gen(nums ...int) <-chan int {
	out := make(chan int)
	go func() {
		defer close(out)
		for _, n := range nums {
			out <- n
		}
	}()
	return out
}

func square(in <-chan int) <-chan int {
	out := make(chan int)
	go func() {
		defer close(out)
		for n := range in {
			out <- n * n // transform, forward
		}
	}()
	return out
}

// వాడకం: for r := range square(gen(1, 2, 3)) { ... } → 1, 4, 9
```

Concurrency patterns comparison

Pattern	ఉద్దేశం	కీలకం
Worker pool	bounded parallelism	fixed goroutines, jobs channel
Pipeline	staged stream processing	channel per stage, `defer close`
Fan-out	ఒక source → multiple workers	multiple goroutines, same in-channel
Fan-in	multiple sources → ఒక channel	merge with WaitGroup
Pub-Sub	ఒక event → multiple subscribers	Section 43

Go idiom

> **Go's concurrency LLD rules:**
>
> - **Bounded parallelism:** ప్రతి request కి unlimited goroutines కాదు – **worker pool** తో bound చేయి (resource exhaustion తప్పించడం).
> - **Channel ownership:** ఒక channel ని **ఎవరు close చేయాలి?** → **sender close చేస్తాడు** (receiver ఎప్పుడూ కాదు). Closed channel కి send = panic.
> - `context.Context` cancellation కి – long-running goroutines ని `ctx.Done()` తో ఆపు (goroutine leak తప్పించడం).
> - `sync.WaitGroup` goroutines complete అయ్యేదాకా wait; `select` multiple channels handle.
> - **"Share memory by communicating"** – shared mutable state + mutex కంటే channels ద్వారా data pass చేయడం prefer (కానీ mutex కూడా valid – Go proverb absolute కాదు).
>
> ఇవి LLD interviews లో (rate limiter, logger, pub-sub) constant గా వస్తాయి – Sections 41-43 case studies.

ఎప్పుడు వాడాలి / వద్దు

- **Worker pool:** bounded concurrent processing (image resize, API calls, batch jobs) – unbounded goroutines తప్పించడానికి.
- **Pipeline:** multi-stage stream/data processing (ETL, log processing).
- **Fan-out/in:** parallelize independent work then aggregate.
- **వద్దు:** simple sequential work కి concurrency – complexity + bugs. "Concurrency is not parallelism, and not always needed."
- **వద్దు:** shared state everywhere with mutex maze – channels తో redesign.

Gotchas

- **Goroutine leak:** goroutine channel మీద forever block (no close, no context) → leak. `defer close`, `context`, buffered channels.

- **Closed channel panic:** closed channel కి send = panic; closed channel receive = zero value immediately. Sender-closes rule.
- **Deadlock:** అందరూ send/receive కి wait చేస్తే → `fatal error: all goroutines are deadlocked`. Buffering/select/ordering.
- **Race conditions:** shared variable ని goroutines access చేస్తే → `go run -race` తో detect. Channels/mutex.
- **Unbounded goroutines:** `for { go handle() }` = resource exhaustion. Worker pool తో bound.
- **WaitGroup misuse:** `wg.Add()` goroutine ముందు (లోపల కాదు); `wg.Done()` `defer` తో.

Key Points

- Go: **concurrency = first-class LLD building block** (channels + goroutines) – worker pool, pipeline, fan-out/in.
- **Worker pool** = bounded parallelism (unbounded goroutines తప్పించడం); **pipeline** = staged channels.
- **Sender closes channel**; **`context` for cancellation**; `WaitGroup` for completion; `select` for multiplexing.
- Goroutine leaks, closed-channel panic, deadlock, races (`-race`) – Go concurrency's main hazards.

38. Error-handling Design (Sentinel vs Wrapped vs Typed)

వివరణ

Go లో **errors are values** – exceptions లేవు (`panic` ఉంది కానీ అది exceptional cases కి, control flow కి కాదు). ఒక function error ని **return value** గా ఇస్తుంది (`(T, error)`), caller దాన్ని explicitly handle చేస్తాడు. ఇది Go LLD యొక్క ముఖ్య design decision – error handling ని API design లో భాగంగా చూడాలి.

మూడు error strategies:

1. **Sentinel errors** – predefined error values (`var ErrNotFound = errors.New(...)`), `errors.Is()` తో compare.
2. **Wrapped errors** – `fmt.Errorf("...: %w", err)` తో context జోడించి wrap, `errors.Is/As` తో unwrap.
3. **Typed/custom errors** – struct implementing `error` interface, extra fields (code, details), `errors.As()` తో extract.

Real-life Scenario

> **Courier delivery failure** analogy: package deliver కాకపోతే – courier నీకు ఒక **slip** (error value) ఇస్తాడు, silent గా మాయం కాడు (exception కాదు). Slip మీద reason: "address not found" (sentinel – standard reason), లేదా "attempted at 3pm, gate locked, will retry" (wrapped – context added at each step), లేదా structured form with code 404 + details (typed). నువ్వు slip చదివి decide చేస్తావు – retry, refund, escalate.

Code – Three strategies

```
package main

import (
	"errors"
	"fmt"
)

// ---- 1. Sentinel error ----
var ErrNotFound = errors.New("record not found") // predefined value

// ---- 3. Typed/custom error ----
type ValidationError struct {
	Field string
	Msg   string
}
func (e *ValidationError) Error() string {
	return fmt.Sprintf("validation failed on %s: %s", e.Field, e.Msg)
}

// ---- repository: sentinel + wrapping ----
func findUser(id int) (string, error) {
	if id <= 0 {
		return "", &ValidationError{Field: "id", Msg: "positive గా ఉండాలి"} //
typed
	}
	if id > 100 {
		// 2. wrap – context జోడించి sentinel ని preserve
		return "", fmt.Errorf("findUser(%d): %w", id, ErrNotFound)
	}
	return fmt.Sprintf("User%d", id), nil
}

func main() {
	// sentinel check – errors.Is (wrapping గుండా కూడా match అవుతుంది)
	if _, err := findUser(200); err != nil {
		if errors.Is(err, ErrNotFound) {
			fmt.Println("→ not found (sentinel matched through wrap):",
err)
		}
	}

	// typed check – errors.As (concrete type extract)
	if _, err := findUser(-1); err != nil {
		var ve *ValidationError
		if errors.As(err, &ve) {
```

```
                        fmt.Printf("→ validation error on field %q: %s\n", ve.Field,
ve.Msg)
                }
        }

        if name, err := findUser(5); err == nil {
                fmt.Println("→ found:", name)
        }
}
```

Three strategies comparison

Strategy	ఏమిటి	Check with	ఎప్పుడు
Sentinel	predefined `var Err...`	`errors.Is()`	known, comparable error conditions (`io.EOF` , `sql.ErrNoRows`)
Wrapped	`fmt.Errorf("...: %w", err)`	`errors.Is/As()`	context జోడించడం (call stack trace)
Typed	struct implementing `error`	`errors.As()`	extra data అవసరం (field, code, HTTP status)

Go idiom

> **Go error design rules:**
>
> - `if err != nil { return ..., err }` – verbose కానీ explicit. దీన్ని embrace చెయ్యి, hide చేయడానికి prumaturely helpers రాయకు.
> - **Wrap with `%w`** – error propagate చేసేటప్పుడు context జోడించు (`fmt.Errorf("saving order: %w", err)`). `errors.Is/As` wrapping గుండా చూస్తాయి.
> - `errors.Is` = value comparison (sentinel); `errors.As` = type extraction (typed). `==` వాడకు (wrapping break చేస్తుంది).
> - **Sentinel errors ని exported వాడు జాగ్రత్తగా** – అవి API contract అవుతాయి (callers `errors.Is` వాడతారు).
> - **`panic` control flow కి కాదు** – programmer errors (nil deref, index out of bounds), truly unrecoverable startup failures కి మాత్రమే. Library boundaries లో `recover()` (కానీ అరుదు).
> - **error strings:** lowercase, no punctuation (`"file not found"`, not `"File not found."`) – అవి wrap అవుతాయి కాబట్టి.
>
> **nil interface trap (BIG Go gotcha):**
>
> ```
> func do() error {
>     var e *ValidationError = nil
>     return e // ⚠️ typed nil — interface కి type ఉంది, value nil
> }
> // if do() != nil → TRUE! (bug — nil అనుకున్నా non-nil interface)
> ```
>
> Concrete error pointer ని return చేసేటప్పుడు – nil అయితే `return nil` (explicit), typed nil కాదు.

Sentinel vs Typed – ఎప్పుడు ఏది

పరిస్థితి	ఎంచుకో
"ఈ specific condition జరిగిందా?"	Sentinel + `errors.Is`
"error తో పాటు data (field/code) కావాలి"	Typed + `errors.As`
"context/trace జోడించాలి"	Wrap with `%w`
Multiple error conditions	Typed with a `Code` field

ఎప్పుడు వాడాలి / వద్దు

- **Sentinel వాడు:** stable, comparable conditions (`io.EOF`, `ErrNotFound`).
- **Wrap వాడు:** ఎప్పుడూ error propagate చేసేటప్పుడు context జోడించడానికి (`%w`).
- **Typed వాడు:** error carry extra data (validation field, HTTP status, retryable flag).

- **వద్దు (panic):** normal error conditions కి panic వద్దు – return error.
- **వద్దు (over-wrap):** ప్రతి layer లో redundant wrap ("failed to X: failed to Y: failed to Z") – meaningful context మాత్రమే.

Gotchas

- **nil interface trap:** typed nil (`var e *T = nil; return e`) → `err != nil` true. Concrete nil ని `return nil` గా.
- **`==` vs `errors.Is` :** wrapped error ని `err == ErrNotFound` false (wrap చేసింది). ఎప్పుడూ `errors.Is` .
- **Ignoring errors:** `_ = doThing()` – errors silently drop చేయకు (linters catch చేస్తాయి). `errcheck` .
- **Wrapping sensitive data:** error messages లో passwords/PII wrap చేయకు (logs లోకి leak).
- **Panic across goroutines:** ఒక goroutine లో panic recover కాకపోతే మొత్తం program crash – goroutine boundaries లో handle.
- **Over-abstraction:** custom error hierarchy (Java-style exception tree) Go లో అనవసరం – flat sentinels/typed errors చాలు.

Key Points

- Go: **errors are values** (`(T, error)`), explicit handling; `panic` control flow కి కాదు.
- **Sentinel** (`errors.Is`), **Wrapped** (`%w` + `errors.Is/As`), **Typed** (`errors.As` , extra data) – 3 strategies.
- **Wrap with** `%w` to add context; `errors.Is` (value), `errors.As` (type) – `==` వాడకు.
- **nil interface trap** (typed nil → non-nil interface) = Go's classic error bug; concrete nil ని `return nil` .

Part 7 – LLD Case Studies in Go

> ఇప్పుడు theory ని కలిపి **full designs** – Parking Lot, LRU Cache, Rate Limiter, Logger, Pub/Sub, Vending Machine. ప్రతిదానికి entities → interfaces → patterns → runnable Go code. Interview లో exactly ఇలా approach చెయ్యి. చివర్లో interview framework + Memory Tips + Common Mistakes tables.

---

39. Design a Parking Lot

వివరణ

Requirements: Multiple vehicle types (bike, car, truck) → different slot sizes. Park/unpark, find available slot, calculate fee. Pricing strategy pluggable (hourly, flat, weekend). Multiple floors (extensible).

Entities (nouns → types): `Vehicle`, `Slot`, `Floor`, `ParkingLot`, `Ticket`, `FeeStrategy`.
Behaviors (verbs → methods/interfaces): park, unpark, calculateFee. Pricing polymorphic → **Strategy**. Slot assignment → could be Strategy too.

Real-life Scenario

> **Mall parking** analogy: గేటు దగ్గర bike/car/truck బట్టి సరైన floor/slot కేటాయిస్తారు (slot allocation). బయటికి వచ్చేటప్పుడు entry time బట్టి fee లెక్కిస్తారు (fee strategy – weekday/weekend వేరు). Full అయితే "no space" board. ఇదే మనం model చేస్తున్నాం.

Design decisions

Concern	Pattern / Go idiom
Vehicle types	enum (`iota`) + `Size()` method
Pricing (varies)	**Strategy** (`FeeStrategy` – func/interface)
Slot allocation	Strategy (nearest/random) – simplified here
Thread-safety	`sync.Mutex` (concurrent park/unpark)
Fee calc	interface `FeeStrategy`

Code

```
package parking

import (
	"errors"
	"fmt"
	"sync"
	"time"
)

// ---- vehicle types (enum) ----
type VehicleType int

const (
	Bike VehicleType = iota
	Car
	Truck
)

func (v VehicleType) String() string { return [...]string{"Bike", "Car", "Truck"}[v]
}

type Vehicle struct {
	Number string
	Type   VehicleType
}

// ---- slot ----
type Slot struct {
	ID      int
	Size    VehicleType // ఈ slot ఏ max size fit అవుతుంది
	vehicle *Vehicle    // nil = ఖాళీ
}

func (s *Slot) isFree() bool { return s.vehicle == nil }

// ---- Strategy: fee calculation (pluggable pricing) ----
type FeeStrategy interface {
	Fee(v VehicleType, dur time.Duration) int
}

type HourlyFee struct{ rates map[VehicleType]int } // ₹/hour by type
func (h HourlyFee) Fee(v VehicleType, d time.Duration) int {
	hours := int(d.Hours()) + 1 // ceil (partial hour = full)
	return hours * h.rates[v]
}

type FlatFee struct{ amount int }
func (f FlatFee) Fee(_ VehicleType, _ time.Duration) int { return f.amount }

// ---- Ticket ----
type Ticket struct {
	Vehicle  Vehicle
	Slot     *Slot
	EntryAt  time.Time
}
```

```go
// ---- ParkingLot (thread-safe) ----
type ParkingLot struct {
	mu       sync.Mutex
	slots    []*Slot
	fee      FeeStrategy // injected (DI)
}

func NewParkingLot(counts map[VehicleType]int, fee FeeStrategy) *ParkingLot {
	lot := &ParkingLot{fee: fee}
	id := 1
	for vtype, n := range counts {
		for i := 0; i < n; i++ {
			lot.slots = append(lot.slots, &Slot{ID: id, Size: vtype})
			id++
		}
	}
	return lot
}

var ErrNoSpace = errors.New("no available slot")

// Park — సరైన size ఖాళీ slot వెతికి assign
func (l *ParkingLot) Park(v Vehicle) (*Ticket, error) {
	l.mu.Lock()
	defer l.mu.Unlock()
	for _, s := range l.slots {
		if s.isFree() && s.Size >= v.Type { // small vehicle పెద్ద slot లో fit
అవుతుంది
			s.vehicle = &v
			return &Ticket{Vehicle: v, Slot: s, EntryAt: time.Now()}, nil
		}
	}
	return nil, ErrNoSpace
}

// Unpark — slot ఖాళీ చేసి fee లెక్కించు
func (l *ParkingLot) Unpark(t *Ticket) (int, error) {
	l.mu.Lock()
	defer l.mu.Unlock()
	if t.Slot.isFree() {
		return 0, errors.New("slot already free")
	}
	dur := time.Since(t.EntryAt)
	fee := l.fee.Fee(t.Vehicle.Type, dur) // Strategy invoke
	t.Slot.vehicle = nil                  // free
	return fee, nil
}

func demo() {
	lot := NewParkingLot(
		map[VehicleType]int{Bike: 2, Car: 3, Truck: 1},
		HourlyFee{rates: map[VehicleType]int{Bike: 10, Car: 20, Truck: 40}},
	)
	ticket, err := lot.Park(Vehicle{Number: "AP01-1234", Type: Car})
	if err != nil {
		fmt.Println(err)
		return
	}
	fmt.Printf("Parked %s at slot %d\n", ticket.Vehicle.Number, ticket.Slot.ID)
```

```
        fee, _ := lot.Unpark(ticket)
        fmt.Println("Fee:", fee)
}
```

Design highlights

అంశం	ఎందుకు ఇలా
`FeeStrategy` interface	pricing తరచూ మారుతుంది (encapsulate what varies + OCP)
DI (fee injected)	testability + flexibility (Section 36)
`sync.Mutex`	park/unpark concurrent (multiple gates)
`VehicleType >= Size`	small vehicle పెద్ద slot లో fit
enum + `String()`	readable, type-safe

Extensions (interviewer అడిగితే)

- **Multiple floors:** `Floor` struct with slots; `ParkingLot` holds `[]*Floor`.
- **Slot allocation strategy:** `SlotStrategy` interface (nearest-to-entrance, random) – Strategy.
- **Payment:** `PaymentStrategy` (cash/UPI/card) – Strategy.
- **Reservation, EV charging slots, ticketing persistence** – extend without breaking (OCP).

Gotchas

- **Concurrency:** park/unpark ని lock చేయకపోతే రెండు cars ఒకే slot → race. `sync.Mutex`.
- **Fee rounding:** partial hour = full hour (ceil) – business rule clarify.
- **Slot fit logic:** `>=` (small fits big) కావాలా, exact match కావాలా – requirement.
- **O(n) slot search:** పెద్ద lot లో linear scan slow – free slots per type ని separate queue/heap లో maintain.

Key Points

- Entities: `Vehicle` (enum type), `Slot`, `Ticket`, `ParkingLot`; behavior: park/unpark/fee.
- **`FeeStrategy` interface** = pricing varies (Strategy + OCP + DI); mutex = thread-safety.
- Extensions (floors, allocation strategy, payment) OCP-friendly – interfaces తో.
- Interview లో: clarify → entities → interfaces (Strategy for pricing) → code → concurrency + edge cases.

40. Design an LRU Cache

వివరణ

Requirements: Fixed-capacity cache. `Get(key)` మరియు `Put(key, value)` రెండూ **O(1)**. Capacity దాటితే **Least Recently Used (LRU)** entry ని evict చెయ్యి. Thread-safe.

Core insight: O(1) get/put + LRU ordering కావాలంటే – **hash map** (O(1) lookup) + **doubly linked list** (O(1) reorder/evict) కలిపి. Map key → list node pointer. Access చేసినప్పుడు node ని list front కి move. Evict = list back (tail) తీసేయి.

Go లో `container/list` (doubly linked list) standard library లో ఉంది – దాన్ని వాడతాం.

Real-life Scenario

> **Study table books** analogy: table మీద 5 books పెట్టగలవు (capacity). ఒక book వాడినప్పుడు దాన్ని పైకి (most recent) పెడతావు. కొత్త book కావాలి, table full అయితే – అట్టడుగున ఉన్న (చాలా కాలం ముట్టని = LRU) book ని shelf కి పంపి, కొత్తది పైన పెడతావు. పైన = recently used, కింద = evict candidate.

Design decisions

Concern	Solution
O(1) lookup	`map[key]*list.Element`
O(1) reorder/evict	`container/list` (doubly linked list)
LRU order	front = MRU, back = LRU
Thread-safety	`sync.Mutex`
Generic key/value	generics (Go 1.18+) – cleaner

Code

```
package lru

import (
	"container/list"
	"sync"
)

// entry — linked list node లో store చేసేది (key కూడా — evict కి అవసరం)
type entry[K comparable, V any] struct {
	key   K
	value V
}

type LRUCache[K comparable, V any] struct {
	mu       sync.Mutex
	capacity int
	ll       *list.List             // doubly linked list: front=MRU, back=LRU
	items    map[K]*list.Element    // key → list node (O(1) access)
}

func New[K comparable, V any](capacity int) *LRUCache[K, V] {
	return &LRUCache[K, V]{
		capacity: capacity,
```

```go
		ll:       list.New(),
		items:    make(map[K]*list.Element, capacity),
	}
}

// Get — O(1). Found అయితే node ని front కి move (recently used).
func (c *LRUCache[K, V]) Get(key K) (V, bool) {
	c.mu.Lock()
	defer c.mu.Unlock()
	if elem, ok := c.items[key]; ok {
		c.ll.MoveToFront(elem) // MRU గా mark
		return elem.Value.(*entry[K, V]).value, true
	}
	var zero V
	return zero, false
}

// Put — O(1). Exists అయితే update + front. లేకపోతే insert; full అయితే evict LRU.
func (c *LRUCache[K, V]) Put(key K, value V) {
	c.mu.Lock()
	defer c.mu.Unlock()

	if elem, ok := c.items[key]; ok { // update existing
		elem.Value.(*entry[K, V]).value = value
		c.ll.MoveToFront(elem)
		return
	}

	// insert new at front
	elem := c.ll.PushFront(&entry[K, V]{key: key, value: value})
	c.items[key] = elem

	// capacity దాటితే evict LRU (list back)
	if c.ll.Len() > c.capacity {
		c.evictLRU()
	}
}

func (c *LRUCache[K, V]) evictLRU() {
	back := c.ll.Back() // least recently used
	if back == nil {
		return
	}
	c.ll.Remove(back)
	kv := back.Value.(*entry[K, V])
	delete(c.items, kv.key) // map నుండి కూడా remove (key అందుకే store చేశాం)
}

func (c *LRUCache[K, V]) Len() int {
	c.mu.Lock()
	defer c.mu.Unlock()
	return c.ll.Len()
}
```

```go
// వాడకం:
// cache := lru.New[string, int](2)
// cache.Put("a", 1); cache.Put("b", 2)
```

```
// cache.Get("a")         // a recently used
// cache.Put("c", 3)      // capacity=2, b (LRU) evicted
// _, ok := cache.Get("b") // ok=false (evicted)
```

Why map + doubly linked list?

Operation	Map alone	List alone	Map + DLL (LRU)
Lookup	O(1)	O(n)	**O(1)** (map)
Reorder	❌ no order	O(1) if node ref	**O(1)** (DLL)
Evict LRU	❌	O(1) (tail)	**O(1)** (DLL back)
Combined	❌	❌	✅ both O(1)

Go idiom

- `container/list` = Go's doubly linked list (`PushFront` , `MoveToFront` , `Back` , `Remove`) – LRU కి perfect.
- **`entry` లో key ఎందుకు?** – evict చేసేటప్పుడు list node నుండి key తెలిస్తేనే `map` నుండి కూడా delete చేయగలం (map memory leak తప్పించడం).
- **Generics (Go 1.18+):** `LRUCache[K comparable, V any]` – type-safe, reusable. Pre-generics లో `interface{}` (type assertion boilerplate).
- **`sync.Mutex`** – Get కూడా write (MoveToFront) చేస్తుంది కాబట్టి `RWMutex` కాదు, plain `Mutex` .
- Production లో `hashicorp/golang-lru` వాడతారు – కానీ interview లో scratch నుండి రాయాలి.

ఎప్పుడు వాడాలి / Extensions

- **TTL (time-based expiry):** entry కి `expireAt` field + lazy/background eviction.
- **LFU (least frequently used):** frequency counter + different eviction – వేరే pattern.
- **Sharding:** high concurrency కి multiple LRU shards (lock contention తగ్గించడం).
- **Metrics:** hit/miss ratio counters.

Gotchas

- **Map leak on evict:** list నుండి remove చేసి map నుండి delete మర్చిపోతే → memory leak. `entry.key` దీనికే.
- **Get is a write:** `Get` MoveToFront చేస్తుంది (list modify) → RWMutex read lock సరిపోదు, full `Mutex` అవసరం.
- **Type assertion panic:** `elem.Value.(*entry)` – generics లేని version లో wrong type → panic. Generics దీన్ని fix చేస్తాయి.
- **Capacity 0/negative:** guard – capacity ≤ 0 అయితే error/panic.

- **Concurrent access:** lock లేకపోతే map + list races (`fatal error: concurrent map`).

Key Points

- **LRU = hash map (O(1) lookup) + doubly linked list (O(1) reorder/evict)**; front=MRU, back=LRU.
- Go: `container/list` + `map[K]*list.Element` + **generics** + `sync.Mutex` .
- `entry` **లో key store చెయ్యి** – evict సమయంలో map నుండి delete చేయడానికి (leak తప్పించడం).
- `Get` **కూడా write** (MoveToFront) → plain `Mutex` (RWMutex కాదు). Extensions: TTL, LFU, sharding.

41. Design a Rate Limiter

వివరణ

Requirements: ఒక client (user/IP/API key) ఎన్ని requests ఒక time window లో చేయవచ్చే limit చెయ్యి. Excess requests reject/throttle. Thread-safe (concurrent requests). Common algorithms:

1. **Token Bucket** – bucket లో tokens (capacity). ప్రతి request ఒక token తీసుకుంటుంది. Tokens ఒక rate తో refill అవుతాయి. Token లేకపోతే reject. **Bursts allow చేస్తుంది** (bucket full అయితే).
2. **Sliding Window** – గత N seconds లో requests count. Window slide అవుతూ ఉంటుంది. Smoother కానీ ఖరీదు.

Real-life Scenario

> **Token bucket = movie theater** analogy: hall లో 100 seats (bucket capacity). ప్రతి person ఒక seat తీసుకుంటాడు (token). Seats అయిపోతే "house full" (reject). Show అయ్యాక seats మళ్ళీ ఖాళీ అవుతాయి ఒక rate తో (refill). ఒకేసారి 100 మంది రావొచ్చు (burst) – seats ఉన్నంతవరకు.

Design decisions

Concern	Solution
Algorithm	interface `Limiter` – pluggable (Strategy)
Token bucket	tokens + lastRefill time + `sync.Mutex`
Refill	lazy (Allow() లో elapsed time బట్టి compute)
Per-client	`map[clientID]*Bucket`
Thread-safety	`sync.Mutex`

Code – Token Bucket

```
package ratelimit

import (
	"sync"
	"time"
)

// Limiter – pluggable algorithm (Strategy)
type Limiter interface {
	Allow() bool
}

// TokenBucket – capacity tokens, refillRate per second
type TokenBucket struct {
	mu         sync.Mutex
	capacity   float64
	tokens     float64   // ప్రస్తుత tokens (float – fractional refill)
	refillRate float64   // tokens per second
	lastRefill time.Time
}

func NewTokenBucket(capacity, refillPerSec float64) *TokenBucket {
	return &TokenBucket{
		capacity:   capacity,
		tokens:     capacity, // full గా start
		refillRate: refillPerSec,
		lastRefill: time.Now(),
	}
}

// Allow – token ఉంటే తీసుకుని true, లేకపోతే false. Lazy refill.
func (tb *TokenBucket) Allow() bool {
	tb.mu.Lock()
	defer tb.mu.Unlock()

	// 1. lazy refill – గత call నుండి elapsed time బట్టి tokens జోడించు
	now := time.Now()
	elapsed := now.Sub(tb.lastRefill).Seconds()
	tb.tokens += elapsed * tb.refillRate
	if tb.tokens > tb.capacity {
		tb.tokens = tb.capacity // cap
	}
	tb.lastRefill = now

	// 2. token ఉందా?
	if tb.tokens >= 1 {
		tb.tokens--
		return true // allowed
	}
	return false // rate limited
}
```

Code – Per-client limiter registry

```
// RateLimiter – per-client buckets (map). Thread-safe.
type RateLimiter struct {
	mu       sync.Mutex
	buckets  map[string]*TokenBucket
	capacity float64
	rate     float64
}

func NewRateLimiter(capacity, rate float64) *RateLimiter {
	return &RateLimiter{buckets: map[string]*TokenBucket{}, capacity: capacity,
rate: rate}
}

func (rl *RateLimiter) Allow(clientID string) bool {
	rl.mu.Lock()
	bucket, ok := rl.buckets[clientID]
	if !ok {
		bucket = NewTokenBucket(rl.capacity, rl.rate)
		rl.buckets[clientID] = bucket
	}
	rl.mu.Unlock()          // client's bucket has its own lock
	return bucket.Allow()
}

// వాడకం (HTTP middleware – Chain of Responsibility!):
// if !rl.Allow(clientIP) { http.Error(w, "429 Too Many Requests", 429); return }
```

Code – Sliding Window (sketch)

```
// SlidingWindow – గత window duration లో timestamps store, పాతవి prune
type SlidingWindow struct {
	mu       sync.Mutex
	window   time.Duration
	limit    int
	requests []time.Time // timestamps (deque)
}

func (s *SlidingWindow) Allow() bool {
	s.mu.Lock()
	defer s.mu.Unlock()
	now := time.Now()
	cutoff := now.Add(-s.window)
	// prune old timestamps (window దాటినవి)
	i := 0
	for i < len(s.requests) && s.requests[i].Before(cutoff) {
		i++
	}
	s.requests = s.requests[i:]
	if len(s.requests) >= s.limit {
		return false
	}
	s.requests = append(s.requests, now)
	return true
}
```

Algorithm comparison

Algorithm	Bursts	Memory	Accuracy	Go idiom
Token bucket	✅ allows	O(1) per client	good	lazy refill (default)
Leaky bucket	❌ smooths	O(1)	good	queue-based
Fixed window	⚠️ edge bursts	O(1)	rough (boundary spikes)	counter + reset
Sliding window	❌ smooth	O(n) timestamps	precise	deque prune

Go idiom

> - **Lazy refill** = Go idiom: background goroutine (timer) కాకుండా, `Allow()` లో elapsed time బట్టి tokens compute. Simpler, goroutine లేదు.
> - `golang.org/x/time/rate` = production token bucket (`rate.Limiter`) – interview లో scratch, production లో ఇది.
> - `Limiter` **interface** = algorithm pluggable (Strategy) – token bucket / sliding window swap.
> - **Per-client** `map` **cleanup:** idle clients buckets ని evict చేయాలి (LRU/TTL) – లేకపోతే map unbounded growth.
> - **Middleware integration:** rate limiter = HTTP middleware (Section 30 CoR).

Gotchas

- **Map unbounded growth:** per-client buckets ఎప్పుడూ create అవుతాయి, idle clients cleanup లేకపోతే memory leak. TTL/LRU eviction + background cleanup goroutine.
- **Lock granularity:** registry lock ని బుక్కెట్ Allow() మొత్తం hold చేయకు – bucket కి own lock (above). లేకపోతే contention.
- **Clock issues:** `time.Now()` monotonic (Go handles), కానీ distributed లో clock skew → distributed rate limiter (Redis) అవసరం.
- **Fractional tokens:** `float64` tokens – integer వాడితే refill precision పోతుంది.
- **Sliding window memory:** high-traffic client కి timestamps slice పెద్దది → memory. Token bucket O(1) better.
- **Distributed:** single-instance limiter multiple servers లో పని చేయదు – Redis/centralized store.

Key Points

- **Token bucket** = tokens + lazy refill (elapsed × rate); bursts allow; **O(1)** per client – default choice.
- **Sliding window** = timestamp deque, precise కానీ O(n) memory.

- Go: **lazy refill** (goroutine లేదు), `sync.Mutex`, `Limiter` interface (Strategy), production `x/time/rate`.
- **Per-client map cleanup** (TTL/LRU – unbounded growth); distributed → Redis. Middleware = CoR (Sec 30).

42. Design a Thread-safe Logger

వివరణ

Requirements: Log levels (DEBUG, INFO, WARN, ERROR). Thread-safe (multiple goroutines log). Configurable output (file/stdout/network). **Async logging** – logging shouldn't block the caller (hot path). Level filtering. Pluggable formatters.

Design: Async logger = κaller ఒక channel లోకి log message పంపుతాడు (non-blocking), ఒక background goroutine వాటిని serialize చేసి write చేస్తుంది. ఇది **producer-consumer** pattern (Section 37) + Strategy (output/format).

Real-life Scenario

> **Restaurant order counter** analogy: waiters (goroutines) orders (log messages) ని counter (channel) మీద పెట్టి వెళ్ళిపోతారు – kitchen ready అయ్యేదాకా ఆగరు (async, non-blocking). ఒక్క kitchen staff (background goroutine) orders ని one-by-one serialize చేసి process చేస్తాడు (no jumbling – thread-safe output). Rush అయితే counter (channel buffer) నిండుతుంది.

Design decisions

Concern	Solution
Levels	enum (`iota`) + filtering
Thread-safety	channel (serialize writes) OR mutex
Async (non-block)	buffered channel + background goroutine
Output pluggable	`io.Writer` (Strategy – file/stdout/net)
Graceful shutdown	`Close()` – drain channel, `sync.WaitGroup`

Code – Async, channel-based logger

```
package logger

import (
	"fmt"
	"io"
	"sync"
```

```
	"time"
)

// ---- levels ----
type Level int

const (
	DEBUG Level = iota
	INFO
	WARN
	ERROR
)

func (l Level) String() string { return [...]string{"DEBUG", "INFO", "WARN", "ERROR"}
[l] }

type logMsg struct {
	level Level
	text  string
	time  time.Time
}

// ---- Logger ----
type Logger struct {
	minLevel Level
	out      io.Writer     // pluggable output (Strategy)
	ch       chan logMsg   // async buffer
	wg       sync.WaitGroup
	once     sync.Once     // Close ఒక్కసారే
}

func New(out io.Writer, minLevel Level, bufSize int) *Logger {
	l := &Logger{
		minLevel: minLevel,
		out:      out,
		ch:       make(chan logMsg, bufSize), // buffered — non-blocking
(usually)
	}
	l.wg.Add(1)
	go l.process() // single background consumer → writes serialized (thread-
safe)
	return l
}

// process — background goroutine. అన్ని writes ఒక్క goroutine నుండి → no interleaving.
func (l *Logger) process() {
	defer l.wg.Done()
	for msg := range l.ch { // channel close అయ్యేదాకా
		fmt.Fprintf(l.out, "[%s] %s %s\n",
			msg.level, msg.time.Format("15:04:05"), msg.text)
	}
}

// log — level filter + async send
func (l *Logger) log(level Level, format string, args ...any) {
	if level < l.minLevel {
		return // filtered — cheap
	}
	msg := logMsg{level: level, text: fmt.Sprintf(format, args...), time:
```

```
time.Now()}
        select {
        case l.ch <- msg: // non-blocking send
        default:
                // buffer full — drop (or block, depending on policy)
                fmt.Fprintln(l.out, "[LOGGER] buffer full, dropped a log")
        }
}

func (l *Logger) Debug(f string, a ...any) { l.log(DEBUG, f, a...) }
func (l *Logger) Info(f string, a ...any)  { l.log(INFO, f, a...) }
func (l *Logger) Warn(f string, a ...any)  { l.log(WARN, f, a...) }
func (l *Logger) Error(f string, a ...any) { l.log(ERROR, f, a...) }

// Close — graceful shutdown: channel close → process drains remaining → wait
func (l *Logger) Close() {
        l.once.Do(func() {
                close(l.ch) // process() range ఆగుతుంది (remaining drain అయ్యాక)
                l.wg.Wait() // అన్ని logs write అయ్యేదాకా wait
        })
}
```

```
// వాడకం:
// log := logger.New(os.Stdout, logger.INFO, 1000)
// defer log.Close()  // ముఖ్యం — pending logs flush
// log.Info("server started on port %d", 8080)
// log.Debug("this is filtered")  // INFO min కాబట్టి skip
```

Async (channel) vs Sync (mutex) logger

అంశం	Async (channel)	Sync (mutex)
Caller blocks	❌ (non-blocking, hot-path safe)	✅ (write duration block)
Ordering	✅ (single consumer serializes)	✅ (lock)
Log loss risk	⚠️ (buffer full drop / crash before drain)	❌ (synchronous)
Shutdown	must drain (`Close()`)	simple
Throughput	high (batching possible)	lower (lock contention)

Go idiom

> - **Single consumer goroutine** = thread-safety without per-write mutex – అన్ని writes ఒక్క goroutine నుండి కాబట్టి interleaving ఉండదు ("share memory by communicating").
> - `io.Writer` **output** = Strategy – file, stdout, network, `bytes.Buffer` (test) ఏదైనా (accept interfaces).
> - **Buffered channel +** `select default` = non-blocking; buffer full policy (drop/block) design decision.
> - `Close()` **graceful shutdown** = critical – channel close, drain, `wg.Wait()`. లేకపోతే pending logs lost.
> - **Level filter cheap:** `if level < minLevel { return }` – filtered logs channel కి కూడా వెళ్ళవు.
> - Production: `log/slog` (Go 1.21+ structured logging), `zap`, `zerolog` – interview లో scratch.

Gotchas

- **Lost logs on crash:** async logger – buffered messages program crash అయితే lost (not yet written). Critical logs కి sync/flush.
- **Close() forgetting:** `Close()` (drain) లేకపోతే program exit అయినప్పుడు pending logs lost. `defer log.Close()`.
- **Buffer full policy:** buffer నిండితే drop చేయాలా, block చేయాలా? Drop = fast కానీ log loss; block = no loss కానీ caller slow. Explicit decision.
- **Double close panic:** channel ని రెండుసార్లు close చేస్తే panic. `sync.Once` (above).
- `time.Now()` **cost:** filtered logs లో కూడా `time.Now()` పిలిస్తే waste – filter ముందు (above సరిగ్గా చేసింది).
- **Formatting cost:** `fmt.Sprintf` filtered logs కి చేయకు – level check ముందు.

Key Points

- **Async logger** = caller channel లోకి push (non-blocking), **single background goroutine** serializes writes (thread-safe, no interleaving).
- Levels (enum + filter), `io.Writer` **output** (Strategy), buffered channel + `select default` (non-blocking).
- `Close()` **graceful drain** (`close(ch)` + `wg.Wait()`) – critical, లేకపోతే logs lost; `sync.Once` (double-close panic).
- Filter/format **before** channel send (cheap); production `log/slog` / `zap` / `zerolog`.

43. Design a Pub/Sub / Notification System

వివరణ

Requirements: Publishers events ని **topics** కి publish చేస్తారు. Subscribers topics కి subscribe అవుతారు, matching events అందుకుంటారు. Publisher కి subscribers ఎవరో తెలియనవసరం లేదు (decoupled). Multiple subscribers per topic. Concurrent-safe. Subscribe/unsubscribe dynamic.

ఇది **Observer pattern** (Section 25) యొక్క scaled-up, topic-based, channel-based Go implementation.

Real-life Scenario

> **Radio station** analogy: FM station (publisher) ఒక frequency (topic) మీద broadcast చేస్తుంది. ఎన్ని radios (subscribers) ఆ frequency కి tune అయ్యాయో station కి తెలియదు – అది broadcast చేస్తుంది, tuned radios అందుకుంటాయి. కొత్త radio tune అవ్వొచ్చు, off చేయొచ్చు (subscribe/unsubscribe) – station కి అనవసరం.

Design decisions

Concern	Solution
Topics	`map[topic][]subscriber`
Subscriber delivery	channels (async, per-subscriber)
Decoupling	publisher topics కి, subscribers channels
Concurrent-safe	`sync.RWMutex` (read-heavy: publish)
Slow subscriber	buffered channel + `select default` (drop)
Cleanup	`Unsubscribe` closes channel, removes

Code – Channel-based Pub/Sub

```
package pubsub

import (
	"sync"
)

type Message struct {
	Topic string
	Data  any
}

type Broker struct {
	mu   sync.RWMutex
	subs map[string][]chan Message // topic → subscriber channels
```

```
}

func NewBroker() *Broker {
	return &Broker{subs: make(map[string][]chan Message)}
}

// Subscribe — topic కి కొత్త channel, subscriber దీన్ని range చేస్తాడు
func (b *Broker) Subscribe(topic string, bufSize int) <-chan Message {
	b.mu.Lock()
	defer b.mu.Unlock()
	ch := make(chan Message, bufSize) // buffered — slow subscriber అందరినీ block
చేయకూడదు
	b.subs[topic] = append(b.subs[topic], ch)
	return ch
}

// Publish — topic subscribers అందరికీ (async, non-blocking)
func (b *Broker) Publish(topic string, data any) {
	b.mu.RLock() // read lock — publish read-heavy, concurrent publishes OK
	defer b.mu.RUnlock()
	msg := Message{Topic: topic, Data: data}
	for _, ch := range b.subs[topic] {
		select {
		case ch <- msg: // non-blocking send
		default:
			// subscriber buffer full → drop (backpressure policy)
			// alternative: block, or grow buffer, or metrics
		}
	}
}

// Unsubscribe — channel remove + close (cleanup, leak తప్పించడం)
func (b *Broker) Unsubscribe(topic string, target <-chan Message) {
	b.mu.Lock()
	defer b.mu.Unlock()
	subs := b.subs[topic]
	for i, ch := range subs {
		if ch == target {
			close(ch)                                    // subscriber's
range ఆగుతుంది
			b.subs[topic] = append(subs[:i], subs[i+1:]...) // remove
			break
		}
	}
}

// Close — అన్ని channels close (shutdown)
func (b *Broker) Close() {
	b.mu.Lock()
	defer b.mu.Unlock()
	for topic, subs := range b.subs {
		for _, ch := range subs {
			close(ch)
		}
		delete(b.subs, topic)
	}
}
```

```
// వాడకం:
// broker := pubsub.NewBroker()
// orders := broker.Subscribe("orders", 10)
// go func() { for msg := range orders { fmt.Println("got:", msg.Data) } }()
// broker.Publish("orders", "Order #123")  // subscriber అందుకుంటాడు
```

Notification system (multi-channel) – Strategy + Observer combined

```
// notification = వేర్వేరు channels (email/SMS/push) — Strategy
type NotificationChannel interface {
	Send(userID, msg string) error
}
type EmailChannel struct{}
func (EmailChannel) Send(u, m string) error { /* ... */ return nil }
type SMSChannel struct{}
func (SMSChannel) Send(u, m string) error { /* ... */ return nil }

// event → subscribed channels అందరికీ (Observer + Strategy)
type Notifier struct {
	channels []NotificationChannel // user preference బట్టి
}
func (n *Notifier) Notify(userID, msg string) {
	for _, ch := range n.channels {
		go ch.Send(userID, msg) // concurrent delivery
	}
}
```

Pub/Sub vs direct Observer

అంశం	Simple Observer (Sec 25)	Pub/Sub broker (this)
Topics	❌ (single subject)	✅ (topic-based routing)
Decoupling	subject knows observers	broker mediates (fully decoupled)
Delivery	sync callback	async channels
Scale	in-object	many topics, many subs

Go idiom

> - **Channels = Go-native pub/sub** — subscriber ఒక channel range చేస్తాడు, broker publish చేస్తుంది.
> - `RWMutex` **:** publish (read subs) frequent, subscribe/unsubscribe (write) rare → `RWMutex` concurrent publishes allow చేస్తుంది.
> - **Buffered channel +** `select default` = slow subscriber మిగతా వాళ్ళను block చేయకుండా (drop policy). Alternative: per-subscriber goroutine, unbounded queue (danger).
> - **Unsubscribe closes channel** — subscriber's `range` cleanly ఆగుతుంది; leak తప్పించడం.
> - **Distributed pub/sub:** in-memory broker single-process. Cross-service → NATS, Kafka, Redis Pub/Sub, Google Pub/Sub.

Gotchas

- **Slow subscriber blocks publisher:** unbuffered channel లో ఒక slow subscriber మొత్తం publish ని block చేస్తుంది. Buffered + `select default` (drop) లేదా per-subscriber goroutine.
- **Send on closed channel panic:** unsubscribe/close తర్వాత publish → panic. Lock ordering + close carefully.
- **Goroutine/channel leak:** unsubscribe మర్చిపోతే channels + subscriber goroutines leak. Explicit cleanup.
- **Message loss:** drop policy → messages lost (buffer full). At-least-once కావాలంటే persistent queue (Kafka).
- **Ordering:** single topic single subscriber = ordered; multiple subscribers/goroutines = no cross-order guarantee.
- **Deadlock:** publish లో lock hold చేసి, subscriber callback తిరిగి broker call చేస్తే → deadlock. Callbacks lock బయట.

Key Points

- **Pub/Sub** = topic-based, fully decoupled Observer (broker mediates); publisher subscribers ని తెలియదు.
- Go: **channels per subscriber**, `map[topic][]chan`, `RWMutex` (read-heavy publish), buffered + `select default` (slow subscriber drop).
- **Unsubscribe closes channel** (subscriber `range` ఆగుతుంది, leak తప్పించడం); send-on-closed panic జాగ్రత్త.
- In-memory = single process; distributed → Kafka/NATS/Redis. Notification = Pub/Sub + Strategy (channels).

44. Design a Vending Machine (State Pattern)

వివరణ

Requirements: States – Idle (no money), HasMoney (money inserted), Dispensing, OutOfStock. Actions – InsertMoney, SelectProduct, Dispense, Refund. Behavior state బట్టి మారుతుంది (Idle లో select చేయలేం; HasMoney లో మళ్ళీ money insert చేయకూడదు). ఇది classic **State pattern** (Section 27) use case.

Design: `State` interface with actions, ఒక్కో state ఒక struct, machine ప్రస్తుత state కి delegate చేస్తుంది. Invalid actions ఆ state లో reject అవుతాయి. Transitions states మారుస్తాయి.

Real-life Scenario

> **నిజమైన vending machine** analogy: డబ్బు వేయకముందు button నొక్కితే ఏమీ జరగదు (Idle state). డబ్బు వేశాక (HasMoney) product select చేయవచ్చు. Select చేశాక అది dispense అవుతుంది (Dispensing), తర్వాత change ఇచ్చి Idle కి తిరిగి వెళ్తుంది. Stock అయిపోతే (OutOfStock) డబ్బు తీసుకోదు. ప్రతి state లో machine behavior వేరు.

Design decisions

Concern	Solution
State-based behavior	**State pattern** (interface + structs)
Transitions	states set next state on machine
Invalid actions	each state rejects inapplicable actions
Inventory	`map[product]count`
Shared context	`Machine` holds state + inventory + balance

Code – State pattern vending machine

```
package vending

import "fmt"

// ---- State interface – అన్ని actions (ఒక్కో state ఏది valid అనేది decide) ----
type State interface {
	InsertMoney(amount int)
	SelectProduct(name string)
	Dispense()
	Name() string
}

// ---- Machine (context) – state + shared data ----
type Machine struct {
	state     State
```

```
        inventory map[string]int
        prices    map[string]int
        balance   int
        selected  string

        idle       State
        hasMoney   State
        dispensing State
}

func NewMachine(inventory, prices map[string]int) *Machine {
        m := &Machine{inventory: inventory, prices: prices}
        m.idle = &IdleState{m}
        m.hasMoney = &HasMoneyState{m}
        m.dispensing = &DispensingState{m}
        m.state = m.idle // start Idle
        return m
}

// delegate methods — ప్రస్తుత state కి forward
func (m *Machine) InsertMoney(a int)     { m.state.InsertMoney(a) }
func (m *Machine) SelectProduct(n string) { m.state.SelectProduct(n) }
func (m *Machine) Dispense()             { m.state.Dispense() }
func (m *Machine) setState(s State)      { m.state = s }

// ---- IdleState: money insert మాత్రమే valid ----
type IdleState struct{ m *Machine }
func (s *IdleState) Name() string { return "Idle" }
func (s *IdleState) InsertMoney(a int) {
        s.m.balance += a
        fmt.Printf("₹%d inserted (balance ₹%d)\n", a, s.m.balance)
        s.m.setState(s.m.hasMoney) // transition
}
func (s *IdleState) SelectProduct(string) { fmt.Println("ముందు డబ్బు వేయండి") }
func (s *IdleState) Dispense()            { fmt.Println("ముందు select చేయండి") }

// ---- HasMoneyState: select valid ----
type HasMoneyState struct{ m *Machine }
func (s *HasMoneyState) Name() string { return "HasMoney" }
func (s *HasMoneyState) InsertMoney(a int) {
        s.m.balance += a
        fmt.Printf("₹%d added (balance ₹%d)\n", a, s.m.balance)
}
func (s *HasMoneyState) SelectProduct(name string) {
        if s.m.inventory[name] == 0 {
                fmt.Println("out of stock:", name)
                return
        }
        if s.m.balance < s.m.prices[name] {
                fmt.Printf("insufficient — %s needs ₹%d, balance ₹%d\n", name,
s.m.prices[name], s.m.balance)
                return
        }
        s.m.selected = name
        s.m.setState(s.m.dispensing) // transition
        s.m.Dispense()               // auto-dispense
}
func (s *HasMoneyState) Dispense() { fmt.Println("ముందు product select చేయండి") }
```

```
// ---- DispensingState: dispense valid ----
type DispensingState struct{ m *Machine }
func (s *DispensingState) Name() string          { return "Dispensing" }
func (s *DispensingState) InsertMoney(int)        { fmt.Println("dispensing జరుగుతోంది,
ఆగండి") }
func (s *DispensingState) SelectProduct(string)   { fmt.Println("dispensing జరుగుతోంది,
ఆగండి") }
func (s *DispensingState) Dispense() {
	name := s.m.selected
	s.m.inventory[name]--
	change := s.m.balance - s.m.prices[name]
	fmt.Printf("Dispensed %s. Change: ₹%d\n", name, change)
	// reset + transition back to Idle
	s.m.balance = 0
	s.m.selected = ""
	s.m.setState(s.m.idle)
}

func demo() {
	m := NewMachine(
		map[string]int{"Coke": 2, "Water": 5},
		map[string]int{"Coke": 30, "Water": 20},
	)
	m.SelectProduct("Coke") // Idle → "ముందు డబ్బు వేయండి"
	m.InsertMoney(50)       // Idle → HasMoney
	m.SelectProduct("Coke") // HasMoney → Dispensing → dispense (change ₹20) →
Idle
}
```

State transition table

Current state	InsertMoney	SelectProduct	Dispense
Idle	→ HasMoney	reject	reject
HasMoney	add balance	valid → Dispensing	reject
Dispensing	reject (busy)	reject (busy)	dispense → Idle

Go idiom

> - **State structs hold `*Machine` back-reference** – shared context (inventory, balance) access + transitions (`m.setState`).
> - **States pre-created in `NewMachine`** – ప్రతి transition లో కొత్త state allocate చేయకుండా (reuse).
> - **Interface = all actions** – ప్రతి state అన్ని actions implement చేస్తుంది, invalid వాటిని reject చేస్తుంది (explicit, compile-safe).
> - **Alternative (simple FSM):** ఇంత rich behavior లేకపోతే enum + transition table (Section 27). Vending machine per-state behavior rich కాబట్టి interface-based worth.
> - **Thread-safety:** real vending machine single-user, కానీ concurrent API అయితే `sync.Mutex` on `Machine`.

Gotchas

- **Invalid action handling:** ప్రతి state invalid actions ని gracefully reject చేయాలి (message/error), silent ignore/panic కాదు.
- **State explosion:** చాలా states × actions = పెద్ద interface + many structs. Hierarchical states / table hybrid.
- **Shared context race:** concurrent access అయితే machine state + inventory race – `sync.Mutex`.
- **Transition bugs:** state మార్చడం మర్చిపోతే stuck. Transitions ని clearly test చెయ్యి (table).
- **Terminal/error states:** OutOfStock, MaintenanceMode లాంటివి add చేస్తే transitions update.
- **Back-reference cycle:** state → machine → state – Go GC handles, కానీ design గుర్తుంచుకో.

Key Points

- **Vending machine = classic State pattern** – behavior state (Idle/HasMoney/Dispensing) బట్టి మారుతుంది.
- Go: `State` interface (all actions) + state structs holding **`*Machine` back-reference** (context + transitions); states pre-created.
- Each state **rejects invalid actions** gracefully; transitions via `m.setState()`.
- Rich per-state behavior → interface-based; simple → enum+table (Sec 27). Concurrent → `sync.Mutex`.

45. LLD Interview Approach in Go + Checklist + Memory Tips

వివరణ

LLD interview లో "Design X in Go" అడిగినప్పుడు – panic అవ్వకుండా, structured approach follow చెయ్యి. ముఖ్యం: **idiomatic Go** చూపించు (Java-in-Go కాదు), **think aloud** చెయ్యి, **trade-offs** మాట్లాడు.

Interviewer perfect design కంటే నీ **reasoning** ని చూస్తాడు.

5-step LLD framework (Go flavor)

```
1. CLARIFY (2–3 నిమిషాలు)
   – Scope, scale, features. "Parking lot — bikes కూడానా? Payment? Multiple floors?
Concurrent?"
   – Concurrency requirement అడుగు (Go లో ఇది design ని మారుస్తుంది)

2. ENTITIES + INTERFACES (nouns → structs, verbs → interfaces)
   – ముఖ్య structs list చెయ్యి (data)
   – Behaviors ని small interfaces గా (accept interfaces)
   – "ఏది వేరియబుల్?" → interface (Strategy candidate)

3. RELATIONSHIPS + PATTERNS
   – Composition (embedding/fields) — inheritance కాదు
   – ఎక్కడ Strategy (func/interface)? State? Decorator (middleware)?
   – SOLID check — ISP (small interfaces), DIP (inject dependencies)

4. CODE (idiomatic Go)
   – Constructors (NewX), functional options అవసరమైతే
   – DI via constructor; interfaces consumer-side
   – Concurrency: mutex vs channels — justify

5. EDGE CASES + CONCURRENCY
   – Full, empty, invalid input, nil
   – Race conditions (go run -race mindset), goroutine leaks
   – Error handling (return error, not panic)
```

Go-specific interview checklist

✅ చూపించాల్సినవి (idiomatic Go)	❌ తప్పించాల్సినవి (Java-in-Go smell)
Small interfaces (1-3 methods)	Fat "god" interfaces
Accept interfaces, return structs	Return interfaces everywhere
Composition (embedding/fields)	"inheritance" hacks / deep embedding
`func` types for single-behavior strategy	interface for every single method
Constructor injection (DI)	global singletons / hidden `init()` deps
Functional options for config	10-arg constructors / setter chains
`sync.Mutex` / channels justified	shared state without synchronization
Errors as values (`return err`)	`panic` for control flow
Interfaces consumer-side	producer-side abstract factories
Concrete first, abstract when needed	premature interfaces (YAGNI violation)

Pattern → Go idiom quick reference (cheat sheet)

Classic pattern	Go idiom (ఇది చెప్పు)
Singleton	`sync.Once` (or prefer DI)
Factory	`NewX()` returning interface
Builder	**Functional options** (`...Option`)
Prototype	value copy + `Clone()` (deep copy ref fields)
Object Pool	`sync.Pool` / buffered channel
Adapter	wrapper struct / `http.HandlerFunc`
Decorator	`io.Reader/Writer` wrap / middleware `func(next)`
Proxy	interface wrapper + `sync.Once` (lazy)
Facade	package with clean exported API
Composite	interface + `[]Component`
Bridge	struct holds implementation interface
Flyweight	map cache of shared immutable objects
Strategy	`func` **type** (single behavior)
Observer	channels (async) / callbacks (sync)
Command	`func()` / interface (undo)
State	interface + state structs (or enum+table)
Template Method	interface injection (**not** embedding)
Iterator	`range` / **range-over-func** (Go 1.23)
Chain of Responsibility	middleware `func(next Handler) Handler`
Mediator	central struct / channel hub
Memento	snapshot struct (unexported fields)
Visitor	**type switch** (visitor awkward in Go)
Interpreter	interface + expression tree (small DSLs)

Memory Tips – Real-life analogies (మర్చిపోకూడదంటే)

Concept / Pattern	గుర్తుంచుకోవడానికి
struct vs interface	ఇటుకలు (struct) vs తలుపు spec (interface)

Concept / Pattern	**గుర్తుంచుకోవడానికి**
Composition (embed)	skills నేర్చుకోవడం (has-a, not is-a)
Implicit interface	"code రాయగలవా?" – certificate అవసరం లేదు
Accept interfaces	power socket (ఏ plug అయినా)
Singleton	ఒకే CEO (`sync.Once` = ఒక్కసారే appoint)
Factory	restaurant kitchen (order → dish)
Functional options	pizza toppings (కావలసినవి మాత్రమే)
Prototype	resume copy (deep copy చేయకపోతే leak)
Object Pool	cricket ball bucket (`sync.Pool`)
Adapter	travel plug adapter
Decorator	coffee + milk + sugar (io.Reader wrap)
Proxy	credit card (account కి ప్రతినిధి)
Facade	car start button
Composite	folder లో folder (tree)
Bridge	remote × device (mix & match)
Flyweight	అడవిలో చెట్లు (shared type)
Strategy	Maps route (`func` type)
Observer	radio station (channels)
Command	order slip (`func()` / undo)
State	vending machine / traffic signal
Template Method	tea/coffee recipe (interface injection)
Iterator	TV remote next (`range`)
Chain of Resp.	support escalation (middleware)
Mediator	ATC tower (channel hub)
Memento	game save point (snapshot)
Visitor	tax auditor (Go: type switch)
Worker pool	kitchen with 5 cooks (bounded)
Pub/Sub	radio broadcast (topics + channels)

Common Mistakes in Go LLD (Interview & Production)

❌ Mistake	✅ Fix
Java patterns ని blindly port చేయడం	Go idiom ఆలోచించు (Strategy=func, no factories everywhere)
ప్రతి struct కి interface (premature)	Concrete first, extract interface when needed
Fat "god" interfaces	Small interfaces (ISP – `io.Reader` style)
Return interfaces everywhere	Accept interfaces, **return structs**
Inheritance simulate చేయడానికి deep embed	Composition + interface satisfaction
Global singletons everywhere	Dependency injection (constructor)
10-arg constructors	Functional options
Shallow copy → reference field leak	Deep copy (`Clone()`) for slice/map/pointer
Shared state without lock	`sync.Mutex` / channels (`go run -race`)
Goroutine leaks (no close/context)	`defer close` , `context.Context` , buffered ch
`panic` for normal errors	Errors as values (`return err`)
`err == ErrX` on wrapped errors	`errors.Is` / `errors.As`
Typed nil returned as error	`return nil` explicitly (nil interface trap)
Template Method via embedding (broken)	Interface injection (no virtual dispatch)
Visitor pattern boilerplate	Type switch (simpler in Go)
Value receiver mutation (silent no-op)	Pointer receiver for mutation
Mutex struct copied by value	Pointer receiver / don't copy (`go vet`)

3 Categories గుర్తుంచుకో (23 GoF)

Category	ఏం చేస్తాయి	Go లో ముఖ్య మార్పు
Creational (5)	Objects పుట్టడం	`NewX` + functional options (no `new`)
Structural (7)	Objects కలవడం	embedding + interfaces (io wrapping)
Behavioral (11)	Objects మాట్లాడటం	`func` types + channels (Strategy/Observer)

What to say vs not say (interview)

❌ చెప్పకు	✅ చెప్పు
నేరుగా code మొదలుపెట్టడం	"ముందు requirements + concurrency clarify చేస్తాను"
"ఇది Java లో ఇలా..."	"Go లో ఇది idiomatic గా ఇలా – func type/composition"
అన్ని patterns కూరడం	"ఇక్కడ Strategy సరిపోతుంది ఎందుకంటే..."
ఒకే perfect solution	Trade-offs (mutex vs channel, interface vs concrete)
Silent coding	Think aloud – reasoning చెప్పు

Key Points

- **5 steps:** Clarify (+ concurrency) → Entities/Interfaces → Relationships/Patterns → Code → Edge cases/Concurrency.
- **Idiomatic Go చూపించు:** small interfaces, accept interfaces/return structs, composition, func-type strategies, DI, functional options, errors as values.
- **Java-in-Go smells తప్పించు:** premature interfaces, god interfaces, global singletons, `panic` for errors, inheritance hacks.
- **Think aloud + trade-offs** – interviewer reasoning ని చూస్తాడు, perfect design ని కాదు.

ముగింపు (Conclusion)

> **గుర్తుంచుకో – Go లో LLD యొక్క సారాంశం:**
>
> - Go లో **class లేదు, inheritance లేదు** – `struct` (data) + `interface` (behavior) + `composition` (reuse). ఇదే మొదటి, ముఖ్యమైన mindset shift.
> - **"Accept interfaces, return structs"** + **small interfaces (ISP)** + **interfaces belong to the consumer** – ఈ మూడు Go design proverbs గుండె.
> - Classic GoF patterns Go లో **simpler / different / sometimes unnecessary** అవుతాయి: Strategy = `func`, Singleton = `sync.Once`, Builder = functional options, Decorator = `io` wrapping, Iterator = `range`, Chain of Responsibility = middleware. కొన్ని (Template Method, Visitor) Go లో awkward – వాటిని force చేయకు.
> - **Concurrency = first-class LLD building block** – worker pool, pub-sub, pipeline. "Share memory by communicating."
> - **Errors are values** – `return err`, wrap with `%w`, `errors.Is/As`. `panic` control flow కి కాదు.
> - **Java-in-Go రాయకు.** "Concrete first, abstract when needed." Premature abstraction, god interfaces, global singletons = Go smells.
> - ప్రతి pattern కి **real-life analogy** గుర్తుపెట్టుకో – అప్పుడు మర్చిపోలేవు!

ఈ document + `GO_Telugu.md` (language basics) + `HLD_Go_Telugu.md` (high-level design) కలిస్తే – **Go language పునాది నుండి LLD నైపుణ్యం వరకు** పూర్తి తెలుగు reference. SDE2 / SSE interviews మరియు production Go design కి ఇది నీ companion.

అన్ని 45 topics పూర్తి – Go LLD mastery నీ చేతిలో. 🚀